

**KLASIFIKASI KELOMPOK USIA BERDASARKAN CITRA WAJAH
MENGUNAKAN CONVOLUTIONAL NEURAL NETWORK
DENGAN OPTIMASI PARTICLE SWARM OPTIMIZATION**

TUGAS AKHIR



Disusun Oleh :
ANUGRAHA GALIH SAPUTRA
123220119

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

**KLASIFIKASI KELOMPOK USIA BERDASARKAN CITRA WAJAH
MENGUNAKAN CONVOLUTIONAL NEURAL NETWORK
DENGAN OPTIMASI PARTICLE SWARM OPTIMIZATION**

TUGAS AKHIR

Tugas Akhir ini sebagai salah satu syarat untuk memperoleh gelar sarjana Informatika
Universitas Pembangunan Nasional “Veteran” Yogyakarta



Disusun Oleh :
ANUGRAHA GALIH SAPUTRA
123220119

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN”
YOGYAKARTA
2026**

HALAMAN PENGESAHAN PEMBIMBING

KLASIFIKASI KELOMPOK USIA BERDASARKAN CITRA WAJAH MENGUNAKAN CONVOLUTIONAL NEURAL NETWORK DENGAN OPTIMASI PARTICLE SWARM OPTIMIZATION

Disusun Oleh:

Anugraha Galih Saputra

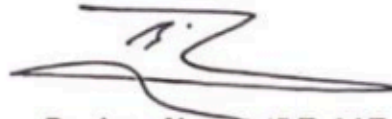
123220119

Telah diuji dan dinyatakan lulus oleh pembimbing

pada tanggal: *11 Mei 2020*

Menyetujui,

Pembimbing

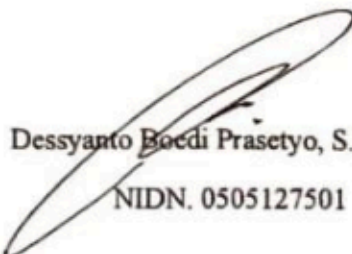


Bambang Yuwono, S.T., M.T.

NIDN. 0512027401

Mengetahui,

Koordinator Program Studi



Dessyanto Boedi Prasetyo, S.T., M.T.

NIDN. 0505127501

HALAMAN PENGESAHAN PENGUJI

KLASIFIKASI KELOMPOK USIA BERDASARKAN CITRA WAJAH MENGUNAKAN CONVOLUTIONAL NEURAL NETWORK DENGAN OPTIMASI PARTICLE SWARM OPTIMIZATION

Disusun Oleh:

Anugraha Galih Saputra

123220119

Telah diuji dan dinyatakan lulus oleh penguji
pada tanggal: 11 Mei 2026

Menyetujui,

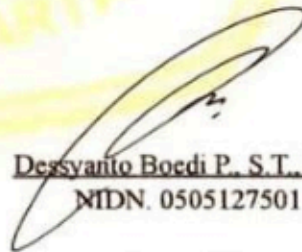
Penguji 1



Bambang Yuwono, S.T., M.T.

NIDN. 0512027401

Penguji 2



Dessyanto Boedi P., S.T., M.T.

NIDN. 0505127501

Penguji 3



Budi Santosa, S.Si., M.T.

NIDN. 0510097001

Penguji 4



Bagus Muhammad Akbar, S.ST., M.Kom.

NIDN. 0001088905

SURAT PERNYATAAN KARYA ASLI TUGAS AKHIR

Sebagai mahasiswa Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional “Veteran” Yogyakarta, yang bertanda tangan di bawah ini, saya:

Nama : Anugraha Galih Saputra
NIM : 123220119

Menyatakan bahwa karya ilmiah saya yang berjudul:

**KLASIFIKASI KELOMPOK USIA BERDASARKAN CITRA WAJAH
MENGUNAKAN CONVOLUTIONAL NEURAL NETWORK DENGAN
OPTIMASI PARTICLE SWARM OPTIMIZATION**

Merupakan karya asli saya dan belum pernah dipublikasikan dimanapun. Apabila di kemudian hari, karya saya disinyalir bukan merupakan karya asli saya, maka saya bersedia menerima konsekuensi apa pun yang diberikan Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional “Veteran” Yogyakarta kepada saya.

Demikian surat pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Yogyakarta
Pada Tanggal : 13 Desember 2025

Yang Menyatakan



Anugraha Galih Saputra
NIM. 123220119

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini:

Nama : Anugraha Galih Saputra
NIM : 123220119
Prodi : Informatika

Dengan ini saya menyatakan bahwa judul Tugas Akhir

**KLASIFIKASI KELOMPOK USIA BERDASARKAN CITRA WAJAH
MENGUNAKAN CONVOLUTIONAL NEURAL NETWORK DENGAN
OPTIMASI PARTICLE SWARM OPTIMIZATION**

adalah hasil kerja saya dan benar bebas dari plagiasi kecuali cuplikan serta ringkasan yang terdapat di dalamnya telah saya jelaskan sumbernya (Sitasi) dengan jelas. Apabila pernyataan ini terbukti tidak benar maka saya bersedia menerima sanksi sesuai peraturan Mendiknas RI No 17 Tahun 2010 dan Peraturan Perundang-undangan yang berlaku.

Demikian surat pernyataan ini saya buat dengan penuh tanggung jawab.

Dibuat di : Yogyakarta
Pada Tanggal : 13 Desember 2025

Yang Menyatakan



Anugraha Galih Saputra
NIM. 123220119

ABSTRAK

Klasifikasi kelompok usia berdasarkan citra wajah merupakan tantangan dalam visi komputer akibat kemiripan karakteristik visual antar kelompok usia, terutama pada fase transisi. Pendekatan Convolutional Neural Network (CNN) terbukti efektif mengekstraksi fitur spasial wajah, namun performa model ini sangat bergantung pada pemilihan hyperparameter yang sering kali tidak optimal jika disetel secara manual. Penelitian ini menerapkan algoritma Particle Swarm Optimization (PSO) untuk mengoptimasi hyperparameter CNN guna meningkatkan performa klasifikasi usia serta mengimplementasikan model ke dalam aplikasi deteksi usia berbasis realtime.

Metode penelitian menggunakan UTKFace Cropped Dataset yang berisi 23.708 citra wajah. Evaluasi kinerja model dilakukan melalui dua skenario pengujian. Skenario pertama mengklasifikasikan data ke dalam empat kelas usia, yaitu Child (0–12 tahun), Teenager (13–18 tahun), Adult (19–40 tahun), dan Old (di atas 40 tahun). Skenario kedua mengklasifikasikan data ke dalam lima kelas usia, yaitu Child, Teenager, Adult, Middle Age, dan Old. Tahapan pra-pemrosesan meliputi pengubahan ukuran citra menjadi 150×150 piksel, normalisasi nilai piksel, serta augmentasi data secara online untuk mencegah overfitting. Algoritma PSO ditugaskan mencari kombinasi optimal pada empat parameter utama, yaitu learning rate, dropout CNN, dropout dense, dan jumlah dense unit. Model hasil pelatihan kemudian diimplementasikan ke dalam aplikasi berbasis Streamlit untuk melakukan pengujian citra statis dan deteksi usia realtime menggunakan webcam.

Hasil penelitian membuktikan bahwa PSO berhasil meningkatkan performa klasifikasi dibandingkan model CNN baseline pada kedua skenario. Pada skenario pertama (empat kelas), tingkat akurasi meningkat dari 83,87% pada model baseline menjadi 83,99% setelah dioptimasi dengan PSO, dengan F1-Score mencapai 74,88%. Pada skenario kedua (lima kelas), tingkat akurasi model meningkat dari 79,20% menjadi 80,34%, dengan metrik precision mencapai 73,42%, recall sebesar 67,88%, dan F1-Score sebesar 69,51%. Meskipun peningkatan akurasi yang diperoleh relatif kecil, model hasil optimasi menunjukkan performa evaluasi yang lebih baik dibandingkan model baseline dan berhasil diimplementasikan secara fungsional ke dalam aplikasi deteksi usia realtime. Secara keseluruhan, integrasi PSO mampu memberikan fine-tuning terhadap hyperparameter CNN sehingga menghasilkan model yang lebih optimal, meskipun penanganan terhadap ketidakseimbangan data pada kelas minoritas, khususnya kelas teenager, masih menjadi aspek penting untuk pengembangan penelitian mendatang.

Kata Kunci: Klasifikasi Kelompok Usia, Convolutional Neural Network, Particle Swarm Optimization, Computer Vision, Citra Wajah, Hyperparameter Optimization, Realtime Detection

ABSTRACT

Age group classification based on facial images is a challenge in computer vision due to the similarity of visual characteristics between age groups, particularly during transitional phases. The Convolutional Neural Network (CNN) approach has been proven effective in extracting spatial features from facial images. However, the performance of this model highly depends on the selection of hyperparameters, which are often not optimal when tuned manually. This study applies the Particle Swarm Optimization (PSO) algorithm to optimize CNN hyperparameters in order to improve age classification performance and implement the model into a real-time age detection application.

This study used the UTKFace Cropped Dataset, which consists of 23,708 facial images. Model performance evaluation was conducted through two testing scenarios. The first scenario classified the data into four age classes: Child (0–12 years), Teenager (13–18 years), Adult (19–40 years), and Old (above 40 years). The second scenario classified the data into five age classes: Child, Teenager, Adult, Middle Age, and Old. The preprocessing stages included resizing images to 150×150 pixels, normalizing pixel values, and applying online data augmentation to prevent overfitting. The PSO algorithm was assigned to search for the optimal combination of four main parameters, namely learning rate, CNN dropout, dense dropout, and the number of dense units. The trained model was then implemented into a Streamlit-based application to test static images and perform real-time age detection using a webcam.

The results show that PSO successfully improved classification performance compared to the baseline CNN model in both scenarios. In the first scenario with four classes, the accuracy increased from 83.87% in the baseline model to 83.99% after optimization using PSO, with an F1-score of 74.88%. In the second scenario with five classes, the model accuracy increased from 79.20% to 80.34%, with precision reaching 73.42%, recall reaching 67.88%, and an F1-score of 69.51%. Although the improvement in accuracy was relatively small, the optimized model demonstrated better evaluation performance than the baseline model and was successfully implemented functionally into a real-time age detection application. Overall, the integration of PSO was able to fine-tune CNN hyperparameters, resulting in a more optimal model. However, handling data imbalance in minority classes, particularly the teenager class, remains an important aspect for future research development.

Keywords: Age Group Classification, Convolutional Neural Network, Particle Swarm Optimization, Computer Vision, Facial Image, Hyperparameter Optimization, Real-Time Detection

KATA PENGANTAR

Segala puji dan syukur penulis panjatkan ke hadirat Allah SWT atas limpahan rahmat, karunia, serta kemudahan yang telah diberikan, sehingga penulis dapat menyelesaikan Tugas Akhir yang berjudul “Klasifikasi Kelompok Usia Berdasarkan Citra Wajah Menggunakan Convolutional Neural Network dengan Optimasi Particle Swarm Optimization”. Tugas Akhir ini disusun sebagai salah satu persyaratan untuk memperoleh gelar Sarjana Informatika pada Program Studi Informatika, Jurusan Informatika, Fakultas Teknik Industri, Universitas Pembangunan Nasional “Veteran” Yogyakarta.

Penulis menyadari bahwa penyusunan Tugas Akhir ini tidak dapat terselesaikan tanpa adanya bantuan, arahan, dukungan, dan doa dari berbagai pihak. Oleh karena itu, dengan penuh rasa hormat, penulis menyampaikan ucapan terima kasih kepada:

1. Allah SWT yang telah memberikan rahmat, kesehatan, kekuatan, dan kemudahan kepada penulis sehingga Tugas Akhir ini dapat diselesaikan dengan baik.
2. Bapak, Ibu, dan keluarga penulis yang selalu memberikan doa, kasih sayang, semangat, serta dukungan moral maupun material selama masa perkuliahan hingga proses penyusunan Tugas Akhir ini.
3. Bapak Bambang Yuwono, S.T., M.T. selaku dosen pembimbing yang telah meluangkan waktu untuk memberikan bimbingan, arahan, masukan, serta motivasi dengan penuh kesabaran selama proses penyusunan Tugas Akhir ini.
4. Bapak Dessyanto Boedi P., S.T., M.T., Bapak Bagus Muhammad Akbar, S.S.T., M.Kom., dan Bapak Budi Santosa, S.Si., M.T. selaku dosen penguji yang telah memberikan kritik, saran, dan masukan yang bermanfaat dalam penyempurnaan Tugas Akhir ini.
5. Bapak Mangaras Yanu Florestiyanto, S.T., M.Eng. selaku dosen wali yang telah memberikan arahan dan bimbingan selama penulis menempuh pendidikan di Program Studi Informatika Universitas Pembangunan Nasional “Veteran” Yogyakarta.
6. Rekan-rekan seperjuangan Program Studi Informatika angkatan 2022 yang telah menjadi teman berdiskusi, berbagi pengalaman, serta memberikan dukungan dan semangat selama masa perkuliahan hingga penyelesaian Tugas Akhir ini.
7. Seluruh pihak yang telah membantu penulis dalam proses penyusunan Tugas Akhir ini, yang tidak dapat disebutkan satu per satu.

Penulis menyadari bahwa Tugas Akhir ini masih memiliki keterbatasan, baik dalam aspek penyusunan, pembahasan, maupun hasil penelitian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun sebagai bahan perbaikan di masa mendatang. Penulis berharap Tugas Akhir ini dapat memberikan manfaat, khususnya dalam pengembangan metode klasifikasi kelompok usia berdasarkan citra wajah menggunakan Convolutional Neural Network dan optimasi Particle Swarm Optimization, serta dapat menjadi referensi bagi penelitian selanjutnya di bidang computer vision dan deep learning.

Yogyakarta, 13 Desember 2025

DAFTAR ISI

HALAMAN PENGESAHAN PEMBIMBING	iii
HALAMAN PENGESAHAN PENGUJI.....	iv
SURAT PERNYATAAN KARYA ASLI TUGAS AKHIR.....	v
PERNYATAAN BEBAS PLAGIASI	vi
DAFTAR ISI	x
DAFTAR TABEL	xii
DAFTAR GAMBAR.....	xiv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah.....	3
1.4 Tujuan Penelitian.....	3
1.5 Manfaat Penelitian.....	4
1.6 Tahapan Penelitian	4
1.7 Sistematika Penulisan.....	6
BAB II TINJAUAN LITERATUR	8
2.1 Citra Wajah.....	8
2.2 Klasifikasi Kelompok Usia.....	8
2.3 <i>Deep Learning</i>	9
2.4 Pengolahan Citra Digital	9
2.5 <i>Preprocessing</i>	10
2.6 <i>Convolutional Neural Network (CNN)</i>	13
2.7 <i>Hyperparameter</i>	21
2.8 <i>Confusion Matrix</i>	22
2.9 Implementasi Model dalam Sistem Realtime	24
2.10 Penelitian Sebelumnya	24
BAB III METODOLOGI PENELITIAN	31
3.1 Pengumpulan Data.....	31
3.2 Pre-Processing.....	35
3.3 Penerapan Arsitektur CNN.....	44
3.4 Simulasi Perhitungan Manual CNN	47
3.5 Penerapan Arsitektur PSO	53
3.6 Simulasi Perhitungan Manual Particle Swarm Optimization (PSO).....	55
3.7 Strategi Pelatihan Model	60
3.8 Prosedur Evaluasi	62
3.9 Implementasi Antarmuka Pengujian Model	63
3.10 Implementasi Model dalam Aplikasi Realtime	65
BAB IV HASIL PENGUJIAN DAN PEMBAHASAN	67

4.1	Implementasi	67
4.2	Hasil.....	85
4.3	Pembahasan.....	128
BAB V PENUTUP.....		132
5.1	Kesimpulan.....	132
5.2	Saran.....	133
DAFTAR PUSTAKA.....		134



DAFTAR TABEL

Tabel 2.1 <i>Confusion Matrix</i> (M Afif Rizky A, 2023)	23
Tabel 2.2 State Of the Art.....	27
Tabel 2.3 State Of The Art (Lanjutan)	28
Tabel 2.4 State of The Art (Lanjutan)	29
Tabel 2.5 State of The Art (Lanjutan)	30
Tabel 3.1 Rincian Data 4 Kelas.....	33
Tabel 3.2 Rincian Data 5 Kelas.....	34
Tabel 3.3 Pengelompokan Label Usia Skenario 4 kelas	36
Tabel 3.4 Pengelompokan Label Usia Skenario 5 Kelas.....	36
Tabel 3.5 Konfigurasi Parameter Augmentasi Data.....	39
Tabel 3.6 Perhitungan Normalisasi	47
Tabel 3.7 Perhitungan konvolusi.....	49
Tabel 4.1 Distribusi Data Model Baseline Jurnal 4 Kelas.....	85
Tabel 4.2 Distribusi Data Model Baseline Penelitian 5 Kelas	86
Tabel 4.3 Hasil Training Model CNN Baseline Jurnal 4 Kelas per Epoch.....	87
Tabel 4.4 Hasil Training Model CNN Baseline Jurnal 4 Kelas per Epoch.....	88
Tabel 4.5 Metrik Evaluasi Model CNN Baseline Jurnal 4 Kelas.....	89
Tabel 4.6 Hasil Evaluasi Perkelas (4 kelas)	89
Tabel 4.7 Parameter Particle Swarm Optimization	92
Tabel 4.8 Iterasi Ke-1	93
Tabel 4.9 Iterasi Ke-2	93
Tabel 4.10 Iterasi Ke-3	94
Tabel 4.11 Iterasi Ke-4	94
Tabel 4.12 Iterasi Ke-5	95
Tabel 4.13 Iterasi Ke-6.....	95
Tabel 4.14 Iterasi Ke-7	96
Tabel 4.15 Iterasi Ke-8.....	96
Tabel 4.16 Iterasi Ke-9	97
Tabel 4.17 Iterasi Ke-10.....	97
Tabel 4.18 Hasil Training Model CNN + PSO Jurnal 4 Kelas per Epoch	99
Tabel 4.19 Hasil Training Model CNN + PSO Jurnal 4 Kelas per Epoch	100
Tabel 4.20 Metrik Evaluasi Model CNN + PSO Jurnal 4 Kelas	100
Tabel 4.21 Evaluasi Per Kelas Model CNN + PSO Jurnal 4 Kelas	101
Tabel 4.22 Hasil Training Model CNN Baseline per Epoch.....	104
Tabel 4.23 Hasil Training Model CNN Baseline per Epoch.....	105
Tabel 4.24 Metrik Evaluasi Model CNN Baseline (Macro Average)	105
Tabel 4.25 Evaluasi per Kelas	105
Tabel 4.26 Parameter Particle Swarm Optimization (PSO).....	107
Tabel 4.27 Tabel Iterasi ke-1.....	108
Tabel 4.28 Tabel Iterasi ke-2.....	110
Tabel 4.29 Tabel Iterasi ke-3.....	110

Tabel 4.30 Tabel Iterasi ke-4.....	111
Tabel 4.31 Tabel Iterasi ke-5.....	111
Tabel 4.32 Tabel Iterasi ke-6.....	112
Tabel 4.33 Tabel Iterasi ke-7.....	112
Tabel 4.34 Tabel Iterasi ke-8.....	113
Tabel 4.35 Tabel Iterasi ke-9.....	113
Tabel 4.36 Tabel Iterasi ke-10.....	114
Tabel 4.37 Perbandingan Nilai Hyperparameter Baseline dan Hasil Optimasi PSO.....	115
Tabel 4.38 Hasil Training Model CNN + PSO per Epoch.....	116
Tabel 4.39 Hasil Training Model CNN + PSO per Epoch.....	117
Tabel 4.40 Metrik Evaluasi Model CNN + PSO.....	118
Tabel 4.41 Evaluasi Per Kelas.....	118
Tabel 4.42 Hasil Uji Coba Antarmuka Pengujian Model.....	121
Tabel 4.43 Hasil Uji Coba Aplikasi Realtime pada 10 Orang.....	125

DAFTAR GAMBAR

Gambar 2.1 <i>Layer Deep Learning</i> (Nima & Shila, 2020)	9
Gambar 2.2 Proses Pengolahan Citra (Zaid Munantri dkk., 2019)	10
Gambar 2.3 Arsitektur <i>Convolutional Neural Network</i> (Hendriyana Hendriyana & Hilman Maulana, 2020)	13
Gambar 2.4 Piksel Citra Input dan Filter 3x3 (Febri Hafifah dkk., 2021)	14
Gambar 2.5 Proses Konvolusi (Febri Hafifah dkk., 2021)	15
Gambar 2.6 Hasil <i>Feature Map</i> (Febri Hafifah dkk., 2021)	15
Gambar 2.7 Proses Aktivasi ReLU (Kurnia & Wibowo, 2021)	15
Gambar 2.8 <i>Max Pooling</i> (Ibrahim dkk., 2022)	17
Gambar 2.9 Ilustrasi <i>Particle Swarm Optimization</i> (Li dkk., 2023)	19
Gambar 3.1 Diagram alir metodologi penelitian	32
Gambar 3.2 Contoh hasil setiap augmentasi	40
Gambar 3.3 Contoh citra setelah mendapatkan 7 kombinasi augmentasi	40
Gambar 3.4 Diagram gambaran umum arsitektur CNN	44
Gambar 3.5 Red Channel Citra Wajah	47
Gambar 3.6 Hasil Normalisasi	48
Gambar 3.7 Proses Konvolusi	49
Gambar 3.8 Hasil Konvolusi	50
Gambar 3.9 Hasil Aktivasi ReLU	51
Gambar 3.10 Hasil <i>Max Pooling</i>	52
Gambar 3.11 Diagram gambaran umum arsitektur PSO	54
Gambar 3.12 Rancangan Halaman Input Aplikasi	64
Gambar 3.13 Rancangan Halaman Output Aplikasi	65
Gambar 4.1 Confusion Matrix CNN + PSO	102
Gambar 4.2 Training Curve CNN + PSO	103
Gambar 4.3 Confusion Matrix Baseline CNN	106
Gambar 4.4 Training Curve Baseline CNN	107
Gambar 4.5 Konvergensi PSO	115
Gambar 4.6 Confusion Matrix CNN + PSO	119
Gambar 4.7 Training Curve CNN + PSO	120
Gambar 4.8 Uji Coba Antarmuka Pengujian Model Citra 1	122
Gambar 4.9 Uji Coba Antarmuka Pengujian Model Citra 2	122
Gambar 4.10 Uji Coba Antarmuka Pengujian Model Citra 3	122
Gambar 4.11 Uji Coba Antarmuka Pengujian Model Citra 4	123
Gambar 4.12 Uji Coba Antarmuka Pengujian Model Citra 5	123
Gambar 4.13 Uji coba aplikasi inisial W	125
Gambar 4.14 Uji coba aplikasi Inisial D	125
Gambar 4.15 Uji coba aplikasi inisial F	125
Gambar 4.16 Uji coba aplikasi inisial M	125
Gambar 4.17 Uji coba aplikasi inisial P	126
Gambar 4.18 Uji coba aplikasi inisial A	126

Gambar 4.19 Uji coba aplikasi inisial S	126
Gambar 4.20 Uji coba aplikasi inisial T	126
Gambar 4.21 Uji coba aplikasi inisial R.....	126
Gambar 4.22 Uji coba aplikasi inisial I.....	126

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi *computer vision* dan *deep learning* telah mendorong pemanfaatan citra digital sebagai sumber utama dalam proses pengenalan dan klasifikasi objek. Salah satu objek yang banyak diteliti dalam bidang informatika adalah wajah manusia, karena wajah menyimpan berbagai informasi penting seperti identitas, jenis kelamin, ekspresi, hingga usia. Informasi usia memiliki peranan yang signifikan dalam berbagai aplikasi, antara lain sistem keamanan, *human-computer interaction*, sistem rekomendasi, analisis demografi, serta layanan berbasis personalisasi.

Klasifikasi kelompok usia berdasarkan citra wajah merupakan permasalahan yang menantang karena dipengaruhi oleh berbagai faktor, seperti variasi pencahayaan, pose wajah, ekspresi, kualitas citra, serta perbedaan karakteristik wajah antar individu. Oleh karena itu, diperlukan metode yang mampu mengekstraksi fitur wajah secara otomatis dan *robust*. *Convolutional Neural Network* (CNN) merupakan salah satu metode *deep learning* yang terbukti efektif dalam pengolahan citra karena kemampuannya mengekstraksi fitur spasial secara hierarkis tanpa memerlukan perancangan fitur secara manual.

Berbagai penelitian telah membuktikan keberhasilan CNN dalam klasifikasi usia berdasarkan citra wajah. Penelitian oleh Niasiska (2024), Sriani dan Nabila (2024), Arifandi (2022), serta Marcelio dkk. (2024) menunjukkan bahwa CNN mampu menghasilkan akurasi yang cukup baik dalam melakukan identifikasi atau prediksi usia manusia. Selain itu, CNN juga telah digunakan dalam berbagai variasi tugas terkait wajah, seperti klasifikasi rentang usia, *multi-label classification* (usia, gender, dan ras), serta integrasi dengan model tertentu seperti MobileNetV2. Meskipun demikian, sebagian besar penelitian tersebut masih menggunakan konfigurasi *hyperparameter* CNN secara *default* atau ditentukan secara manual, sehingga performa model sangat bergantung pada pengalaman peneliti dan belum tentu optimal.

Di sisi lain, beberapa penelitian telah mengembangkan pendekatan optimisasi berbasis *metaheuristik*, khususnya *Particle Swarm Optimization* (PSO), untuk meningkatkan kinerja model CNN. Nopitriana (2025) menggunakan PSO untuk mengoptimasi *learning rate* dan *dropout* pada CNN dalam klasifikasi penyakit kulit kucing dan menunjukkan adanya peningkatan performa model. Kilichev dan Kim (2023) bahkan mengoptimasi sembilan *hyperparameter* CNN menggunakan PSO dan membuktikan bahwa pendekatan tersebut mampu mempertahankan akurasi pada karakteristik data yang berbeda. Namun, objek penelitian pada kedua penelitian tersebut bukan citra wajah manusia dan bukan permasalahan klasifikasi usia, sehingga hasilnya belum dapat digeneralisasikan secara langsung pada domain tersebut.

Selain itu, penelitian oleh Syulistyo dkk. (2016) menggunakan PSO untuk mengoptimasi CNN dengan pendekatan yang berbeda, yaitu melakukan penyesuaian kecil pada vektor *output* CNN (δx) guna meningkatkan kepercayaan prediksi. Pendekatan ini menunjukkan bahwa PSO dapat membantu meningkatkan akurasi CNN, tetapi tidak

berfokus pada optimasi *hyperparameter* model serta belum diterapkan pada permasalahan klasifikasi usia wajah. Penelitian lain yang mengombinasikan PSO dan pengenalan wajah, seperti Fathul Hakim dkk. (2017) dan Alfaritsi & Hardjianto (2024), lebih menitikberatkan pada seleksi fitur atau optimasi metode klasifikasi lain, bukan pada optimasi CNN secara *end-to-end* untuk klasifikasi usia.

Berdasarkan kajian terhadap penelitian-penelitian terdahulu, dapat diidentifikasi adanya *research gap*, yaitu masih terbatas penelitian yang secara khusus mengintegrasikan CNN dengan optimasi PSO untuk klasifikasi kelompok usia berdasarkan citra wajah manusia, khususnya dengan memanfaatkan PSO sebagai mekanisme optimasi *hyperparameter* CNN untuk meningkatkan performa model. Sebagian besar penelitian klasifikasi usia masih menggunakan CNN tanpa optimasi *metaheuristik*, sementara penelitian yang mengoptimasi CNN dengan PSO lebih banyak diterapkan pada domain selain citra wajah atau pada pendekatan optimasi yang berbeda.

Selain aspek pengembangan model, implementasi sistem klasifikasi usia dalam bentuk aplikasi nyata juga menjadi hal yang penting untuk dikaji. Dalam praktiknya, sistem klasifikasi usia banyak digunakan dalam skenario realtime, seperti pada sistem pengawasan, layanan interaktif, dan aplikasi berbasis kamera lainnya. Relevansi penelitian mengenai klasifikasi usia juga semakin meningkat seiring munculnya wacana kebijakan pemerintah terkait perlindungan anak dan remaja di ruang digital, termasuk melalui Peraturan Pemerintah tentang Tata Kelola Penyelenggaraan Sistem Elektronik dalam Perlindungan Anak (PP TUNAS) yang menyoroti pembatasan akses media sosial berdasarkan kelompok usia. Kebijakan tersebut menunjukkan bahwa identifikasi kelompok usia menjadi salah satu aspek penting dalam pengembangan layanan digital yang lebih aman bagi anak dan remaja. Oleh karena itu, model klasifikasi usia berbasis citra wajah dapat menjadi salah satu pendekatan teknologi yang berpotensi mendukung proses verifikasi usia secara otomatis. Namun, implementasi teknologi tersebut tetap menghadapi tantangan dari sisi akurasi, variasi karakteristik wajah, serta penerapan pada kondisi nyata, sehingga diperlukan pengembangan model yang optimal dan mampu diimplementasikan secara praktis dalam lingkungan realtime.

Oleh karena itu, penelitian ini mengusulkan penerapan CNN yang dioptimasi menggunakan *Particle Swarm Optimization* (PSO) untuk melakukan klasifikasi kelompok usia berdasarkan citra wajah. Diharapkan optimasi *hyperparameter* CNN menggunakan PSO dapat meningkatkan performa validasi dan membantu proses pelatihan menjadi lebih terkontrol berdasarkan hasil training curve model dibandingkan CNN tanpa optimasi. Penelitian ini diharapkan dapat memberikan kontribusi ilmiah dalam pengembangan metode klasifikasi usia berbasis *deep learning*, sekaligus memperkaya kajian penerapan PSO dalam optimasi model CNN pada domain *computer vision*, khususnya citra wajah manusia.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana penerapan algoritma *Particle Swarm Optimization* (PSO) dalam mengoptimasi *hyperparameter* arsitektur *Convolutional Neural Network* (CNN) untuk klasifikasi kelompok usia berdasarkan citra wajah?
2. Sejauh mana penggunaan PSO sebagai metode optimasi *hyperparameter* mampu meningkatkan kinerja model CNN pada skenario klasifikasi kelompok usia 4 kelas dan 5 kelas dibandingkan dengan model CNN tanpa optimasi.
3. Bagaimana implementasi model CNN hasil optimasi PSO ke dalam aplikasi klasifikasi usia berbasis *realtime*?

1.3 Batasan Masalah

Agar penelitian yang dilakukan memiliki ruang lingkup yang jelas dan terfokus, maka batasan masalah dalam penelitian ini ditetapkan sebagai berikut:

1. Objek penelitian dibatasi pada citra wajah manusia, tanpa mempertimbangkan identitas individu, latar belakang etnis, maupun kondisi kesehatan tertentu
2. Klasifikasi yang dilakukan hanya terbatas pada pengelompokan usia, bukan pada prediksi usia secara numerik (regresi).
3. Metode utama yang digunakan dalam penelitian ini adalah *Convolutional Neural Network* (CNN) sebagai model klasifikasi citra wajah, tanpa melakukan perbandingan dengan algoritma *machine learning* atau *deep learning* lainnya
4. *Particle Swarm Optimization* (PSO) digunakan hanya untuk mengoptimasi *hyperparameter* CNN yang telah ditentukan oleh peneliti, dan tidak digunakan untuk menentukan atau mengubah arsitektur CNN.
5. *Hyperparameter* yang dioptimasi dibatasi pada *hyperparameter* tertentu yang berpengaruh terhadap proses pelatihan CNN, sedangkan *hyperparameter* lainnya ditetapkan secara tetap berdasarkan pertimbangan literatur dan eksperimen awal.
6. Evaluasi kinerja model dibatasi pada metrik pengujian tertentu, seperti akurasi, presisi, recall, *F1-Score*, dan confusion matrix, tanpa membahas aspek efisiensi komputasi dan performa sistem secara *realtime* secara mendalam.
7. Dataset yang digunakan dalam penelitian ini terbatas pada citra wajah dari sumber dataset tertentu dengan karakteristik dan distribusi usia yang telah ditentukan, sehingga hasil penelitian tidak secara langsung digeneralisasikan untuk seluruh populasi dengan variasi kondisi, etnis, dan lingkungan yang berbeda.
8. Implementasi model ke dalam aplikasi *realtime* hanya dilakukan sebagai bentuk penerapan model dan tidak mencakup analisis mendalam terhadap performa sistem secara *realtime*, seperti pengukuran latency, optimasi kecepatan, atau perbandingan performa pada berbagai perangkat.
9. Pengujian dilakukan pada dua skenario pengelompokan usia, yaitu skenario 4 kelas yang mengacu pada jurnal acuan dengan kelas child, teenager, adult, dan old, serta skenario 5 kelas penelitian dengan penambahan kelas middle age untuk memisahkan usia 41 sampai 60 tahun dari kelas Old

1.4 Tujuan Penelitian

Mengacu pada rumusan masalah yang telah ditetapkan, tujuan dari penelitian ini adalah:

1. Menerapkan *Particle Swarm Optimization* (PSO) sebagai metode optimasi *hyperparameter Convolutional Neural Network* (CNN) untuk klasifikasi kelompok usia berdasarkan citra wajah manusia.
2. Menganalisis kinerja model CNN setelah dilakukan optimasi menggunakan PSO pada skenario klasifikasi kelompok usia 4 kelas dan 5 kelas dibandingkan dengan model CNN tanpa optimasi
3. Mengimplementasikan model CNN hasil optimasi ke dalam aplikasi klasifikasi usia berbasis realtime.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik secara teoretis maupun praktis sebagai berikut:

1. Manfaat Teoretis: Memberikan kontribusi keilmuan dalam bidang informatika, khususnya pada pengembangan metode klasifikasi usia berdasarkan citra wajah menggunakan *deep learning*. Penelitian ini diharapkan dapat menjadi referensi ilmiah terkait penerapan *Particle Swarm Optimization* (PSO) sebagai metode optimasi *hyperparameter* pada *Convolutional Neural Network* (CNN). Selain itu, penelitian ini juga menambah kajian mengenai integrasi metode CNN dan PSO dalam domain *computer vision*, khususnya untuk permasalahan klasifikasi kelompok usia manusia. Penelitian ini juga memberikan gambaran mengenai penerapan model hasil optimasi dalam sistem nyata berbasis realtime sebagai bentuk validasi implementatif dari metode yang diusulkan.
2. Manfaat Praktis: Memberikan gambaran penerapan *Convolutional Neural Network* (CNN) yang dioptimasi menggunakan *Particle Swarm Optimization* (PSO) dalam membangun sistem klasifikasi kelompok usia berbasis citra wajah secara realtime. Penelitian ini diharapkan dapat menjadi acuan bagi peneliti maupun pengembang sistem dalam mengimplementasikan model *deep learning* ke dalam aplikasi nyata, khususnya dalam konteks klasifikasi usia. Selain itu, hasil penelitian ini dapat menjadi referensi awal bagi pengembangan sistem klasifikasi usia berbasis citra wajah, khususnya untuk aplikasi non-kritis seperti demonstrasi, personalisasi layanan, atau analisis demografi awal.

1.6 Tahapan Penelitian

Dalam Penelitian ini dilakukan melalui beberapa tahapan yang disusun secara sistematis untuk mencapai tujuan penelitian. Tahapan penelitian yang dilakukan adalah sebagai berikut:

1. Studi Literatur: Tahap awal penelitian diawali dengan studi literatur untuk memperoleh pemahaman teoritis yang kuat mengenai konsep *deep learning*, khususnya *Convolutional Neural Network* (CNN), *Particle Swarm Optimization* (PSO), serta penelitian-penelitian terdahulu yang berkaitan dengan klasifikasi usia berbasis citra wajah. Studi literatur ini digunakan sebagai dasar dalam menentukan metode, pendekatan, serta perancangan sistem penelitian.

2. Pengumpulan Data: Data yang digunakan dalam penelitian ini diperoleh dari dataset publik UTKFace, yang merupakan kumpulan citra wajah manusia dengan label usia, jenis kelamin, dan etnis. Penggunaan dataset ini memungkinkan peneliti untuk memanfaatkan data yang telah memiliki anotasi label sehingga tidak diperlukan proses pelabelan manual.
3. Pra-Pemrosesan: Tahap pra-pemrosesan dilakukan untuk menyiapkan dataset sebelum digunakan dalam proses pelatihan model. Pada tahap ini dilakukan verifikasi dan ekstraksi label usia dari nama file dataset UTKFace untuk memastikan kesesuaian antara data citra dan target klasifikasi. Informasi usia yang diperoleh kemudian dikelompokkan ke dalam beberapa kategori, yaitu Child, Teenager, Adult, Middle Age, dan Old, sesuai dengan kebutuhan penelitian. Setelah proses ekstraksi dan pengelompokan label, dataset dibagi menjadi data latih, data validasi, dan data uji menggunakan metode stratified split untuk menjaga proporsi distribusi kelas. Selanjutnya, diterapkan proses augmentasi data secara online pada data latih untuk meningkatkan variasi citra dan mengurangi risiko overfitting, sedangkan data validasi dan data uji tidak diaugmentasi agar proses evaluasi tetap objektif. Tahap pra-pemrosesan juga mencakup penyesuaian ukuran citra sesuai dengan arsitektur CNN serta normalisasi nilai piksel agar proses pelatihan model dapat berjalan secara optimal.
4. Perancangan dan Pengembangan Sistem: Pada tahap ini dilakukan perancangan arsitektur CNN yang digunakan untuk klasifikasi kelompok usia berdasarkan citra wajah. Selanjutnya, algoritma *Particle Swarm Optimization* (PSO) diterapkan untuk melakukan optimasi *hyperparameter* CNN, seperti *learning rate*, *Dropout CNN*, *Dropout Dense*, dan *Dense Unit*. Metodologi pengembangan sistem yang digunakan dalam penelitian ini mengikuti pendekatan *prototyping*, di mana sistem dikembangkan secara bertahap dan diuji fungsionalitasnya hingga mencapai performa yang diharapkan.
5. Pelatihan Model: Proses pelatihan dilakukan dalam dua kelompok skenario pengelompokan usia, yaitu skenario 4 kelas berdasarkan jurnal acuan dan skenario 5 kelas penelitian. Pada masing-masing skenario, model CNN baseline dilatih terlebih dahulu, kemudian dilakukan optimasi *hyperparameter* menggunakan PSO untuk memperoleh model CNN + PSO.
6. Analisis Hasil dan Kesimpulan: Analisis hasil dilakukan untuk menjawab permasalahan penelitian dan mengukur pencapaian tujuan penelitian. Evaluasi kinerja model dilakukan menggunakan metrik akurasi, presisi, *recall*, dan *confusion matrix*. Hasil pengujian dibandingkan antara model CNN tanpa optimasi dan model CNN yang dioptimasi menggunakan PSO untuk mengetahui peningkatan kinerja yang dihasilkan.
7. Pada tahap ini, model CNN terbaik yang telah diperoleh, baik dari hasil tanpa optimasi maupun hasil optimasi menggunakan PSO, diimplementasikan ke dalam aplikasi berbasis realtime. Implementasi dilakukan dengan memanfaatkan perangkat kamera (webcam) untuk menangkap citra wajah secara langsung, yang kemudian diproses oleh model untuk melakukan klasifikasi kelompok usia. Tahap ini bertujuan

untuk menunjukkan bahwa model yang telah dikembangkan dapat diterapkan dalam sistem nyata secara praktis.

1.7 Sistematika Penulisan

Sistematika penulisan laporan Tugas Akhir ini disusun untuk memberikan gambaran secara sistematis mengenai alur penelitian yang dilakukan. Laporan ini terdiri dari lima bab dengan uraian sebagai berikut:

BAB I PENDAHULUAN

Bab ini memuat latar belakang penelitian yang menjelaskan alasan dan urgensi dilakukannya penelitian klasifikasi kelompok usia berdasarkan citra wajah. Selain itu, bab ini juga menguraikan rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, tahapan penelitian, serta sistematika penulisan yang digunakan sebagai panduan dalam penyusunan laporan Tugas Akhir.

BAB II TINJAUAN LITERATUR

Bab ini membahas landasan teori dan kajian pustaka yang mendukung penelitian. Pembahasan meliputi konsep dasar *computer vision*, *deep learning*, *Convolutional Neural Network* (CNN), *Particle Swarm Optimization* (PSO), klasifikasi usia berbasis citra wajah, serta penelitian-penelitian terdahulu yang relevan. Bab ini bertujuan untuk memperkuat dasar ilmiah dan menunjukkan posisi penelitian terhadap penelitian sebelumnya.

BAB III METODOLOGI PENELITIAN

Bab ini menjelaskan metode penelitian yang digunakan, termasuk tahapan penelitian secara keseluruhan. Pada bab ini diuraikan proses pengumpulan data menggunakan dataset publik UTKFace, tahap pra-pemrosesan data citra, perancangan arsitektur CNN, penerapan PSO untuk optimasi *hyperparameter* CNN, metodologi pengembangan sistem, serta skema pengujian sistem dan penelitian yang digunakan untuk mengevaluasi kinerja model.

BAB IV HASIL PENGUJIAN DAN PEMBAHASAN

Bab ini menyajikan hasil implementasi sistem dan pengujian model yang telah dikembangkan. Pembahasan mencakup hasil pelatihan dan pengujian CNN sebelum dan sesudah dioptimasi menggunakan PSO, analisis performa model berdasarkan metrik evaluasi yang digunakan, serta pembahasan terhadap hasil penelitian dalam menjawab rumusan masalah dan tujuan penelitian.

BAB V PENUTUP

Bab ini berisi kesimpulan yang diperoleh dari hasil penelitian dan pembahasan yang telah dilakukan. Selain itu, bab ini juga menyampaikan saran-saran untuk pengembangan penelitian selanjutnya agar sistem klasifikasi kelompok usia berbasis citra wajah dapat ditingkatkan dari sisi metode, data, maupun penerapannya.

BAB II TINJAUAN LITERATUR

2.1 Citra Wajah

Citra wajah merupakan salah satu objek biometrik yang paling krusial dan umum digunakan dalam penelitian bidang pengenalan pola (*pattern recognition*) dan visi komputer (*computer vision*). Sriani & Nabila (2024) mendefinisikan pengenalan wajah sebagai proses di mana komputer menganalisis citra digital untuk mengidentifikasi atau memverifikasi identitas seseorang berdasarkan fitur-fitur wajah yang terekam. Dalam konteks klasifikasi, citra wajah tidak hanya digunakan untuk mengenali identitas individu, tetapi juga mengandung berbagai informasi atribut lunak (*soft biometrics*) yang kaya, seperti perkiraan usia, jenis kelamin, ras, dan ekspresi emosi. Informasi yang diekstraksi dari citra wajah ini memiliki nilai guna yang tinggi, mulai dari sistem keamanan, interaksi manusia-komputer, hingga aplikasi komersial.

Pengolahan citra wajah memiliki kompleksitas tinggi karena wajah manusia adalah objek dinamis dengan variabilitas visual yang besar. Lusiana Anggraini & Yuni Yamasari (2023) menyoroti bahwa tantangan utama dalam analisis citra wajah meliputi adanya variasi tekstur kulit seperti keriput, perubahan ekspresi wajah, serta keberadaan atribut oklusi seperti kumis, jenggot, atau kacamata yang dapat menutupi fitur asli wajah. Selain faktor fisiologis, faktor eksternal saat pengambilan citra juga sangat berpengaruh. Negara dkk. (2024) dan Robet dkk. (2025) menambahkan bahwa variasi pose (sudut pandang), kondisi pencahayaan yang tidak merata, serta rotasi wajah sering kali menyebabkan penurunan akurasi pada sistem pengenalan konvensional karena fitur wajah menjadi sulit dideteksi secara konsisten.

2.2 Klasifikasi Kelompok Usia

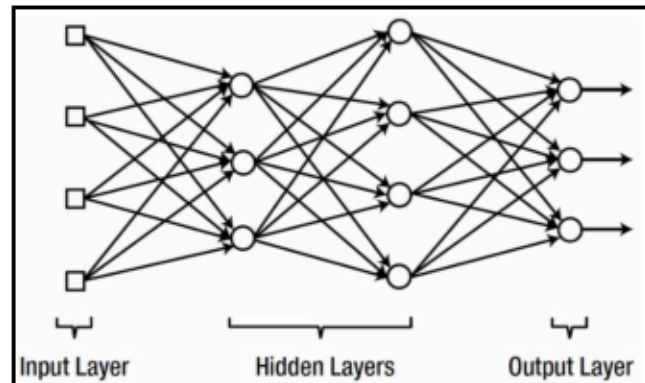
Klasifikasi kelompok usia merupakan proses pengelompokan citra wajah individu ke dalam kategori rentang umur tertentu berdasarkan analisis fitur visual yang terekam pada citra. Dalam penerapannya, rentang usia manusia yang sangat luas, mulai dari 0 hingga 90 tahun, sering kali disederhanakan menjadi beberapa kelas diskrit untuk memudahkan komputasi model. Negara dkk. (2024) membagi dataset wajah ke dalam empat kategori utama, yaitu *child* (0–12 tahun), *teenager* (13–18 tahun), *adult* (19–40 tahun), dan *old* (41 tahun ke atas), guna menangani distribusi data yang kompleks. Pendekatan pengelompokan serupa juga diterapkan oleh Lusiana Anggraini & Yuni Yamasari (2023) yang membagi rentang usia ke dalam tiga kelas besar (anak-anak, remaja, dan dewasa) dengan tujuan mendukung sistem keamanan siber dan penyaringan konten berbasis usia.

Meskipun demikian, klasifikasi usia memiliki tingkat kesulitan yang lebih tinggi dibandingkan klasifikasi atribut biner seperti jenis kelamin. Kendala utama sering ditemukan pada fase transisi pertumbuhan, khususnya antara kategori remaja dan dewasa. Penelitian oleh Robet dkk. (2025) serta Negara dkk. (2024) menunjukkan bahwa model cenderung mengalami kesalahan prediksi (*misclassification*) pada kelas *teenager* yang sering teridentifikasi sebagai *adult*. Hal ini disebabkan oleh kemiripan struktur wajah yang signifikan di antara kedua kelompok usia tersebut serta adanya ketidakseimbangan distribusi data yang membuat model lebih condong memprediksi kelas mayoritas.

2.3 Deep Learning

Deep Learning merupakan salah satu bidang dalam *machine learning* yang memungkinkan sistem untuk mempelajari pola dan representasi data secara otomatis melalui mekanisme pembelajaran berlapis. Pendekatan ini dirancang untuk melakukan analisis data secara mendalam dan berkelanjutan, sehingga mampu mengekstraksi informasi kompleks dari data dalam jumlah besar. Konsep *deep learning* terinspirasi dari cara kerja otak manusia dalam memproses informasi dan mengambil keputusan. Untuk meningkatkan kemampuan analisisnya, *deep learning* mengadopsi algoritma *Artificial Neural Network* (ANN) yang meniru struktur serta fungsi jaringan saraf biologis, sehingga mampu menghasilkan model dengan performa yang lebih baik dalam berbagai permasalahan pengolahan data. (Peryanto dkk., 2020).

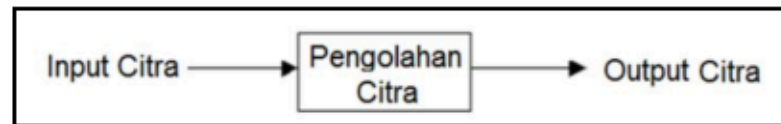
Selain itu, *deep learning* merupakan model jaringan yang tersusun atas beberapa lapisan, di mana setiap lapisan terdiri dari sejumlah neuron atau *node* yang berfungsi untuk melakukan proses perhitungan. Perbedaan utama antara *deep learning* dan jaringan saraf konvensional terletak pada jumlah lapisan tersembunyi (*hidden layer*) yang lebih banyak pada *deep learning*. Secara konseptual, *deep learning* termasuk ke dalam bagian dari *machine learning* yang memiliki lebih dari satu *hidden layer*, sehingga mampu memodelkan hubungan data yang lebih kompleks dan mendalam dalam proses pengolahan informasi. (Rizki dkk., 2020). Ilustrasi struktur layer pada *Deep Learning* dapat dilihat pada Gambar 2.1.



Gambar 2.1 Layer Deep Learning (Nima & Shila, 2020)

2.4 Pengolahan Citra Digital

Pengolahan citra digital merupakan bidang keilmuan yang mempelajari proses pengolahan gambar dengan tujuan meningkatkan kualitas citra, seperti pengaturan warna, peningkatan kontras, dan perbaikan citra. Selain peningkatan kualitas, pengolahan citra digital juga mencakup berbagai bentuk transformasi citra, antara lain transformasi geometrik, perubahan skala, rotasi, dan translasi, serta proses pemilihan fitur citra yang sesuai untuk mendukung kebutuhan analisis. (Zaid Munantri dkk., 2019). Pengolahan citra digital juga mencakup penerapan teknik reduksi dan kompresi data guna meningkatkan efisiensi penyimpanan. Selain itu, dilakukan pula optimasi pada proses transmisi dan pemrosesan citra untuk mendukung peningkatan kinerja sistem secara keseluruhan. Beberapa teknik yang umum digunakan dalam pengolahan citra digital antara lain perubahan ukuran citra, rotasi, pencerminan, penyesuaian skala, serta perbesaran citra.



Gambar 2.2 Proses Pengolahan Citra (Zaid Munantri dkk., 2019)

2.5 *Preprocessing*

Preprocessing merupakan tahapan penting dalam pengolahan data yang berperan dalam menangani berbagai permasalahan yang terdapat pada data mentah. Tahap ini bertujuan untuk meningkatkan kualitas data sehingga hasil analisis yang diperoleh menjadi lebih akurat dan dapat dipercaya. Pada *preprocessing*, data masukan dipersiapkan dan disesuaikan agar memenuhi kriteria yang diperlukan sebelum digunakan pada tahap klasifikasi. Proses ini mencakup berbagai teknik yang digunakan untuk membersihkan, mentransformasikan, dan menata data sehingga sesuai dengan kebutuhan analisis pada tahap selanjutnya. (Verdy & Hartati, 2024). Berikut ini adalah beberapa contoh tahapan dalam proses *pre-processing* :

2.5.1 *Resize*

Resize merupakan teknik yang digunakan untuk mengubah ukuran citra, baik dengan cara memperkecil maupun memperbesar dimensinya, sehingga memudahkan proses pengolahan citra karena seluruh gambar memiliki ukuran yang seragam. (Subandi dkk., 2023). Dalam penerapan *Convolutional Neural Network* (CNN), proses *resize* citra bertujuan untuk menghasilkan representasi fitur yang seragam dari citra yang digunakan sebagai masukan model. Oleh karena itu, proses *resize* perlu dilakukan secara tepat agar informasi penting yang terkandung dalam citra asli tetap terjaga. Selain itu, penggunaan teknik *resize* juga berperan dalam meningkatkan efisiensi proses pelatihan CNN dibandingkan dengan penggunaan citra asli yang memiliki resolusi atau jumlah piksel yang tinggi. (Talebi & Milanfar, 2021).

2.5.2 *Normalisasi*

Normalisasi citra merupakan tahap pra-pemrosesan krusial yang bertujuan menyeragamkan jangkauan nilai intensitas piksel agar model *Deep Learning* dapat bekerja secara optimal. Citra digital 8-bit memiliki nilai intensitas asli bertipe integer dengan rentang 0 hingga 255. Penggunaan rentang nilai mentah yang besar ini berpotensi memperlambat proses konvergensi algoritma optimasi berbasis gradien serta menyebabkan ketidakstabilan numerik selama pelatihan jaringan saraf.

Untuk mengatasi kendala tersebut, dilakukan transformasi skala menggunakan metode *rescaling* guna memetakan nilai piksel ke dalam rentang 0 hingga 1. Proses ini dilakukan dengan membagi setiap nilai piksel asli dengan nilai intensitas maksimumnya, sebagaimana dirumuskan dalam Persamaan 2.1:

$$x' = \frac{x}{255} \dots \dots \dots (2.1)$$

2.5.3 Augmentasi Data

Augmentasi data merupakan teknik yang digunakan untuk meningkatkan keragaman data latih melalui penerapan transformasi acak pada citra, sehingga model dapat mempelajari berbagai variasi pola. Beberapa metode yang umum diterapkan dalam augmentasi data meliputi rotasi citra, pergeseran posisi secara vertikal maupun horizontal, distorsi miring (*shearing*), perbesaran dan pengecilan skala citra, pencerminan secara vertikal atau horizontal, perubahan tingkat kecerahan, serta penambahan distorsi tertentu untuk meningkatkan ketahanan model terhadap perubahan bentuk. Penerapan teknik augmentasi data berperan penting dalam mengurangi risiko *overfitting* serta meningkatkan kemampuan model dalam melakukan generalisasi terhadap data baru yang belum pernah dilihat sebelumnya. (Azhar dkk., 2024).

Berikut adalah contoh beberapa augmentasi beserta persamaannya:

1. Rotasi Citra (*Rotation*)

Transformasi rotasi adalah proses memutar citra sebesar sudut tertentu (θ) dengan titik pusat citra sebagai poros perputaran. Koordinat baru (x', y') untuk setiap piksel ditentukan berdasarkan koordinat aslinya (x, y) menggunakan matriks transformasi trigonometri. Operasi ini memungkinkan model untuk mengenali objek yang muncul dalam berbagai orientasi sudut.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \dots\dots\dots (2.2)$$

2. Perbesaran (*Zoom Range*)

Penskalaan atau *zooming* mengubah ukuran dimensi citra dengan memperbesar atau memperkecil area piksel berdasarkan faktor skala tertentu. Jika s_x merepresentasikan faktor skala horizontal dan s_y adalah faktor skala vertikal, transformasi ini memetakan posisi piksel lama ke posisi baru yang lebih luas atau sempit, sehingga melatih model untuk mendeteksi objek dalam berbagai ukuran jarak pandang.

$$x' = x \times \text{faktor} \quad ; \quad y' = y \times \text{faktor} \dots\dots\dots (2.3)$$

3. Pergeseran Horizontal (*Width Shift*)

Pergeseran horizontal adalah bentuk translasi linier yang memindahkan posisi setiap piksel sepanjang sumbu- x (kiri atau kanan) sejauh jarak t_x , sementara posisi pada sumbu- y tetap konstan. Teknik ini berguna untuk memastikan model tidak hanya mengenali objek yang berada tepat di tengah frame, tetapi juga yang bergeser ke samping.

$$x' = x + t_x \dots\dots\dots (2.4)$$

4. Pergeseran Vertikal (*Height Shift*)

Serupa dengan pergeseran horizontal, transformasi ini melakukan translasi posisi piksel sepanjang sumbu- y (atas atau bawah) sejauh jarak t_y , tanpa mengubah posisi pada sumbu- x . Hal ini meningkatkan ketahanan model terhadap variasi posisi vertikal objek dalam citra.

$$y' = y + t_y \dots\dots\dots (2.5)$$

5. Pencerminkan Horizontal (*Horizontal Flip*)

Teknik pencerminan horizontal membalikkan orientasi citra terhadap sumbu vertikal tengah, menghasilkan representasi bayangan cermin dari citra asli. Untuk citra dengan lebar total W , posisi piksel baru dihitung dengan membalik urutan koordinat horizontal, yang efektif untuk memperbanyak variasi data pada objek simetris seperti wajah.

$$x' = (W - 1) - x \dots\dots\dots (2.6)$$

6. Variasi Kecerahan (*Brightness Range*)

Augmentasi kecerahan memodifikasi intensitas cahaya pada citra untuk mensimulasikan berbagai kondisi pencahayaan lingkungan. Proses ini dilakukan dengan mengalikan nilai intensitas piksel asli (I) dengan faktor pengali γ . Jika $\gamma > 1$, citra menjadi lebih terang, dan sebaliknya jika $\gamma < 1$, citra menjadi lebih gelap.

$$I_{baru} = I_{lama} \times \gamma \dots\dots\dots (2.7)$$

7. Kontras Citra (*Contrast*)

Transformasi kontras adalah proses menyesuaikan perbedaan intensitas antara area terang dan area gelap pada citra. Operasi ini bertujuan untuk mempertegas atau memperlembut detail visual sehingga model dapat mengenali objek pada variasi kualitas pencahayaan dan ketajaman tampilan yang berbeda. Dalam implementasinya, nilai piksel citra disesuaikan terhadap rata-rata intensitas citra menggunakan faktor kontras tertentu. Jika faktor kontras lebih besar dari 1, maka perbedaan terang-gelap menjadi lebih tegas. Sebaliknya, jika faktor kontras lebih kecil dari 1, maka citra menjadi lebih lembut.

$$x' = (x - \mu) \times c + \mu \dots\dots\dots (2.8)$$

8. Kejenuhan Warna (*Saturation*)

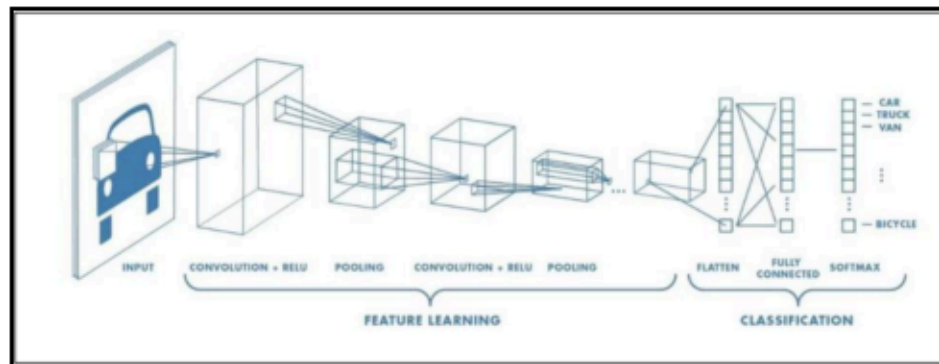
Transformasi saturasi adalah proses mengatur tingkat kejenuhan warna pada citra. Operasi ini bertujuan untuk mensimulasikan variasi kekuatan warna pada kondisi pengambilan gambar yang berbeda, sehingga model dapat belajar dari citra dengan warna yang lebih pudar maupun lebih pekat. Dalam implementasinya, nilai warna citra disesuaikan

menggunakan faktor saturasi tertentu. Jika faktor saturasi lebih besar dari 1, maka warna citra menjadi lebih pekat. Sebaliknya, jika faktor saturasi lebih kecil dari 1, maka warna citra menjadi lebih pudar.

$$RGB' = RGB \times s \dots\dots\dots (2.9)$$

2.6 Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN) merupakan salah satu arsitektur *deep learning* yang dirancang untuk mengekstraksi fitur secara otomatis dari data berdimensi tinggi melalui proses pembelajaran berlapis. CNN bekerja dengan memanfaatkan operasi konvolusi untuk menangkap pola lokal pada data masukan, sehingga mampu merepresentasikan karakteristik penting secara hierarkis. CNN memiliki kemampuan yang baik dalam memodelkan hubungan non-linear pada data kompleks karena struktur berlapisnya memungkinkan proses ekstraksi fitur dilakukan secara bertahap, mulai dari fitur tingkat rendah hingga fitur tingkat tinggi. Kemampuan ini menjadikan CNN banyak digunakan dalam berbagai permasalahan analisis data berbasis citra. (Elbeltagi dkk., 2024). Arsitektur CNN dapat dilihat pada Gambar 2.3.



Gambar 2.3 Arsitektur *Convolutional Neural Network* (Hendriyana Hendriyana & Hilman Maulana, 2020)

2.6.1 Input Layer

Input *layer* merupakan lapisan awal yang menerima citra digital sebagai masukan ke dalam jaringan CNN. Secara matematis, citra masukan dapat direpresentasikan sebagai matriks tiga dimensi

$$X \in \mathbb{R}^{H \times W \times C} \dots\dots\dots (2.10)$$

di mana H dan W menyatakan tinggi dan lebar citra, sedangkan C menyatakan jumlah kanal warna. Input *layer* berfungsi sebagai penyedia data awal tanpa melakukan transformasi atau perhitungan, sehingga seluruh proses ekstraksi fitur dilakukan pada lapisan berikutnya. (Triwijoyo & Triwijoyo, 2019)

2.6.2 Convolutional Layer dan ReLU

Convolutional layer merupakan komponen utama dalam arsitektur *Convolutional Neural Network* (CNN) yang berperan dalam melakukan operasi konvolusi terhadap keluaran dari lapisan sebelumnya. Lapisan ini memanfaatkan filter yang nilainya dipelajari

selama proses pelatihan untuk mengekstraksi fitur dari citra masukan, sehingga mampu merepresentasikan informasi spasial yang terkandung di dalamnya. Proses konvolusi menghasilkan transformasi linear terhadap data masukan, di mana bobot pada lapisan ini menentukan kernel konvolusi yang dapat disesuaikan dengan karakteristik data yang diproses oleh CNN. (Rafly Alwanda dkk., 2020). Lapisan konvolusi tersusun atas neuron-neuron yang membentuk filter dengan dimensi panjang, lebar, dan kedalaman yang menyesuaikan jumlah kanal pada citra masukan. Sebagai contoh, pada lapisan awal ekstraksi fitur, filter dapat memiliki ukuran tertentu sesuai dengan karakteristik data input. Filter tersebut digeser ke seluruh area citra dan melakukan operasi perkalian titik antara nilai piksel masukan dan bobot filter pada setiap pergeseran, sehingga menghasilkan peta aktivasi atau *feature map*. Proses ini memungkinkan jaringan mempertahankan informasi spasial citra, dengan bobot filter berfungsi sebagai kernel konvolusi yang dapat dilatih sesuai dengan data yang diproses oleh CNN. (Peryanto dkk., 2020).

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

0	1	0
1	0	1
0	1	0

Gambar 2.4 Piksel Citra Input dan Filter 3x3 (Febri Hafifah dkk., 2021)

Setelah proses konvolusi dilakukan, hasil keluaran diperoleh melalui operasi perkalian antara nilai piksel citra masukan dan bobot filter yang digunakan. (Febri Hafifah dkk., 2021). Perhitungan dalam operasi konvolusi dapat ditentukan menggunakan rumus berikut:

$$C_{ij} = AxP_1 + BxP_2 + CxP_3 + DxP_4 + ExP_5 + FxP_6 + GxP_7 + HxP_8 + IxP_9 \dots\dots\dots (2.11)$$

Keterangan:

C_{ij} = Hasil *feature map* dari proses konvolusi

$A - I$ = Nilai matriks citra input

$P_{1,9}$ - Nilai matriks *feature detector*

Gambar 2.5 menunjukkan proses konvolusi, di mana filter 3×3 digeser pada citra input untuk mengekstraksi fitur lokal melalui operasi perkalian dan penjumlahan nilai piksel. Gambar 2.6 memperlihatkan *feature map* yang dihasilkan, yang merepresentasikan respons filter terhadap pola tertentu pada citra dan digunakan sebagai masukan ke lapisan CNN berikutnya.

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

5	0	1	8	4
7	2	4	0	3
9	6	0	1	7
0	1	2	5	3
0	0	4	8	0

Gambar 2.5 Proses Konvolusi (Febri Hafifah dkk., 2021)

20	3	16
12	13	12
8	10	14

Gambar 2.6 Hasil *Feature Map* (Febri Hafifah dkk., 2021)

ReLU merupakan fungsi aktivasi yang digunakan untuk memperkenalkan sifat non-linear ke dalam jaringan, sehingga meningkatkan kemampuan representasi dan pemodelan data. Fungsi ini juga berperan dalam mengatasi permasalahan *vanishing gradient* dengan menerapkan fungsi $f(x) = \max(0, x)$, di mana suatu neuron hanya akan aktif ketika nilai masukannya lebih besar dari nol. (Paraijun dkk., 2022). Proses operasi aktivasi *ReLU* dapat dilihat pada Gambar 2.7 berikut ini.

1	14	-9	4
-2	-20	10	6
-3	3	11	1
2	54	-2	80

ReLU

1	14	0	4
0	0	10	6
0	3	11	1
2	54	0	80

Gambar 2.7 Proses Aktivasi *ReLU* (Kurnia & Wibowo, 2021)

2.6.3 Batch Normalization

Batch Normalization merupakan teknik yang digunakan untuk menormalkan keluaran dari lapisan sebelumnya agar memiliki distribusi yang lebih stabil selama proses pelatihan. Normalisasi ini dilakukan dengan menyesuaikan nilai rata-rata dan varians dari aktivasi pada setiap *batch* data. Dengan adanya *Batch Normalization*, proses pelatihan jaringan menjadi lebih stabil dan konvergen lebih cepat. Selain itu, teknik ini juga membantu mengurangi sensitivitas jaringan terhadap inisialisasi bobot serta berperan sebagai regularisasi ringan yang dapat meningkatkan kemampuan generalisasi model. (Ioffe & Szegedy, 2015). proses *Batch Normalization* diawali dengan menghitung nilai rata-rata dan varians dari suatu *batch* sebagai berikut:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i \dots\dots\dots (2.12)$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \dots\dots\dots (2.13)$$

Selanjutnya, data masukan dinormalisasi menggunakan persamaan:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \dots\dots\dots (2.14)$$

Untuk mempertahankan fleksibilitas representasi jaringan, hasil normalisasi kemudian diskalakan dan digeser menggunakan parameter yang dapat dipelajari, yaitu γ dan β , sehingga diperoleh keluaran akhir:

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv BN_{\gamma, \beta}(x_i) \dots\dots\dots (2.15)$$

x_i : Nilai input (aktivasi) ke- i dalam *mini-batch*.

m : Jumlah data dalam satu *mini-batch*.

B : *Mini-batch* yang sedang diproses.

μ_B : Nilai rata-rata (mean) dari *mini-batch*.

σ_B^2 : Nilai varians dari *mini-batch*.

$\hat{x}_i$: Nilai input yang telah dinormalisasi (memiliki rata-rata 0 dan varians 1).

ϵ (*epsilon*) : Konstanta nilai yang sangat kecil (misal 10^{-5}) untuk menjaga stabilitas numerik dan menghindari pembagian dengan nol.

γ (*gamma*) : Parameter skala (*scale*) yang dapat dipelajari (*learnable parameter*) oleh jaringan.

β (beta) : Parameter geser (*shift*) yang dapat dipelajari (*learnable parameter*) oleh jaringan.

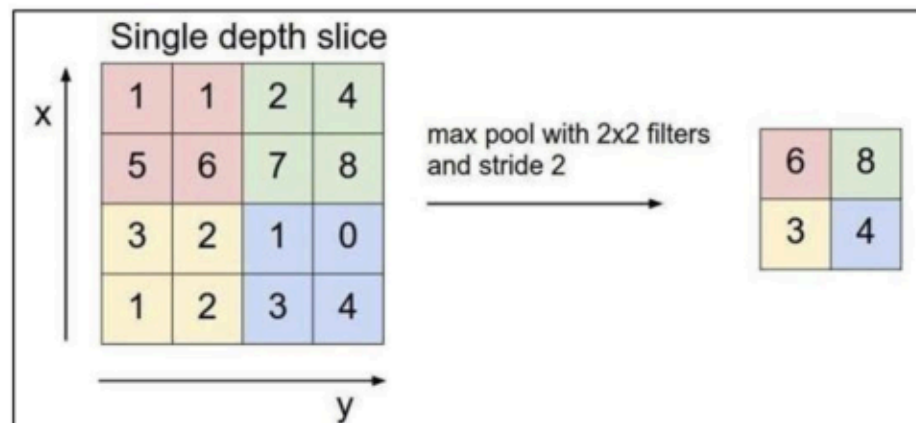
y_i : Nilai keluaran akhir dari operasi *Batch Normalization* setelah penskalaan dan pergeseran.

Dengan penerapan *Batch Normalization*, proses pelatihan jaringan menjadi lebih stabil dan konvergen lebih cepat. Selain itu, teknik ini mengurangi sensitivitas jaringan terhadap inisialisasi bobot awal serta berperan sebagai regularisasi ringan yang dapat meningkatkan kemampuan generalisasi model (Ioffe & Szegedy, 2015).

2.6.4 Max Pooling

Pooling layer berfungsi untuk mengurangi dimensi data masukan melalui proses *downsampling* sebelum diteruskan ke lapisan berikutnya. Lapisan ini bertujuan untuk menjaga kestabilan fitur terhadap perubahan kecil pada data input, menekan kemungkinan terjadinya *overfitting*, serta meningkatkan efisiensi komputasi dalam jaringan. (Antonio dkk., 2024).

Pooling layer umumnya terdiri dari dua jenis utama, yaitu *max pooling* dan *average pooling*. *Max pooling* bekerja dengan memilih nilai terbesar pada *feature map* dalam suatu wilayah tertentu, sedangkan *average pooling* menghitung nilai rata-rata pada wilayah yang sama. Di antara kedua metode tersebut, *max pooling* lebih sering digunakan karena dinilai lebih efektif dalam mempertahankan fitur-fitur penting dari citra masukan. Lapisan *pooling* biasanya menerapkan filter berukuran 2×2 dengan langkah pergeseran dua piksel yang diaplikasikan secara berurutan pada seluruh area *feature map*. (Ibrahim dkk., 2022). Contoh perhitungan *max pooling* diilustrasikan pada Gambar 2.8 dibawah ini.



Gambar 2.8 Max Pooling (Ibrahim dkk., 2022)

2.6.5 Dense

Lapisan *Fully Connected*, yang sering disebut sebagai *Dense* layer, merupakan komponen krusial dalam *Convolutional Neural Network* yang berfungsi untuk memetakan input pada sejumlah *node* dan mengintegrasikan fitur-fitur yang telah diekstraksi dari lapisan sebelumnya untuk tujuan klasifikasi (Aure dkk., 2023). Lapisan ini biasanya ditempatkan setelah lapisan konvolusi, *pooling*, dan normalisasi, di mana *output* berupa *feature map* dari lapisan sebelumnya diubah menjadi vektor satu dimensi melalui proses *flattening*. Setiap

neuron pada lapisan *Dense* terhubung sepenuhnya (*fully connected*) dengan semua neuron pada lapisan sebelumnya, sehingga memungkinkan integrasi informasi secara global untuk pengambilan keputusan akhir.

Secara matematis, operasi pada lapisan *Dense* dapat direpresentasikan sebagai:

$$Z = W \cdot X + b \dots \dots \dots (2.16)$$

di mana X merupakan vektor input fitur, W matriks bobot, dan b vektor bias. Hasil kemudian dilewatkan melalui fungsi aktivasi, seperti *ReLU* untuk lapisan tersembunyi atau *Softmax* untuk lapisan *output* klasifikasi. Fungsi *Softmax* khususnya berguna untuk membatasi rentang nilai keluaran antara 0 hingga 1, sehingga dapat diinterpretasikan sebagai probabilitas kelas (Aure dkk., 2023).

Keberadaan lapisan *Dense* memberikan fleksibilitas dalam mempelajari representasi non-linear yang kompleks, meskipun cenderung memerlukan lebih banyak parameter dibandingkan lapisan konvolusi. Oleh karena itu, lapisan ini sangat efektif dalam tugas klasifikasi citra wajah, termasuk pengelompokan usia, dengan memanfaatkan fitur abstrak yang telah diekstrak sebelumnya. (Aure dkk., 2023)

2.6.6 Softmax

Softmax merupakan salah satu fungsi aktivasi yang umum diterapkan pada lapisan keluaran jaringan saraf tiruan, terutama dalam permasalahan klasifikasi multikelas. Fungsi ini digunakan untuk mengonversi keluaran mentah model (*logit*) menjadi nilai probabilitas yang terdistribusi pada setiap kelas yang tersedia. Nilai keluaran dari fungsi *Softmax* merepresentasikan tingkat kemungkinan suatu data termasuk ke dalam masing-masing kelas, dengan rentang nilai antara 0 hingga 1. (Deep et al., 2021). Selain itu, fungsi *Softmax* memastikan bahwa total nilai probabilitas dari seluruh kelas bernilai satu, sehingga hasil keluaran dapat diinterpretasikan secara probabilistik secara valid. Secara matematis, fungsi *Softmax* dapat dirumuskan sebagai berikut:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \dots \dots \dots (2.17)$$

Keterangan :

$\sigma(z)_i$: Nilai probabilitas keluaran untuk kelas ke- i .

z_i : Nilai input mentah (*logit*) untuk kelas ke- i yang dihasilkan oleh lapisan sebelumnya.

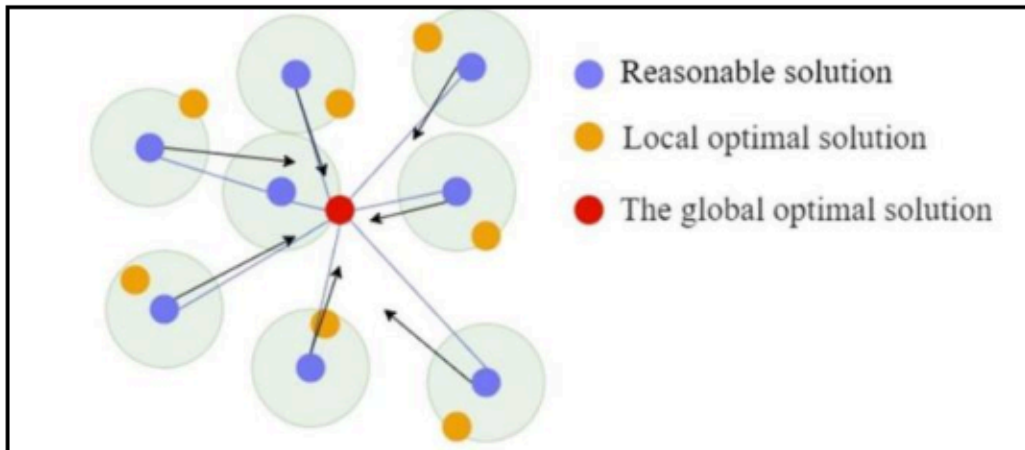
K : Jumlah total kelas atau kategori (dalam penelitian ini $K = 5$).

e : Bilangan *Euler*, yaitu konstanta matematika yang bernilai sekitar 2,718.

$\sum_{j=1}^K e^{z_j}$: Total jumlahan nilai eksponensial dari seluruh kelas, yang berfungsi sebagai pembagi normalisasi agar total probabilitas bernilai 1.

2.6.7 Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) adalah algoritma optimasi komputasi yang termasuk dalam kategori kecerdasan kawanan (*swarm intelligence*) dan terinspirasi oleh perilaku sosial organisme hidup, seperti pergerakan kawanan burung (*bird flocking*) atau kawanan ikan (*fish schooling*) (Azizul Hakim dkk., 2020). Algoritma ini pertama kali diperkenalkan oleh Kennedy dan Eberhart pada tahun 1995 sebagai metode untuk menyelesaikan masalah optimasi non-linear. Berbeda dengan teknik optimasi konvensional lainnya, PSO memiliki keunggulan karena tidak memerlukan informasi turunan (*derivative-free*) dan memiliki parameter penyesuaian yang lebih sedikit, sehingga lebih mudah untuk diimplementasikan dalam memecahkan masalah komputasi yang kompleks (Almufti dkk., 2019).



Gambar 2.9 Ilustrasi *Particle Swarm Optimization* (Li dkk., 2023)

Ilustrasi tersebut menggambarkan konsep dasar Particle Swarm Optimization (PSO) dalam proses pencarian solusi optimal pada suatu ruang pencarian. Metode ini menggunakan sekumpulan partikel yang bergerak di dalam ruang solusi untuk menemukan nilai terbaik dari suatu fungsi objektif. Setiap partikel merepresentasikan kandidat solusi yang berbeda. Pada gambar, titik biru menunjukkan posisi partikel yang mewakili solusi yang sedang dieksplorasi (*reasonable solution*). Selama proses iterasi, setiap partikel akan mengevaluasi posisinya berdasarkan fungsi objektif dan menyimpan posisi terbaik yang pernah dicapai oleh partikel tersebut yang disebut *personal best* (*pBest*), yang pada ilustrasi ditunjukkan dengan titik oranye. Selain itu, seluruh partikel dalam populasi juga berbagi informasi untuk menentukan solusi terbaik yang ditemukan oleh seluruh partikel yang disebut *global best* (*gBest*), yang pada ilustrasi ditunjukkan dengan titik merah. Pergerakan partikel dipengaruhi oleh posisi saat ini, pengalaman terbaik individu (*pBest*), serta pengalaman terbaik global (*gBest*). Melalui proses iteratif tersebut, partikel secara bertahap bergerak menuju solusi yang lebih optimal hingga mencapai konvergensi dalam ruang pencarian.

Secara matematis, posisi partikel ke-*i* pada iterasi ke-*t* dinyatakan sebagai:

$$X_i^t = (x_{i1}^t, x_{i2}^t, x_{i3}^t, x_{i4}^t, x_{i5}^t) \dots \dots \dots (2.18)$$

Sedangkan kecepatan partikel dinyatakan sebagai:

$$V_i^t = (v_{i1}^t, v_{i2}^t, v_{i3}^t, v_{i4}^t, v_{i5}^t) \dots (2.19)$$

X_i^t : Posisi partikel ke- i pada iterasi ke- t dalam ruang pencarian.

$x_{i1}^t, x_{i2}^t, x_{i3}^t, x_{i4}^t, x_{i5}^t$: Nilai variabel pada dimensi ke-1 hingga ke-5 dari partikel ke- i pada iterasi ke- t .

V_i^t : Kecepatan partikel ke- i pada iterasi ke- t yang menentukan perubahan posisi partikel.

$v_{i1}^t, v_{i2}^t, v_{i3}^t, v_{i4}^t, v_{i5}^t$: Nilai kecepatan partikel ke- i pada masing-masing dimensi pada iterasi ke- t .

i : Indeks partikel dalam populasi *swarm*.

t : Indeks iterasi dalam proses optimasi.

Pergerakan setiap partikel ditentukan oleh persamaan matematis yang memperbarui vektor kecepatan dan posisi pada setiap langkah iterasi. Kecepatan baru partikel dihitung dengan mempertimbangkan kecepatan sebelumnya (inersia), jarak kognitif terhadap *pBest*, dan jarak sosial terhadap *gBest*. Persamaan pembaruan kecepatan (v) dan posisi (x) untuk partikel ke- i pada iterasi ke- k dirumuskan sebagai berikut (Azizul Hakim dkk., 2020):

$$v_{ij}^{t+1} = wv_{ij}^t + c_1r_1(pBest_{ij} - x_{ij}^t) + c_2r_2(gBest_j - x_{ij}^t) \dots (2.20)$$

$$x_i^{k+1} = x_i^k + v_i^{k+1} \dots (2.21)$$

Dimana w adalah bobot inersia, sedangkan c_1 dan c_2 adalah konstanta akselerasi positif yang mengatur langkah pembelajaran. Variabel r_1 dan r_2 merupakan nilai acak antara 0 hingga 1 yang memberikan karakteristik stokastik pada algoritma (Azizul Hakim dkk., 2020).

Dalam penerapannya pada *Deep Learning*, khususnya *Convolutional Neural Network* (CNN), PSO digunakan untuk mengatasi tantangan dalam menentukan struktur arsitektur yang ideal yang sering kali masih menjadi masalah yang memerlukan banyak investigasi. PSO diterapkan untuk mempelajari nilai *hyperparameter* CNN yang optimal secara otomatis, seperti jumlah filter dan ukuran kernel pada lapisan konvolusi, guna mengurangi ketergantungan pada perancangan manual (Fouad dkk., 2021). Dalam proses ini, fungsi kebugaran (*fitness function*) yang digunakan adalah akurasi klasifikasi, di mana algoritma bertujuan mencari konfigurasi parameter yang menghasilkan akurasi tertinggi dan kesalahan (*error*) terendah. Eksperimen menunjukkan bahwa pendekatan ini mampu

menemukan arsitektur yang kompetitif dengan model *state-of-the-art* dan memiliki konvergensi yang lebih cepat dibandingkan algoritma genetika (Fouad dkk., 2021).

2.7 Hyperparameter

Parameter-parameter ini perlu dikonfigurasi dengan cermat sebelum melatih model agar kinerja klasifikasi dapat optimal (Dewi et al., 2024). *Hyperparameter* utama yang digunakan dalam pengembangan model mencakup *learning rate*, yang mengatur kecepatan pembelajaran dengan menentukan besaran penyesuaian bobot pada setiap iterasi pelatihan, jumlah filter pada lapisan konvolusi, yang menentukan jumlah peta fitur (*feature map*) untuk mengekstrak pola visual yang beragam dengan tingkat kompleksitas berbeda, serta *dropout rate*, yang berfungsi sebagai teknik regularisasi untuk mencegah *overfitting* dengan secara acak menonaktifkan sebagian neuron selama proses pelatihan, sehingga meningkatkan generalisasi model (Putu Dita Ariani Sukma Dewi dkk., 2024) dan (Ioffe & Szegedy, 2015). Pemilihan nilai *hyperparameter* ini memerlukan eksperimen seperti *grid search* atau *random search* untuk mencapai kinerja terbaik, mengingat pengaruhnya yang signifikan terhadap konvergensi, akurasi, dan stabilitas pelatihan jaringan CNN.

2.7.1 Learning Rate

Learning rate merupakan salah satu *hyperparameter* krusial yang menentukan seberapa besar langkah yang diambil oleh *optimizer* untuk *menyesuaikan* bobot dan bias dalam jaringan saraf selama pelatihan, yang secara langsung memengaruhi kecepatan dan kualitas konvergensi model (Putra dkk., 2021). Secara matematis, *learning rate* digunakan dalam proses pembaruan bobot pada *gradient descent* sebagai berikut:

$$\theta = \theta - \eta \nabla J(\theta) \dots \dots \dots (2.22)$$

di mana θ adalah parameter model, $\nabla J(\theta)$ adalah gradien dari fungsi loss J , dan η mengontrol ukuran langkah penurunan. Nilai learning rate yang terlalu tinggi dapat menyebabkan model melewati titik minimum global, menghasilkan osilasi atau divergensi, sementara nilai yang terlalu rendah akan memperlambat konvergensi dan berisiko terjebak pada minimum lokal (Putra dkk., 2021).

Dalam konteks CNN untuk klasifikasi citra wajah, pemilihan *learning rate* yang optimal sering kali dilakukan melalui eksperimen seperti *grid search* atau dengan *optimizer adaptif* seperti Adam yang secara dinamis menyesuaikan *learning rate* per parameter (Putu Dita Ariani Sukma Dewi dkk., 2024). Nilai umum yang digunakan berkisar antara 10^{-4} hingga 10^{-2} , tergantung pada ukuran dataset dan kompleksitas model. Selain itu, teknik seperti *learning rate scheduling* (misalnya, *exponential decay* atau *cosine annealing*) dapat diterapkan untuk secara bertahap menurunkan *learning rate* selama pelatihan, sehingga meningkatkan stabilitas dan akurasi akhir (Ioffe & Szegedy, 2015).

2.7.2 Jumlah Filter

Jumlah filter pada lapisan konvolusi merupakan *hyperparameter* krusial yang secara langsung memengaruhi kemampuan model untuk menangkap berbagai pola dan fitur pada

citra, di mana setiap filter berfungsi sebagai detektor fitur spesifik seperti tepi atau tekstur (Sari dkk., 2022). Jumlah filter menentukan kedalaman (*depth*) dari *feature map* yang dihasilkan, di mana peningkatan jumlah filter pada lapisan yang lebih dalam memungkinkan ekstraksi fitur hierarkis yang semakin abstrak dan kompleks, mulai dari pola sederhana seperti garis hingga bentuk objek yang lebih rumit (Ratna Juwita dkk., 2021). Secara umum, arsitektur CNN modern seperti yang digunakan dalam klasifikasi citra menerapkan jumlah filter yang bertambah secara eksponensial, misalnya dimulai dari 32 filter pada lapisan pertama, kemudian 64, 128, hingga 512 pada lapisan akhir konvolusi, untuk menyeimbangkan antara daya komputasi dan kemampuan representasi (Putu Dita Ariani Sukma Dewi dkk., 2024). Dalam eksperimen pada deteksi citra wajah bermasker, pengujian dengan jumlah *kernel* 3 dan 6 pada ukuran 5×5 menunjukkan bahwa 6 filter memberikan performa terbaik pada setiap lapisan konvolusi (Sari dkk., 2022). Pemilihan jumlah filter yang optimal memerlukan *tuning* melalui validasi silang atau *grid search*, karena terlalu sedikit filter dapat menyebabkan *underfitting* sedangkan terlalu banyak meningkatkan risiko *overfitting* dan kebutuhan sumber daya komputasi yang tinggi (Ioffe & Szegedy, 2015).

2.7.3 Dropout Rate

Dropout rate merupakan *hyperparameter* regulasi yang berperan penting dalam jaringan saraf tiruan, khususnya pada *Convolutional Neural Network* (CNN), yang bertujuan untuk mengurangi risiko *overfitting* dengan cara menonaktifkan sejumlah neuron beserta koneksinya secara acak selama proses pelatihan (Retno dkk., 2023). Teknik ini mendorong jaringan untuk mempelajari representasi fitur yang lebih *robust*, karena model tidak dapat bergantung pada neuron tertentu saja, melainkan harus mendistribusikan bobot secara lebih merata antar neuron. Pada setiap iterasi pelatihan, setiap neuron (Norhikmah dkk., 2022) memiliki probabilitas p untuk tetap aktif dan probabilitas $1-p$ untuk dinonaktifkan sementara, di mana nilai p tersebut disebut sebagai *dropout rate*.

2.8 Confusion Matrix

Confusion Matrix merupakan salah satu metode evaluasi yang digunakan untuk menilai kinerja model klasifikasi pada permasalahan dengan dua kelas atau lebih. Matriks ini tersusun atas empat komponen utama, yaitu *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). *True Negative* merepresentasikan jumlah data negatif yang berhasil diklasifikasikan dengan benar, sedangkan *False Negative* menunjukkan data negatif yang keliru diklasifikasikan sebagai data positif (Mustofa dkk., 2022). Tabel *confusion matrix* diperlihatkan pada Tabel 2.1 berikut.

Tabel 2.1 *Confusion Matrix* (M Afif Rizky A, 2023)

		Keadaan Data Sebenarnya	
		True	False
Hasil Prediksi	True	TP (True Positive) Disebut Juga Correct Result	FP (False Positive) disebut juga unexpected result false alarm
	False	FN (False Negative) disebut juga missing result	TN (True Negative) disebut juga correct rejection

Empat komponen utama yang terdapat pada *confusion matrix* dapat dijelaskan sebagai berikut. *True Positive* (TP) merupakan jumlah data positif yang berhasil diklasifikasikan secara benar oleh sistem sebagai kelas positif. *True Negative* (TN) menunjukkan jumlah data negatif yang berhasil diklasifikasikan dengan tepat sebagai kelas negatif. *False Positive* (FP) adalah jumlah data negatif yang keliru diklasifikasikan oleh sistem sebagai kelas positif. Sementara itu, *False Negative* (FN) merepresentasikan jumlah data positif yang salah diklasifikasikan oleh sistem sebagai kelas negatif. Selanjutnya, terdapat beberapa metrik evaluasi yang dapat dihitung menggunakan nilai-nilai tersebut, yaitu *accuracy*, *precision*, *recall*, dan *F1-score*, yang masing-masing dijelaskan sebagai berikut :

1. *Accuracy*, menunjukkan proporsi klasifikasi yang benar terhadap keseluruhan data. Rumusnya adalah :

$$Accuracy = \frac{(TP+TN)}{(TP+FP+FN+TN)} \dots \dots \dots (2.23)$$

2. *Precision*, mengukur tingkat akurasi antara data yang diklasifikasikan sebagai positif dengan data yang benar-benar positif. Rumusnya :

$$Precision = \frac{(TP)}{(TP+FP)} \dots \dots \dots (2.24)$$

3. *Recall* atau *sensitivity*, *Recall* menggambarkan kemampuan model dalam menemukan kembali data yang relevan. Dihitung dengan rumus :

$$Recall = \frac{(TP)}{(TP+FN)} \dots \dots \dots (2.25)$$

4. *F1-Score* merupakan metrik evaluasi yang diperoleh dari rata-rata harmonik antara *precision* dan *recall*. Metrik ini umumnya digunakan pada kondisi di mana distribusi kelas tidak seimbang. Apabila data memiliki distribusi yang seimbang antara kesalahan *false positive* dan *false negative*, maka *accuracy* dapat dijadikan sebagai ukuran kinerja model. Namun, pada kondisi distribusi kelas yang tidak seimbang,

penggunaan *F1-Score* dinilai lebih representatif untuk mengevaluasi performa sistem. Rumus *F1-Score* dapat dinyatakan sebagai berikut:

$$F - 1 \text{ Score} = \frac{(2 \times \text{Recall} \times \text{Precision})}{(\text{Recall} + \text{Precision})} \dots\dots\dots (2.26)$$

2.9 Implementasi Model dalam Sistem Realtime

Sistem realtime merupakan sistem yang mampu memproses data dan menghasilkan output dalam waktu yang relatif cepat sehingga dapat digunakan secara langsung pada saat data diterima. Dalam konteks computer vision, sistem realtime sering diimplementasikan menggunakan perangkat kamera untuk menangkap citra secara langsung dan diproses oleh model untuk menghasilkan prediksi secara instan.

Implementasi realtime pada computer vision umumnya melibatkan integrasi model dengan perangkat input seperti webcam, serta penggunaan library tertentu untuk menangani pengambilan dan pemrosesan citra secara berkelanjutan. Model yang telah dilatih kemudian digunakan untuk melakukan inferensi pada setiap frame citra yang diperoleh.

Dalam penelitian ini, implementasi realtime dilakukan sebagai bentuk penerapan model yang telah dikembangkan, tanpa membahas optimasi performa sistem secara mendalam seperti pengukuran latency atau efisiensi komputasi.

2.10 Penelitian Sebelumnya

Penelitian lain yang dilakukan oleh Negara et al. pada tahun 2024, penelitian ini bertujuan untuk meningkatkan kinerja klasifikasi multi-label pada wajah manusia (usia, jenis kelamin, dan ras) menggunakan arsitektur Convolutional Neural Network (CNN). Penelitian ini memanfaatkan dataset UTKFace yang mencakup rentang usia 0-116 tahun, jenis kelamin, dan lima kategori ras. Metode yang diterapkan melibatkan penggunaan teknik augmentasi data (seperti rotasi, zoom, dan penyesuaian kecerahan) serta optimisasi hyperparameter dengan meningkatkan jumlah filter CNN untuk menangani ketidakseimbangan data. Hasil eksperimen terbaik menunjukkan bahwa model mampu mencapai akurasi sebesar 82,98% untuk klasifikasi usia, 90,36% untuk jenis kelamin, dan 79,48% untuk ras. Penelitian ini juga menyoroti tantangan dalam mengklasifikasikan kategori "teenager" yang sering terdeteksi sebagai "adult", serta ras "Indian" dan "Others" akibat distribusi data yang tidak merata.

Penelitian lain yang dilakukan oleh Zein pada tahun 2020, penelitian ini bertujuan untuk mengembangkan metode estimasi usia dan jenis kelamin berdasarkan citra wajah menggunakan pelatihan mendalam Convolutional Neural Networks (CNN). Penelitian ini memanfaatkan dataset Adience yang berisi sekitar 20.000 gambar wajah untuk melatih sistem prototipe menggunakan kerangka kerja Caffe. Metode ini menggabungkan fitur tingkat tinggi (seperti keriput dan rambut) dan fitur tingkat rendah (seperti gradien piksel) untuk melakukan prediksi. Penelitian menyimpulkan bahwa klasifikasi jenis kelamin secara inheren lebih mudah daripada klasifikasi usia karena jumlah kelas yang lebih sedikit dan variasi intra-gender yang lebih menonjol. Hasil akhir menunjukkan tingkat akurasi sebesar 90% untuk klasifikasi usia dan 96% untuk klasifikasi jenis kelamin.

Penelitian lain yang dilakukan oleh Sriani et al. pada tahun 2024, penelitian ini bertujuan untuk mengimplementasikan Deep Learning guna mengidentifikasi umur manusia (rentang 0–90 tahun) berbasis citra wajah. Metode yang digunakan adalah algoritma

Convolutional Neural Network (CNN) dengan input citra digital berformat JPEG yang di-resize menjadi ukuran 48x48 piksel. Arsitektur model terdiri dari tahapan konvolusi, pooling, flattening, hingga fully connected layer menggunakan backpropagation. Dataset yang digunakan sebanyak 9.780 data citra wajah yang dibagi dengan rasio 3:1 (7.335 data latih dan 2.445 data uji). Berdasarkan hasil pelatihan menggunakan Python 3.9.6 dengan 500 epoch dan learning rate 0,001, penelitian ini berhasil mencapai tingkat akurasi sebesar 81,47% (0,8147) dan nilai loss sebesar 0,3756.

Mengacu pada penelitian berjudul "*Hyperparameter Optimization for 1D-CNN-Based Network Intrusion Detection Using GA and PSO*" yang dilakukan oleh Kilichev dan Kim pada tahun 2023, penelitian ini bertujuan untuk mengoptimalkan sistem deteksi intrusi jaringan (NIDS) menggunakan model *one-dimensional convolutional neural network* (1D-CNN). Fokus utama penelitian ini adalah mengoptimasi sembilan *hyperparameter* pada model 1D-CNN—termasuk jumlah filter, ukuran *kernel*, hingga *learning rate*—dengan menggunakan dua metode komputasi evolusioner, yaitu *Genetic Algorithm* (GA) dan *Particle Swarm Optimization* (PSO). Pengujian dilakukan pada tiga dataset besar, yaitu UNSW-NB15, CIC-IDS2017, dan NSL-KDD, dengan hasil yang menunjukkan peningkatan signifikan pada metrik akurasi, presisi, *recall*, dan *F1-Score* dibandingkan model yang tidak dioptimasi. Secara spesifik, pada dataset UNSW-NB15, GA mencapai akurasi 99,31% dan PSO mencapai 99,28%. Penelitian ini menyimpulkan bahwa optimisasi *hyperparameter* menggunakan GA dan PSO sangat krusial dalam membangun sistem keamanan jaringan yang kokoh dan adaptif terhadap ancaman siber yang terus berkembang.

Penelitian lain yang dilakukan oleh Anggraini dan Yamasari pada tahun 2023, penelitian ini bertujuan untuk mengklasifikasikan rentang usia seseorang berdasarkan citra wajah guna mendukung sistem keamanan siber dan penyaringan konten dewasa. Metode yang diusulkan menggunakan *Pretrained CNN* dengan arsitektur VGG16 sebagai ekstraktor fitur yang kemudian diklasifikasikan menggunakan algoritma *Artificial Neural Network* (ANN). Dataset yang digunakan adalah UTKFace yang berisi 23.708 citra, yang dibagi menjadi tiga kelas: anak-anak (0-11 tahun), remaja (12-25 tahun), dan dewasa (25 tahun ke atas). Untuk mengatasi ketidakseimbangan data, peneliti menerapkan teknik *random undersampling*. Setelah melakukan proses *tuning* sebanyak 70 kali pada saat pelatihan, hasil evaluasi menggunakan *confusion matrix* menunjukkan bahwa model berhasil mencapai tingkat akurasi sebesar 87% dengan tingkat kesalahan 12%.

Penelitian lain yang dilakukan oleh Emara et al. pada tahun 2025, penelitian ini bertujuan untuk mengatasi tantangan dalam penyetelan *hyperparameter* manual yang memakan waktu pada model *Deep Learning* dengan mengusulkan kerangka kerja *Accelerated Particle Swarm Optimization* (APSO-CNN). Metode ini dirancang untuk mengotomatisasi optimisasi *hyperparameter* (seperti ukuran *kernel*) pada berbagai arsitektur CNN, termasuk BaseCNN, GoogleNet, ResNet, VGG16, dan DenseNet. Evaluasi dilakukan menggunakan dataset Faces94 yang berisi 3.060 citra wajah. Hasil eksperimen menunjukkan bahwa pendekatan APSO secara konsisten meningkatkan kinerja model dibandingkan dengan konfigurasi *baseline* maupun PSO standar. Sebagai contoh, akurasi GoogleNet meningkat signifikan dari 69% menjadi 83%, dan ResNet dari 69% menjadi 84%. Penelitian ini menyimpulkan bahwa APSO efektif mempercepat konvergensi dan menghasilkan model yang lebih akurat untuk tugas pemrosesan citra.

Penelitian lain yang dilakukan oleh Robet, Chandra, dan Setiawan pada tahun 2025, penelitian ini bertujuan untuk merancang aplikasi deteksi usia otomatis berbasis citra wajah

menggunakan pendekatan Deep Learning dengan arsitektur Convolutional Neural Network (CNN). Model dibangun untuk mengenali dan mengekstraksi fitur wajah guna mengestimasi usia individu. Dataset yang digunakan bersumber dari data publik yang kemudian diperkaya melalui teknik augmentasi seperti rotasi, flipping, dan penyesuaian pencahayaan untuk meningkatkan variabilitas data. Berdasarkan hasil evaluasi, sistem berhasil mencapai akurasi sebesar 82,99% untuk deteksi jenis kelamin, sementara untuk deteksi usia, model menunjukkan kinerja yang baik (exemplary) dengan nilai presisi, recall, dan *F1-Score* yang bervariasi di berbagai kelompok usia.

Penelitian oleh Utama et al. (2022) berfokus pada optimasi arsitektur Convolutional Neural Network (CNN) menggunakan algoritma Particle Swarm Optimization (PSO) untuk analisis time-series multivariat. Studi ini mengintegrasikan PSO untuk menyetel hyperparameter pada fully connected layer—seperti jumlah neuron, fungsi aktivasi, loss function, optimizer, epoch, dan batch size—guna meningkatkan akurasi peramalan data pengunjung jurnal elektronik KEDS (2018–2020).

Hasil eksperimen menunjukkan bahwa metode PSO-CNN berhasil mencapai kinerja optimal dengan nilai Root Mean Square Error (RMSE) terendah sebesar 1,386 menggunakan konfigurasi 178 neuron dan 2 hidden layers. Penelitian ini menyimpulkan bahwa integrasi PSO mampu menghasilkan arsitektur CNN yang jauh lebih efisien dan akurat dibandingkan dengan model CNN standar tanpa optimisasi.

Penelitian oleh Inik (2025) memperkenalkan SwarmCNN, sebuah pendekatan hybrid untuk optimasi hyperparameter CNN yang mencakup parameter desain dan lapisan secara otomatis. Metode ini mengombinasikan algoritma Particle Swarm Optimization (PSO) untuk menentukan struktur arsitektur jaringan dengan Artificial Bee Colony (ABC) untuk menyetel parameter di setiap lapisannya. Melalui evaluasi pada sembilan dataset benchmark, SwarmCNN menunjukkan performa unggul dengan akurasi mencapai 100% pada RECTANGLES, 99,58% pada MNIST, dan 84,77% pada CIFAR-10. Hasil ini menyimpulkan bahwa SwarmCNN efektif dalam menemukan arsitektur optimal secara mandiri dan mampu mengungguli berbagai metode kompetitor pada mayoritas pengujian.

Tabel 2.2 State Of the Art

No	Peneliti (Tahun)	Judul Penelitian	Metode & Dataset	Hasil Penelitian
1.	(Negara et al., 2024)	Implementasi Convolutional Neural Network pada Multi-label Classification Wajah Manusia Berdasarkan Usia, Gender, dan Ras	<i>Convolutional Neural Network</i> (CNN) dengan teknik augmentasi data dan <i>hyperparameter tuning</i> (peningkatan filter).	Akurasi terbaik mencapai 82,98% (usia), 90,36% (gender), dan 79,48% (ras). Dataset UTKFace. Tantangan ditemukan pada klasifikasi kelas minoritas (remaja dan ras tertentu) akibat ketidakseimbangan data.
2.	(Zein, 2020)	Memprediksi Usia dan Jenis Kelamin Menggunakan Convolutional Neural Networks	<i>Convolutional Neural Networks</i> (CNN) dengan pelatihan <i>Deep Learning</i> menggunakan <i>framework</i> Caffe	Akurasi keseluruhan mencapai 90% untuk usia dan 96% untuk jenis kelamin. Penelitian menemukan bahwa melatih pengklasifikasi usia yang berbeda untuk setiap jenis kelamin dapat meningkatkan kinerja klasifikasi usia. Dataset yang digunakan adalah Adience Face Dataset.
3.	(Sriani et al., 2024)	Implementasi Deep Learning untuk Mengidentifikasi Umur Manusia Menggunakan Convolutional Neural Network (CNN)	<i>Convolutional Neural Network</i> (CNN) dengan input citra 48x48 piksel.	Akurasi mencapai 81,47% dan <i>loss</i> 0,3756 dalam mengidentifikasi umur (0-90 tahun). Dataset terdiri dari 9.780 citra (dibagi 3:1 untuk latih:uji). Model dilatih menggunakan 500 <i>epoch</i> dengan <i>learning rate</i> 0,001.

Tabel 2.3 State Of The Art (Lanjutan)

No	Peneliti (Tahun)	Judul Penelitian	Metode & Dataset	Hasil Penelitian
4.	(Kilichev & Kim, 2023)	Hyperparameter Optimization for 1D-CNN-Based Network Intrusion Detection Using GA and PSO	1D-CNN dengan optimisasi <i>hyperparameter</i> menggunakan GA dan PSO.	Akurasi mencapai lebih dari 99% pada tiga dataset utama. Pada UNSW-NB15, GA (99,31%) dan PSO (99,28%). Pada CIC-IDS2017, GA (99,71%) dan PSO (99,74%). Optimisasi sembilan <i>hyperparameter</i> terbukti signifikan meningkatkan kinerja model dalam mendeteksi intrusi dibandingkan metode manual atau tanpa optimisasi.
5.	(Anggraini & Yamasari, 2023)	Klasifikasi Citra Wajah Untuk Rentang Usia Menggunakan Metode Artificial Neural Network	<i>Pretrained</i> CNN (VGG16) sebagai ekstraktor fitur dan <i>Artificial Neural Network</i> (ANN)	Akurasi mencapai 87% dan tingkat kesalahan 12% dalam mengklasifikasikan 3 kategori usia (anak-anak, remaja, dewasa). Dataset UTKFace diseimbangkan menggunakan teknik <i>undersampling</i> . Sistem terbukti cukup akurat untuk identifikasi usia berbasis citra wajah.

Tabel 2.4 State of The Art (Lanjutan)

No	Peneliti (Tahun)	Judul Penelitian	Metode & Dataset	Hasil Penelitian
6.	(Emara et al., 2025)	Fine Tuning Hyperparameters of Deep Learning Models Using Metaheuristic Accelerated Particle Swarm Optimization Algorithm	<i>Accelerated Particle Swarm Optimization</i> (APSO) untuk <i>fine-tuning hyperparameter</i> pada berbagai arsitektur CNN.	Peningkatan signifikan pada akurasi, presisi, <i>recall</i> , dan <i>F1-Score</i> di dataset Faces94. Akurasi BaseCNN naik menjadi 89%, GoogleNet menjadi 83%, dan ResNet menjadi 84%. APSO terbukti memiliki konvergensi lebih cepat dan stabilitas lebih baik daripada PSO standar.
7.	(Robet et al., 2025)	Aplikasi Deteksi Usia Berbasis Citra Menggunakan Model Deep Learning dengan Arsitektur CNN	<i>Deep Learning</i> dengan arsitektur CNN dan teknik augmentasi data.	Akurasi deteksi jenis kelamin mencapai 82,99%. Deteksi usia menunjukkan kinerja baik dengan variasi nilai presisi dan <i>recall</i> antar kelompok usia. Dataset menggunakan citra wajah publik.
8.	(Utama et al., 2022)	PSO based Hyperparameter tuning of CNN Multivariate Time-Series Analysis	CNN dengan optimisasi <i>hyperparameter</i> menggunakan <i>Particle Swarm Optimization</i> (PSO) pada data <i>time-series</i> multivariat.	RMSE terendah mencapai 1.386 pada Model 3 dengan konfigurasi 178 neuron dan 2 <i>hidden layers</i> . PSO-CNN terbukti signifikan lebih baik daripada CNN biasa dalam meramalkan data pengunjung jurnal elektronik. Uji T menunjukkan dampak signifikan jumlah <i>hidden layer</i> terhadap penurunan <i>error</i> .

Tabel 2.5 State of The Art (Lanjutan)

No	Peneliti (Tahun)	Judul Penelitian	Metode & Dataset	Hasil Penelitian
9.	(Inik, 2025)	SwarmCNN: An efficient method for CNN hyperparameter optimization using PSO and ABC metaheuristic algorithms	SwarmCNN: Kombinasi <i>Particle Swarm Optimization</i> (PSO) untuk pencarian arsitektur (DPs) dan <i>Artificial Bee Colony</i> (ABC) untuk optimisasi parameter lapisan (LPs).	Akurasi tinggi tercapai pada 9 dataset (misal: RECTANGLES 100%, MNIST 99,58%, MNIST Fashion 93,47%, CIFAR-10 84,77%). Metode ini terbukti lebih unggul dibanding metode lain pada 5 dari 9 dataset dan menempati peringkat kedua pada sisanya, menunjukkan efisiensi dalam pencarian arsitektur saraf otomatis.

Berdasarkan beberapa penelitian terdahulu yang telah dikaji, penelitian oleh Negara et al. (2024) dipilih sebagai acuan utama dalam penyusunan arsitektur dasar Convolutional Neural Network (CNN) pada penelitian ini. Pemilihan penelitian tersebut didasarkan pada kesesuaian domain penelitian, yaitu klasifikasi usia berbasis citra wajah menggunakan dataset UTKFace, serta penggunaan teknik augmentasi data dan pendekatan CNN yang relevan dengan kebutuhan penelitian. Pada penelitian ini, arsitektur CNN dari penelitian Negara et al. direkonstruksi dan disesuaikan kembali sebagai model baseline, kemudian dilakukan optimasi hyperparameter menggunakan Particle Swarm Optimization (PSO) untuk meningkatkan performa model klasifikasi usia. Dengan demikian, penelitian ini tidak membangun arsitektur CNN yang sepenuhnya baru, melainkan berfokus pada pengembangan performa model melalui pendekatan optimasi hyperparameter berbasis PSO.

BAB III

METODOLOGI PENELITIAN

Penelitian ini menggunakan pendekatan kuantitatif dengan metode eksperimen yang bertujuan untuk membangun, menguji, dan mengevaluasi kinerja model klasifikasi kelompok usia berdasarkan citra wajah menggunakan *Convolutional Neural Network* (CNN) yang dioptimasi dengan *Particle Swarm Optimization* (PSO). Pendekatan kuantitatif dipilih karena penelitian ini melibatkan pengukuran performa model secara numerik melalui metrik evaluasi seperti akurasi, presisi, *recall*, dan *F1-Score*.

Penelitian ini bersifat implementatif, di mana model CNN dibangun sebagai model dasar (*baseline*) untuk melakukan klasifikasi kelompok usia, kemudian dilakukan proses optimasi *hyperparameter* menggunakan algoritma *Particle Swarm Optimization* (PSO) guna meningkatkan performa model. *Hyperparameter* yang dioptimasi meliputi *learning rate*, jumlah neuron pada *dense layer*, serta nilai *dropout*, sehingga diperoleh konfigurasi arsitektur model yang optimal.

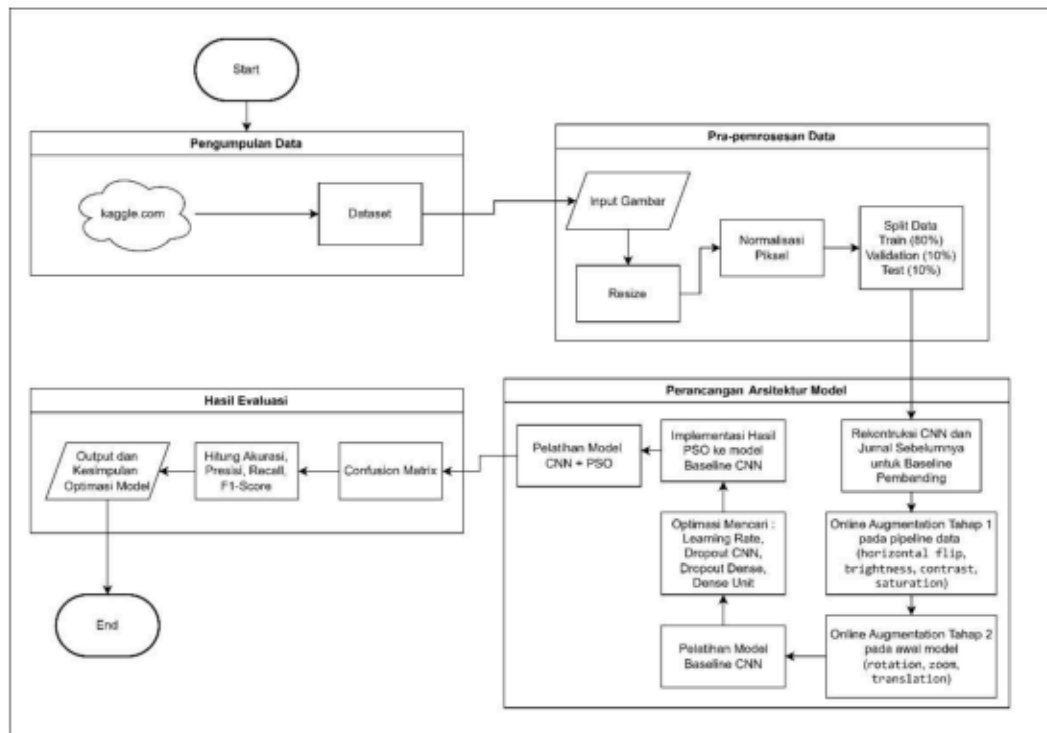
Tahapan penelitian disusun secara sistematis sebagaimana ditunjukkan pada Gambar 3.1. Proses dimulai dari pengumpulan data, di mana dataset citra wajah diperoleh dari platform Kaggle. Selanjutnya, dataset tersebut masuk ke tahap pra-pemrosesan data, yang meliputi proses input citra, *resize* gambar, dan normalisasi nilai piksel untuk menyeragamkan ukuran dan skala data. Setelah itu, data dilakukan augmentasi untuk meningkatkan variasi data latih, kemudian dibagi menjadi data latih (80%), data validasi (10%), dan data uji (10%).

Pada tahap perancangan arsitektur model, data latih digunakan untuk melatih model CNN *baseline*. Secara paralel, algoritma PSO diterapkan untuk mencari kombinasi *hyperparameter* terbaik, yaitu konfigurasi *dense layer*, *dropout*, dan *learning rate*. Model hasil optimasi PSO kemudian diuji menggunakan data uji yang sama dengan model *baseline*. Tahap akhir adalah evaluasi hasil, di mana kinerja model CNN tanpa optimasi dan model CNN dengan optimasi PSO dibandingkan berdasarkan nilai akurasi, presisi, *recall*, dan *F1-Score*, sebelum akhirnya ditarik kesimpulan mengenai pengaruh optimasi PSO terhadap peningkatan performa model.

3.1 Pengumpulan Data

Pada penelitian ini, dataset yang digunakan adalah UTKFace *Cropped* Dataset yang diperoleh dari platform Kaggle. Dataset ini dipilih karena menyediakan citra wajah dengan variasi usia yang luas serta telah dilengkapi dengan label yang relevan untuk penelitian klasifikasi usia berbasis citra wajah.

Dataset UTKFace *Cropped* berisi 23.708 citra wajah dalam format gambar digital. Setiap citra telah dilabeli secara otomatis menggunakan pola penamaan file yang mencakup tiga informasi utama, yaitu usia (*age*), jenis kelamin (*gender*), dan ras (*race*). Informasi usia pada dataset ini dinyatakan dalam bentuk nilai numerik yang merepresentasikan umur individu pada citra wajah tersebut, sehingga sangat sesuai untuk kebutuhan penelitian klasifikasi kelompok usia.



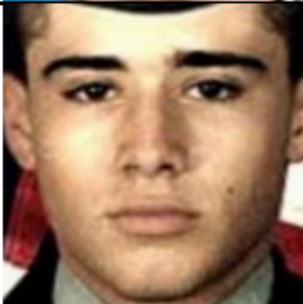
Gambar 3.1 Diagram alir metodologi penelitian

Pada penelitian ini, label usia (*age*) digunakan sebagai fokus utama, sedangkan label gender dan *race* tidak digunakan dalam proses pelatihan model. Seluruh citra wajah pada dataset ini merupakan hasil *cropping* wajah, sehingga area wajah sudah terfokus dengan baik dan meminimalkan gangguan latar belakang yang tidak relevan.

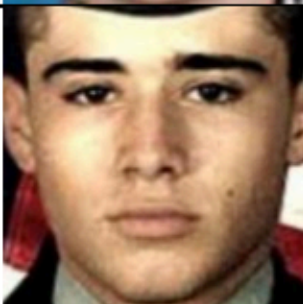
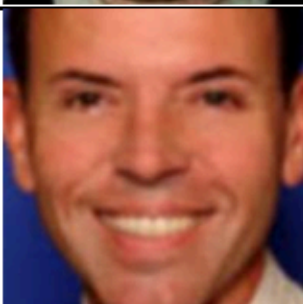
Pemilihan dataset UTKFace *Cropped* juga didasarkan pada pertimbangan metodologis, yaitu penelitian ini merekonstruksi dan mengadaptasi kode sumber dari jurnal penelitian sebelumnya yang menggunakan dataset yang sama. Dengan menggunakan dataset yang identik, proses rekonstruksi model dan implementasi metode optimasi *Particle Swarm Optimization* (PSO) terhadap *Convolutional Neural Network* (CNN) dapat dilakukan secara lebih konsisten dan terkontrol, serta memungkinkan perbandingan hasil yang lebih objektif dengan penelitian terdahulu.

Proses pengumpulan data dilakukan dengan mengunduh dataset secara langsung dari Kaggle, kemudian dilakukan seleksi dan penyesuaian data agar sesuai dengan kebutuhan penelitian. Pada penelitian ini digunakan dua skenario pembagian kelompok usia, yaitu skenario pertama dengan 4 kelas yang terdiri dari child, teenager, adult, dan old tanpa kategori middle age, serta skenario kedua dengan 5 kelas yang menambahkan kategori middle age sebagai kelompok usia tersendiri. Dataset ini selanjutnya digunakan sebagai input pada tahap pra-pemrosesan data sebelum dilakukan pelatihan model CNN dan optimasi hyperparameter menggunakan PSO. Berikut rincian data dapat dilihat pada Tabel 3.1 dan Tabel 3.2.

Tabel 3.1 Rincian Data 4 Kelas

No	Label	Citra Wajah	Jumlah
1.	Umur 0-12		3413
2.	Umur 13-18		1082
3.	Umur 19-40		12505
5.	Umur lebih dari 40		6706

Tabel 3.2 Rincian Data 5 Kelas

No	Label	Citra Wajah	Jumlah
1.	Umur 0-12		3413
2.	Umur 13-18		1082
3.	Umur 19-40		12505
4.	Umur 41-60		4311
5.	Umur lebih dari 60		2397

3.2 Pre-Processing

Tahap pra-pemrosesan data merupakan langkah penting yang dilakukan sebelum dataset digunakan pada proses pelatihan dan pengujian model *Convolutional Neural Network* (CNN). Tahapan ini bertujuan untuk memastikan bahwa seluruh citra wajah yang digunakan memiliki format, ukuran, dan representasi data yang seragam, sehingga dapat diproses secara optimal oleh arsitektur CNN yang digunakan dalam penelitian ini.

Dataset UTKFace yang digunakan dalam penelitian ini menyajikan citra yang telah terfokus pada area wajah (*aligned faces*), sehingga tidak memerlukan tahapan deteksi wajah awal. Meskipun demikian, tantangan utama terletak pada variasi intensitas pencahayaan yang tidak seragam serta perbedaan resolusi spasial antar citra. Selain itu, terdapat masalah ketidakseimbangan distribusi kelas (*class imbalance*) yang signifikan, di mana jumlah sampel pada kategori usia 'Dewasa' jauh mendominasi dibandingkan kategori usia lainnya. Kondisi ini berpotensi menyebabkan model menjadi bias dan cenderung memprediksi kelas mayoritas. Oleh karena itu, tahapan pra-pemrosesan (*pre-processing*) diperlukan untuk menstandarisasi dimensi dan nilai piksel citra, serta menerapkan strategi augmentasi data guna memitigasi dampak dari ketimpangan distribusi tersebut.

Secara umum, tahapan pra-pemrosesan pada penelitian ini meliputi pembacaan dan pelabelan data berdasarkan kelompok usia, penyesuaian ukuran citra (*resize*), normalisasi nilai piksel, pembagian data ke dalam data latih dan data uji, serta penerapan teknik augmentasi data. Seluruh tahapan tersebut diterapkan sebelum proses pelatihan model CNN baseline dan optimasi *hyperparameter* menggunakan *Particle Swarm Optimization* (PSO).

3.2.1 Ekstraksi dan Pengelompokan Label Usia

Dataset UTKFace memberikan informasi label yang tertanam langsung pada nama file setiap citra. Format penamaan file pada dataset ini mengikuti struktur standar yaitu [age]_[gender]_[race]_[date].jpg. Sebagai contoh, file dengan nama 26_1_0_20170116171048.jpg menunjukkan bahwa subjek pada citra berusia 26 tahun, berjenis kelamin laki-laki dengan kode 1, memiliki kode ras 0, serta memuat informasi waktu pengambilan gambar. Karena fokus penelitian ini adalah klasifikasi kelompok usia, maka proses ekstraksi label hanya dilakukan pada segmen pertama dari nama file, yaitu atribut usia atau age, sedangkan atribut gender, race, dan date tidak digunakan dalam proses pelatihan model.

Data usia yang diperoleh dari nama file pada awalnya berbentuk nilai numerik kontinu. Untuk kebutuhan klasifikasi, nilai usia tersebut dikonversi menjadi label kategorikal diskrit berdasarkan rentang usia tertentu. Pada penelitian ini, pengelompokan label usia dilakukan dalam dua skenario. Skenario pertama mengacu pada jurnal acuan dengan empat kelas, yaitu Child, Teenager, Adult, dan Old. Pada skenario ini, seluruh data usia di atas 40 tahun digabung ke dalam kelas Old. Pengelompokan label usia pada skenario 4 kelas ditunjukkan pada Tabel 3.3.

Tabel 3.3 Pengelompokan Label Usia Skenario 4 kelas

No	Rentang Usia	Kelas	Kategori
1	0 sampai 12	0	Child
2	13 sampai 18	1	Teenager
3	19 sampai 40	2	Adult
4	> 40	3	Old

Skenario kedua merupakan skenario penelitian dengan lima kelas, yaitu Child, Teenager, Adult, Middle Age, dan Old. Pada skenario ini, kelas Old pada skenario 4 kelas dipisahkan menjadi dua kategori, yaitu Middle Age untuk usia 41 sampai 60 tahun dan Old untuk usia di atas 60 tahun. Pemisahan ini dilakukan agar kelompok usia menjadi lebih spesifik dan mampu merepresentasikan perbedaan karakteristik wajah pada usia dewasa akhir dan lanjut usia. Pengelompokan label usia pada skenario 5 kelas ditunjukkan pada Tabel 3.4.

Tabel 3.4 Pengelompokan Label Usia Skenario 5 Kelas

No	Rentang Usia	Kelas	Kategori
1	0 sampai 12	0	Child
2	13 sampai 18	1	Teenager
3	19 sampai 40	2	Adult
4	41 sampai 60	3	Middle Age
5	> 60	4	Old

Pendekatan pengelompokan ini dilakukan untuk mengubah permasalahan estimasi usia dari prediksi numerik menjadi klasifikasi multi-kelas. Pendekatan klasifikasi dipilih karena prediksi dalam bentuk rentang usia memiliki toleransi kesalahan yang lebih realistis dibandingkan memprediksi usia secara eksak, terutama karena karakteristik wajah manusia dipengaruhi oleh variasi biologis, ekspresi, pencahayaan, dan kualitas citra.

Penggunaan dua skenario pengelompokan usia bertujuan untuk membandingkan pengaruh jumlah kelas terhadap performa model. Skenario 4 kelas digunakan sebagai pembanding terhadap jurnal acuan, sedangkan skenario 5 kelas digunakan sebagai skenario utama penelitian karena memiliki kategori usia yang lebih rinci melalui penambahan kelas Middle Age. Dengan demikian, proses evaluasi tidak hanya menunjukkan pengaruh optimasi PSO terhadap model CNN, tetapi juga menunjukkan dampak perubahan struktur kelas terhadap tingkat kesulitan klasifikasi usia.

3.2.2 Pembagian Data

Setelah melalui tahapan ekstraksi dan pengelompokan label usia, dataset dibagi menjadi tiga subset independen, yaitu data latih (training set), data validasi (validation set), dan data uji (testing set). Pembagian data dilakukan dengan rasio 80:10:10, di mana 80% dari total data digunakan sebagai data latih, 10% digunakan sebagai data validasi, dan 10% sisanya digunakan sebagai data uji. Data latih digunakan untuk proses pembelajaran model, data validasi digunakan untuk memantau performa model selama pelatihan serta sebagai dasar evaluasi pada proses optimasi hyperparameter menggunakan Particle Swarm

Optimization (PSO), sedangkan data uji digunakan secara khusus untuk evaluasi akhir model.

Pembagian data dilakukan pada dua skenario pengelompokan usia. Skenario pertama menggunakan empat kelas usia, yaitu Child, Teenager, Adult, dan Old, sesuai dengan pengelompokan pada jurnal acuan. Skenario kedua menggunakan lima kelas usia, yaitu Child, Teenager, Adult, Middle Age, dan Old, sesuai dengan skenario utama penelitian. Pada kedua skenario tersebut, rasio pembagian data tetap menggunakan 80:10:10 agar proses pelatihan, validasi, dan pengujian dapat dilakukan secara konsisten.

Data validasi memiliki peran penting dalam penelitian ini karena digunakan sebagai acuan dalam proses optimasi hyperparameter menggunakan PSO. Setiap kombinasi hyperparameter yang direpresentasikan oleh partikel PSO dievaluasi berdasarkan performa model pada data validasi. Dengan demikian, data uji tidak digunakan selama proses pelatihan maupun optimasi, sehingga independensinya tetap terjaga dan dapat digunakan untuk mengukur kemampuan generalisasi model secara objektif pada tahap evaluasi akhir.

Mengingat dataset UTKFace memiliki distribusi kelas yang tidak seimbang, terutama karena jumlah data pada kelas Adult jauh lebih dominan dibandingkan kelas lain, pembagian data tidak dilakukan menggunakan pembagian acak murni (simple random sampling). Sebagai gantinya, digunakan metode Stratified Random Sampling. Metode ini bertujuan untuk menjaga proporsi setiap kelas usia pada data latih, data validasi, dan data uji agar tetap mengikuti distribusi kelas pada dataset asli.

Penerapan stratified sampling penting untuk mengurangi risiko bias pada proses evaluasi. Tanpa pembagian berstrata, terdapat kemungkinan kelas minoritas seperti Teenager memiliki representasi yang terlalu sedikit pada data validasi atau data uji. Kondisi tersebut dapat menyebabkan hasil evaluasi menjadi kurang representatif. Dengan menggunakan stratified sampling, setiap kelas usia tetap terwakili secara proporsional pada seluruh subset data, baik pada skenario 4 kelas maupun skenario 5 kelas.

Secara umum, alur pembagian data dilakukan dengan memisahkan dataset menjadi 80% data latih dan 20% data sementara terlebih dahulu. Selanjutnya, data sementara tersebut dibagi kembali menjadi dua bagian yang sama besar, yaitu 10% sebagai data validasi dan 10% sebagai data uji. Prosedur ini diterapkan secara terpisah pada masing-masing skenario pengelompokan usia, sehingga hasil pelatihan dan evaluasi model pada skenario 4 kelas dan 5 kelas dapat dibandingkan secara lebih adil.

3.2.3 Penyesuaian Ukuran dan Normalisasi Citra

Mengingat dataset UTKFace terdiri dari ribuan citra dengan resolusi spasial yang bervariasi, langkah penyeragaman dimensi menjadi prasyarat mutlak sebelum data dapat diproses oleh arsitektur *Convolutional Neural Network* (CNN). Arsitektur CNN, khususnya pada lapisan *fully connected* (Dense layer), membutuhkan vektor masukan dengan ukuran yang tetap dan konsisten. Oleh karena itu, seluruh citra dalam dataset diubah ukurannya (*resizing*) menjadi dimensi spasial 150×150 piksel. Pemilihan dimensi ini dilakukan untuk menyeimbangkan antara pelestarian detail fitur wajah yang penting dengan efisiensi komputasi selama proses pelatihan.

Selain penyesuaian dimensi, dilakukan pula proses normalisasi nilai piksel (*pixel normalization*). Data citra digital pada umumnya disimpan dalam format *integer* 8-bit dengan rentang nilai intensitas antara 0 hingga 255. Menggunakan nilai mentah yang besar ini secara langsung dapat memperlambat proses konvergensi algoritma optimasi dan menyebabkan ketidakstabilan numerik (*numerical instability*) pada bobot jaringan saraf.

Untuk mengatasi hal tersebut, teknik *rescaling* diterapkan dengan membagi setiap nilai piksel dengan 255 (nilai intensitas maksimum). Proses ini mentransformasi rentang nilai piksel dari $[0, 255]$ menjadi rentang $[0, 1]$. Secara matematis, proses normalisasi ini dihitung dengan rumus 2.1.

Dimana x adalah nilai piksel asli dan x' adalah nilai piksel setelah normalisasi. Langkah ini bertujuan untuk mempercepat proses konvergensi *gradient descent* serta memastikan bahwa seluruh fitur input memiliki skala yang seragam, sehingga model tidak bias terhadap fitur dengan nilai intensitas yang lebih tinggi.

3.2.4 Augmentasi Data

Meskipun dataset UTKFace memiliki jumlah sampel yang cukup besar, variasi pose, pencahayaan, dan kondisi pengambilan gambar pada data latih tetap berpotensi menyebabkan model mengalami overfitting, yaitu kondisi ketika model terlalu menyesuaikan diri terhadap data latih dan kurang mampu melakukan generalisasi pada data baru. Untuk mengurangi risiko tersebut, pada penelitian ini diterapkan teknik augmentasi data pada data latih.

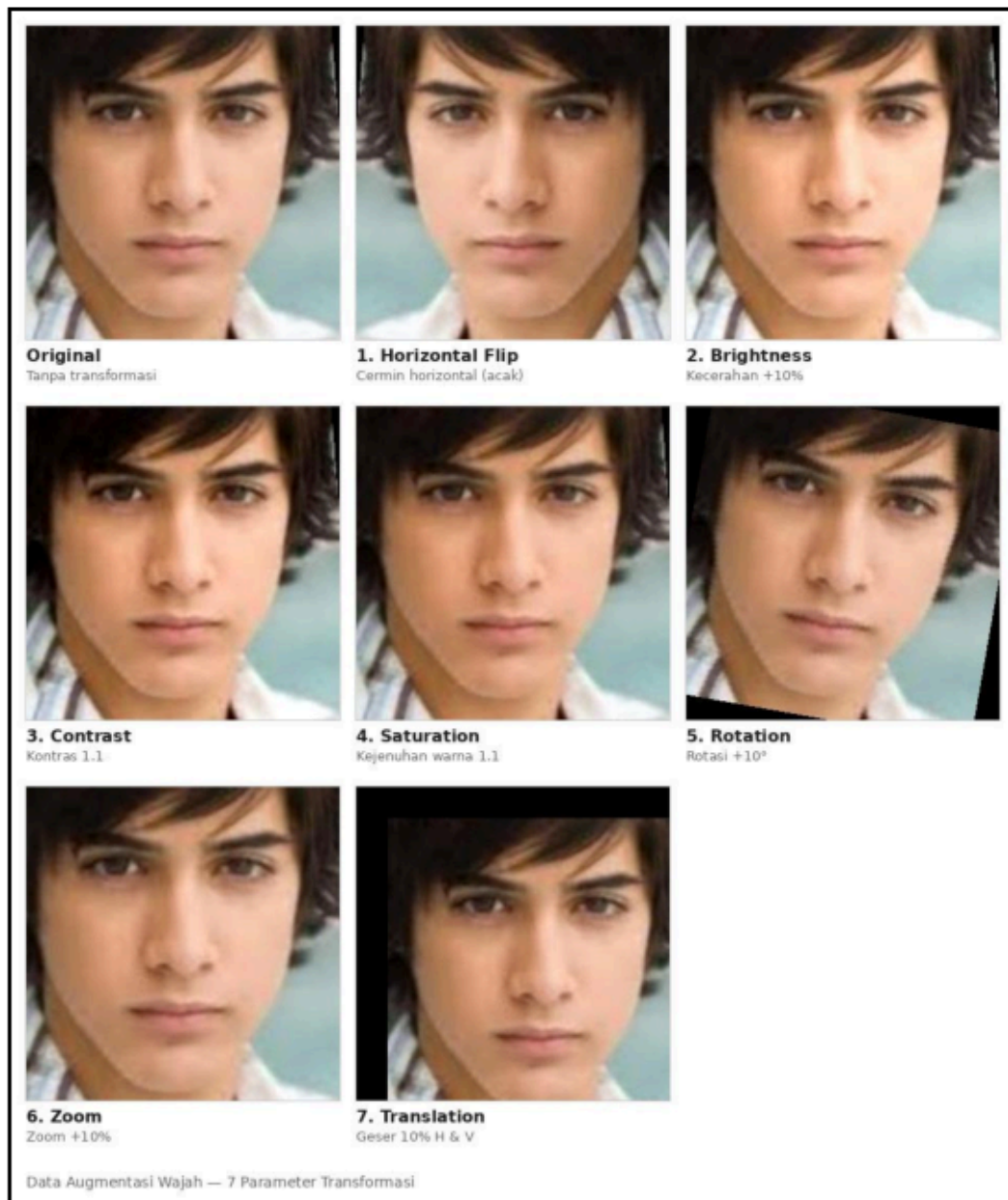
Berbeda dengan offline augmentation yang menghasilkan file citra baru dan menambah jumlah data secara fisik pada media penyimpanan, penelitian ini menggunakan pendekatan online augmentation. Pada pendekatan ini, citra hasil augmentasi tidak disimpan sebagai file baru, tetapi transformasi diterapkan secara langsung pada saat proses pelatihan model berlangsung. Dengan demikian, penggunaan online augmentation lebih efisien dari sisi storage karena dataset yang disimpan tetap berupa citra asli, sedangkan variasi data diperoleh secara dinamis selama training.

Pada implementasinya, augmentasi dilakukan dalam dua tahap. Tahap pertama diterapkan pada pipeline data melalui fungsi `_augment_image()`, yang meliputi horizontal flip, penyesuaian brightness, contrast, dan saturation. Tahap kedua diterapkan pada bagian awal arsitektur model melalui layer augmentasi Keras pada fungsi `build_cnn_model()`, yang meliputi rotation, zoom, dan translation. Dengan mekanisme ini, setiap citra pada data latih melewati dua lapisan augmentasi secara berurutan sebelum diproses oleh lapisan konvolusi. Pendekatan tersebut bertujuan untuk meningkatkan keragaman data latih secara dinamis sehingga model dapat mempelajari pola wajah yang lebih bervariasi.

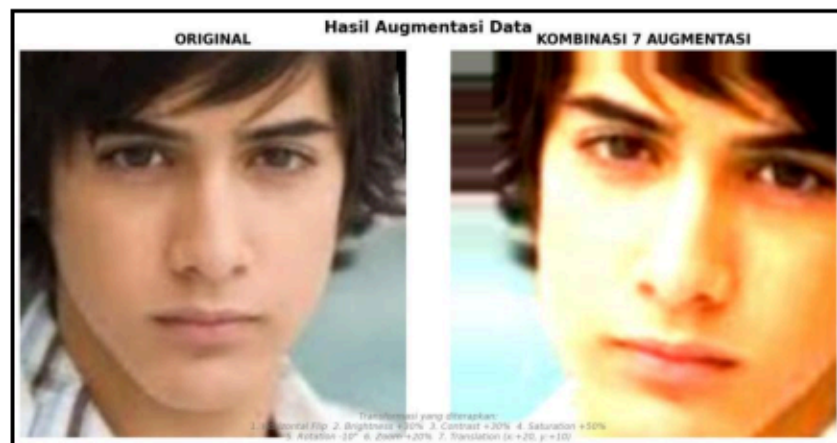
Augmentasi hanya diterapkan pada data training, sedangkan data validation dan data test tetap menggunakan citra asli tanpa transformasi tambahan. Hal ini dilakukan agar proses evaluasi performa model tetap objektif dan mencerminkan kemampuan generalisasi model terhadap data yang belum pernah dilihat sebelumnya. Dengan demikian, augmentasi pada penelitian ini berfungsi sebagai strategi regularisasi untuk memperkaya variasi data latih tanpa meningkatkan kebutuhan penyimpanan secara fisik.

Tabel 3.5 Konfigurasi Parameter Augmentasi Data

No	Parameter	Rentang Nilai	Keterangan
1	Horizontal Flip	Acak, dibalik atau tidak	Membalik citra secara horizontal
2	Brightness	± 0.1 atau sekitar $\pm 10\%$	Menyesuaikan tingkat kecerahan citra
3	Contrast	0.9 - 1.1	Menyesuaikan tingkat kontras citra
4	Saturation	0.9 - 1.1	Menyesuaikan tingkat kejenuhan warna
5	Rotation	$\pm 10^\circ$	Memutar citra secara acak
6	Zoom	-10% sampai +10%	Memperbesar atau memperkecil citra
7	Translation	hingga 10% tinggi dan lebar citra	Menggeser citra secara horizontal dan vertikal



Gambar 3.2 Contoh hasil setiap augmentasi



Gambar 3.3 Contoh citra setelah mendapatkan 7 kombinasi augmentasi

3.2.4.1 Ilustrasi Transformasi Augmentasi

Untuk memperjelas mekanisme kerja augmentasi data yang diterapkan, berikut disajikan simulasi perhitungan manual untuk setiap jenis transformasi. Asumsi Data Awal: Dimensi Citra ($W \times H$): **150 × 150** piksel. Titik Sampel Piksel: Koordinat (x, y) = **(10,10)**. Nilai Warna Awal (RGB): **(200,100,50)**. Berikut adalah perhitungan untuk masing-masing teknik augmentasi berdasarkan parameter pada Tabel 3.5:

1. Pencerminkan Horizontal (*Horizontal Flip*): Pencerminkan horizontal dilakukan dengan membalik posisi koordinat piksel terhadap sumbu vertikal tengah citra menggunakan transformasi koordinat sebagaimana dirumuskan pada Persamaan (2.6). Pada citra berukuran 150 piksel, koordinat piksel pada posisi $x = 10$ akan dipetakan ke posisi simetris di sisi berlawanan.

a. Perhitungan:

$$x' = (150 - 1) - 10 = 139$$

b. Hasil: Piksel berpindah dari sisi kiri ($x = 10$) ke sisi kanan ($x = 139$) dengan nilai warna yang sama.

2. Variasi Kecerahan (*Brightness Range* = [0.9, 1.1]): Variasi kecerahan diterapkan dengan memodifikasi nilai intensitas piksel melalui operasi transformasi nilai piksel yang mengacu pada Persamaan (2.7). Pada contoh ini, sistem memilih faktor kecerahan 1.1, sehingga nilai intensitas setiap kanal warna mengalami peningkatan sebesar 10% dari nilai awalnya.

a. Perhitungan

$$R' = 200 \times 1.1 = 220$$

$$G' = 100 \times 1.1 = 110$$

$$B' = 50 \times 1.1 = 55$$

b. Hasil: Nilai piksel menjadi lebih terang **(220,110,55)**.

3. Kontras Citra (*Contrast* = 1.1): Kontras dilakukan untuk menyesuaikan perbedaan intensitas antara area terang dan gelap pada citra. Proses perubahan kontras citra pada tahap augmentasi data dihitung menggunakan persamaan transformasi kontras sebagaimana ditunjukkan pada Persamaan (2.8).

a. Perhitungan: Diketahui nilai piksel awal $x = 200$, rata-rata intensitas citra $\mu = 150$, dan faktor kontras $c = 1.1$.

$$x' = (200 - 150) \times 1.1 + 150$$

$$x' = 50 \times 1.1 + 150$$

$$x' = 55 + 150 = 205$$

- b. Hasil: Nilai piksel berubah dari 200 menjadi 205, sehingga kontras citra meningkat.
4. Kejenuhan Warna (*Saturation* = 1.1): Saturasi dilakukan untuk mengatur tingkat kejenuhan warna pada citra. Proses perubahan saturasi citra pada tahap augmentasi data dihitung menggunakan persamaan transformasi saturasi sebagaimana ditunjukkan pada Persamaan (2.9).

- a. Perhitungan: Diketahui nilai warna awal $RGB = (200, 100, 50)$ dan faktor saturasi $s = 1.1$.

$$R' = 200 \times 1.1 = 220$$

$$G' = 100 \times 1.1 = 110$$

$$B' = 50 \times 1.1 = 55$$

- b. Hasil: Nilai warna berubah dari (200,100,50) menjadi (220,110,55), sehingga warna citra menjadi lebih pekat.
5. Rotasi (*Rotation Range* = 10°): Rotasi dilakukan untuk memutar koordinat piksel sebesar sudut θ . Proses rotasi koordinat piksel pada tahap augmentasi data dihitung menggunakan persamaan transformasi rotasi sebagaimana ditunjukkan pada Persamaan (2.2).

- a. Perhitungan: Diketahui $\cos 10^\circ \approx 0.98$ dan $\sin 10^\circ \approx 0.17$.

$$x' = (10 \times 0.98) - (10 \times 0.17) = 9.8 - 1.7 = 8.1 \approx 8$$

$$y' = (10 \times 0.17) + (10 \times 0.98) = 1.7 + 9.8 = 11.5 \approx 12$$

- b. Hasil: Piksel berpindah dari (10,10) ke koordinat baru (8,12).
6. Perbesaran (*Zoom Range* = 0.1): Proses perbesaran citra dilakukan dengan menskalakan koordinat piksel terhadap titik pusat citra menggunakan transformasi geometris sebagaimana dirumuskan pada Persamaan (2.3). Pada contoh ini, sistem memilih faktor zoom-in sebesar 1.1 (110%), sehingga setiap koordinat piksel mengalami pembesaran secara proporsional terhadap titik pusat citra.

- a. Perhitungan:

$$x' = 10 \times 1.1 = 11$$

$$y' = 10 \times 1.1 = 11$$

- b. Hasil: Piksel berpindah ke koordinat (11,11), memberikan efek objek terlihat lebih dekat.
7. Pergeseran Horizontal (*Width Shift* = 0.1): Pergeseran horizontal dilakukan dengan menggeser koordinat piksel pada sumbu x menggunakan transformasi translasi yang

mengacu pada Persamaan (2.4). Dengan lebar citra sebesar 150 piksel dan width shift sebesar 0.1, maka pergeseran maksimum yang dapat diterapkan adalah 15 piksel ke arah horizontal.

- a. Perhitungan: Jika digeser ke kanan sejauh 15 piksel:

$$x' = 10 + 15 = 25$$

- b. Hasil: Koordinat x berubah menjadi 25 (koordinat y tetap).

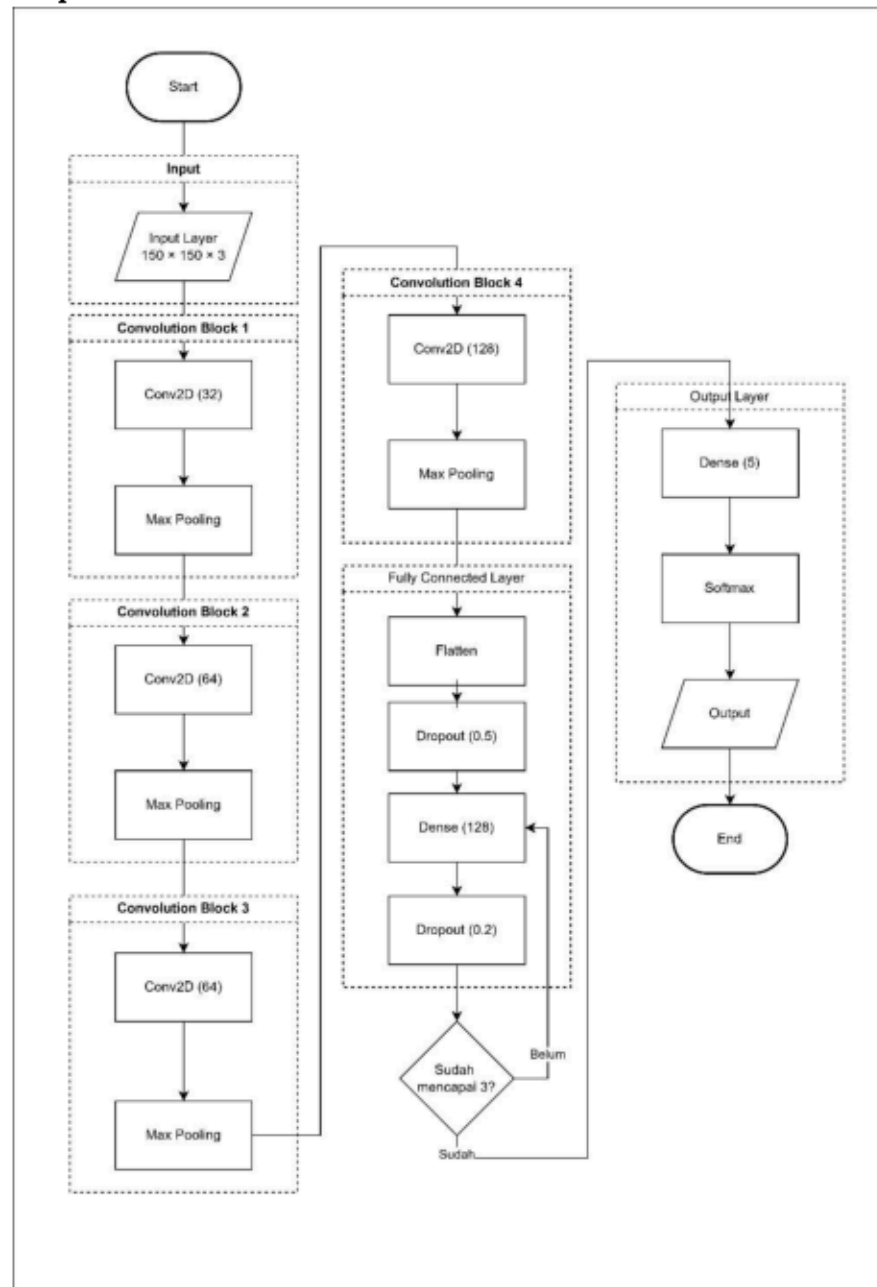
8. Pergeseran Vertikal (*Height Shift* = 0.1): Pergeseran vertikal dilakukan dengan menggeser koordinat piksel pada sumbu y berdasarkan transformasi translasi sesuai Persamaan (2.5). Dengan tinggi citra sebesar 150 piksel dan nilai height shift sebesar 0.1, diperoleh pergeseran maksimum sebesar 15 piksel ke arah vertikal.

- a. Perhitungan: Jika digeser ke bawah sejauh 15 piksel:

$$y' = 10 + 15 = 25$$

- b. Hasil: Koordinat y berubah menjadi 25 (koordinat x tetap).

3.3 Penerapan Arsitektur CNN



Gambar 3.4 Diagram gambaran umum arsitektur CNN

Implementasi model CNN dalam penelitian ini mengacu pada desain arsitektur yang digambarkan secara komprehensif dalam diagram alir (*flowchart*) pada Gambar 3.4. Secara garis besar, arsitektur yang digunakan untuk klasifikasi usia ini terdiri dari tiga bagian utama: tahap pra-pemrosesan (*preprocessing*), tahap ekstraksi fitur (*feature extraction*), dan tahap klasifikasi (*classification*).

3.3.1 Pra-pemrosesan Data (*Data Preprocessing*)

Sebelum dimasukkan ke dalam model, data citra wajah mentah yang berasal dari dataset UTKFace melalui serangkaian tahapan persiapan data (*preprocessing*) sebagai berikut:

1. Ekstraksi Label dari Nama File (Label Extraction): Dataset UTKFace memberikan informasi label yang tertanam langsung pada nama file setiap citra (*embedded labels*). Penamaan file pada dataset ini mengikuti struktur standar yaitu [age]_[gender]_[race]_[date].jpg. Sebagai contoh, sebuah file dengan nama 26_1_0_20170116171048.jpg mengindikasikan subjek berusia 26 tahun (segmen pertama), berjenis kelamin laki-laki (kode 1), ras kulit putih (kode 0), serta waktu pengambilan gambar. Mengingat fokus penelitian ini adalah klasifikasi usia, maka dilakukan proses *parsing string* pada nama file untuk mengekstraksi segmen pertama saja (atribut *age*) sebagai label data, sementara atribut lainnya (gender, ras, dan tanggal) diabaikan.
2. Pelabelan Kategori (*Categorical Encoding*): Label usia kontinu dikelompokkan ke dalam lima kelas diskrit, yaitu: *Child* (0-12 tahun), *Teenager* (13-18 tahun), *Adult* (19-40 tahun), *Middle Age* (41-60), dan *Old* (>60 tahun).
3. Setelah proses pelabelan dan pengelompokan kategori usia selesai dilakukan, dataset dibagi menjadi tiga subset independen, yaitu data latih (training set), data validasi (validation set), dan data uji (testing set). Pembagian dilakukan dengan rasio 80:10:10 menggunakan metode Stratified Random Sampling untuk mempertahankan proporsi distribusi setiap kelas usia pada masing-masing subset. Tahap pertama dilakukan pemisahan 80% data sebagai data latih dan 20% data sementara. Selanjutnya, 20% data sementara tersebut dibagi kembali menjadi dua bagian yang sama besar (masing-masing 10%) sebagai data validasi dan data uji. Data latih digunakan dalam proses pembelajaran model, data validasi digunakan untuk evaluasi sementara serta optimasi hyperparameter menggunakan Particle Swarm Optimization (PSO), sedangkan data uji digunakan secara eksklusif untuk evaluasi akhir terhadap model terbaik guna memastikan objektivitas hasil penelitian.
4. Pengubahan Ukuran (*Resizing*): Seluruh citra input diubah ukurannya menjadi dimensi seragam **150 × 150** piksel dengan 3 kanal warna (RGB) agar sesuai dengan struktur input layer model.
5. Penskalaan Nilai Piksel (*Rescaling*): Untuk mempercepat konvergensi model selama pelatihan, dilakukan normalisasi nilai intensitas piksel. Setiap piksel yang semula berada dalam rentang integer 0–255 dibagi dengan 255 sehingga berada dalam rentang pecahan **0.0** hingga **1.0**.
6. Augmentasi Data (Data Augmentation) diterapkan untuk mengurangi risiko overfitting serta meningkatkan kemampuan generalisasi model dalam mengenali variasi citra wajah. Pada penelitian ini, augmentasi dilakukan secara online pada data latih selama proses pelatihan model, sehingga tidak menghasilkan file citra baru yang disimpan ke dalam media penyimpanan. Implementasi augmentasi dilakukan dalam dua tahap. Tahap pertama diterapkan pada pipeline data, yang meliputi horizontal flip, penyesuaian brightness, contrast, dan saturation. Tahap kedua diterapkan pada bagian awal model melalui layer augmentasi, yang meliputi rotation, zoom, dan translation. Nilai rotation diterapkan hingga $\pm 10^\circ$, zoom dalam rentang $\pm 10\%$, translation hingga 10% secara horizontal dan vertikal, brightness dengan perubahan sekitar $\pm 10\%$, serta contrast dan saturation pada rentang 0,9 hingga 1,1. Kombinasi

teknik augmentasi ini bertujuan menghasilkan variasi data latih yang lebih beragam tanpa menghilangkan karakteristik utama wajah, sekaligus lebih efisien dalam penggunaan storage karena augmentasi dilakukan secara dinamis saat proses training berlangsung.

3.3.2 Arsitektur Ekstraksi Fitur (*Feature Extraction Layer*)

Bagian inti dari arsitektur ini terdiri dari empat blok konvolusi bertingkat yang berfungsi untuk mengekstraksi fitur visual dari citra wajah secara hierarkis. Seluruh lapisan konvolusi pada blok-blok ini menggunakan parameter padding = same untuk mempertahankan dimensi spasial citra pada bagian tepinya selama proses ekstraksi fitur. Setiap lapisan konvolusi juga menggunakan fungsi aktivasi ReLU untuk memperkenalkan sifat non-linear pada model, serta regularisasi L2 sebesar 0,001 untuk membantu mengurangi risiko overfitting.

Pada tahap awal, Blok Konvolusi 1 dikonfigurasi menggunakan 32 filter berukuran 3×3 dan diikuti oleh lapisan Max Pooling berukuran 2×2 untuk mereduksi dimensi spasial fitur. Selanjutnya, Blok Konvolusi 2 menggunakan 64 filter berukuran 3×3 dengan konfigurasi aktivasi, padding, dan regularisasi yang sama, kemudian kembali diikuti oleh Max Pooling 2×2 . Proses ekstraksi berlanjut pada Blok Konvolusi 3 yang juga menggunakan 64 filter berukuran 3×3 , lalu diakhiri dengan Max Pooling 2×2 . Pada Blok Konvolusi 4, jumlah filter ditingkatkan menjadi 128 dengan ukuran kernel 3×3 untuk menangkap fitur-fitur yang lebih kompleks dan lebih abstrak pada level jaringan yang lebih dalam, kemudian kembali diikuti oleh Max Pooling 2×2 .

Berbeda dengan penjelasan sebelumnya, pada implementasi model ini tidak diterapkan dropout pada setiap blok konvolusi. Setelah seluruh proses ekstraksi fitur selesai, hasil keluaran dari blok konvolusi terakhir diubah menjadi vektor satu dimensi menggunakan lapisan Flatten. Selanjutnya diterapkan Dropout_CNN sebesar 0,50 sebagai mekanisme regularisasi sebelum data diteruskan ke bagian fully connected layer. Dengan susunan tersebut, proses ekstraksi fitur pada model baseline tetap berfokus pada kombinasi empat lapisan konvolusi dan max pooling, sedangkan regularisasi utama diterapkan setelah tahap flattening sebelum masuk ke lapisan klasifikasi.

3.3.3 Arsitektur Klasifikasi (*Classification Layer*)

Setelah melewati tahap ekstraksi fitur, peta fitur (*feature map*) hasil dari blok konvolusi terakhir diproses pada lapisan klasifikasi dengan urutan sebagai berikut:

1. *Flatten*: Mengubah data matriks multidimensi dari hasil *pooling* terakhir menjadi vektor satu dimensi (vector) agar dapat diproses oleh jaringan saraf tiruan (Dense Layer).
2. *Dense Layer (Hidden)*: Lapisan tersembunyi yang terdiri dari 128 neuron dengan fungsi aktivasi *ReLU*. Lapisan ini juga dilengkapi dengan regularisasi L2 (**0.0001**) dan diikuti oleh lapisan Dropout sebesar 0.20.
3. *Output Layer*: Lapisan akhir yang terdiri dari 5 neuron, merepresentasikan jumlah kelas target (*Child, Teenager, Adult, Middle Age, Old*). Fungsi aktivasi *Softmax*

digunakan pada lapisan ini untuk menghasilkan *output* berupa probabilitas prediksi untuk setiap kelas, di mana total probabilitas dari kelima kelas akan berjumlah 1.

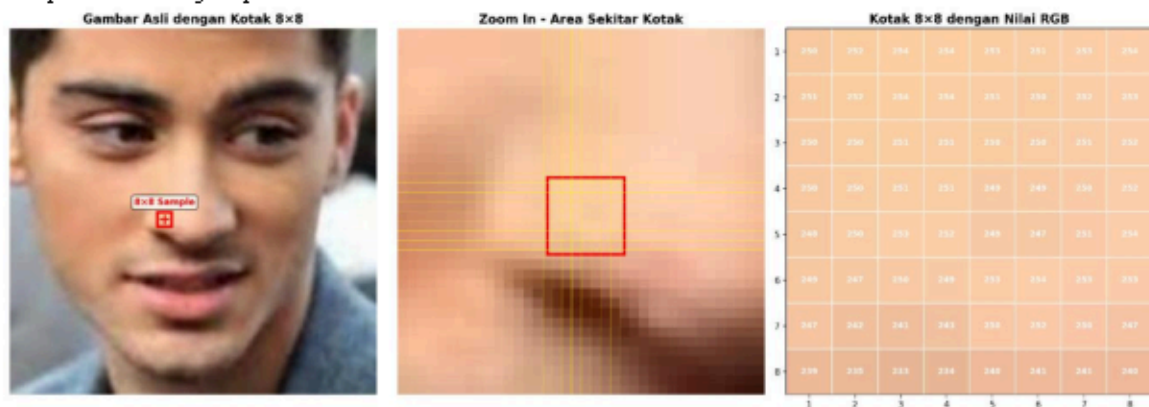
Model ini dikompilasi menggunakan pengoptimal (*optimizer*) Adam dengan laju pembelajaran (*learning rate*) sebesar **0,00092** dan fungsi kerugian (*loss function*) Categorical Crossentropy.

3.4 Simulasi Perhitungan Manual CNN

Setelah desain arsitektur dipaparkan pada sub-bab 3.3, bagian ini menyajikan simulasi perhitungan manual untuk memvalidasi alur pemrosesan data di dalam model. Simulasi dilakukan dengan menelusuri transformasi nilai piksel dari tahap input hingga *output* klasifikasi menggunakan sampel data statis. Tujuannya adalah untuk membuktikan secara matematis bahwa operasi konvolusi, aktivasi, dan *pooling* yang dirancang berjalan sesuai dengan kaidah teoritis CNN sebelum dioptimasi menggunakan algoritma PSO.

3.4.1 Normalisasi

Citra digital pada umumnya disimpan dalam format integer 8-bit dengan rentang nilai intensitas piksel antara 0 hingga 255. Namun, rentang nilai yang besar ini dapat memperlambat proses konvergensi model CNN saat melakukan optimasi gradien (*gradient descent*). Oleh karena itu, diperlukan proses normalisasi untuk mengubah skala data (*rescaling*) menjadi rentang **0.0** hingga **1.0**. Berikut adalah matriks 8x8 yang diambil dari sampel citra wajah pada Gambar 3.5.



Gambar 3.5 Red Channel Citra Wajah

Proses normalisasi dilakukan dengan membagi setiap nilai piksel dengan angka 255 agar rentang nilainya berada di antara 0 hingga 1. Berikut adalah rincian proses perhitungannya:

Tabel 3.6 Perhitungan Normalisasi

No.	Perhitungan Normalisasi	Hasil
1.	$250 / 255$	0.980
2.	$252 / 255$	0.988
3.	$254 / 255$	0.996
4.	$254 / 255$	0.996
...	...	...

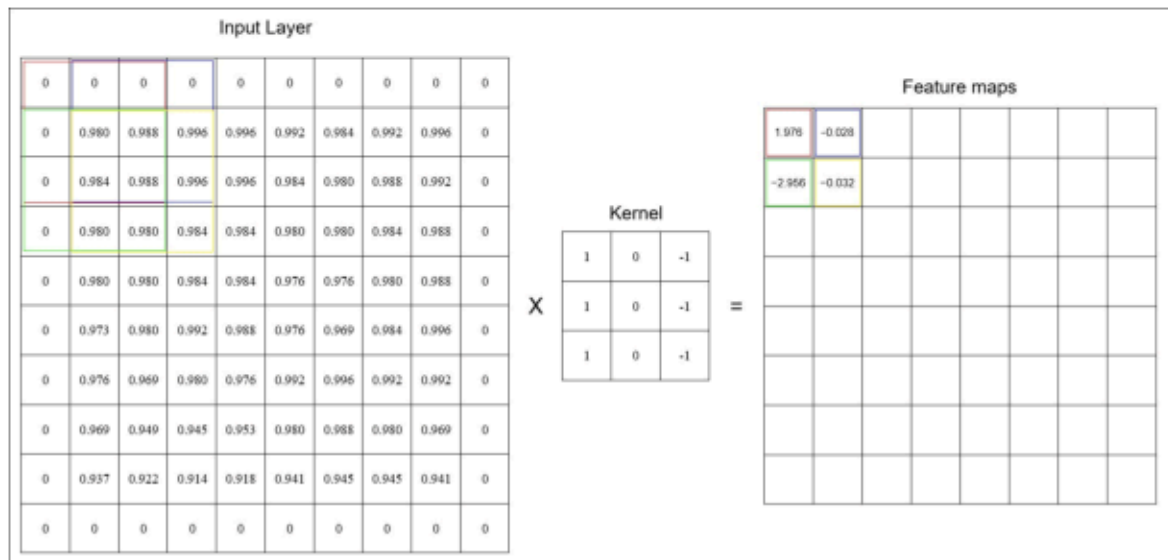
Setelah dilakukan proses normalisasi pada setiap piksel, diperoleh matriks baru yang disajikan pada Gambar 3.6 berikut.

0.980	0.988	0.996	0.996	0.992	0.984	0.992	0.996
0.984	0.988	0.996	0.996	0.984	0.980	0.988	0.992
0.980	0.980	0.984	0.984	0.980	0.980	0.984	0.988
0.980	0.980	0.984	0.984	0.976	0.976	0.980	0.988
0.973	0.980	0.992	0.988	0.976	0.969	0.984	0.996
0.976	0.969	0.980	0.976	0.992	0.996	0.992	0.992
0.969	0.949	0.945	0.953	0.980	0.988	0.980	0.969
0.937	0.922	0.914	0.918	0.941	0.945	0.945	0.941

Gambar 3.6 Hasil Normalisasi

3.4.2 *Convolutional Layer*

Convolutional Layer merupakan komponen fundamental dalam arsitektur *Convolutional Neural Network* (CNN) yang berfungsi sebagai pengekstraksi fitur utama. Tahapan ini bekerja dengan menerapkan operasi konvolusi menggunakan kernel atau filter berukuran 3×3 piksel terhadap citra input yang direpresentasikan sebagai matriks. Melalui mekanisme ini, model dapat mengidentifikasi pola-pola spasial esensial, mulai dari deteksi tepi (edge detection) hingga tekstur visual yang kompleks. Ilustrasi proses konvolusi tersebut disajikan pada Gambar 3.7 berikut.



Gambar 3.7 Proses Konvolusi

Berdasarkan visualisasi pada Gambar 3.7, terlihat bahwa proses konvolusi diawali dengan penambahan nilai nol (*zero padding*) di sekeliling matriks input ternormalisasi. Teknik ini diterapkan untuk mempertahankan dimensi spasial citra agar tetap berukuran 8×8 piksel setelah operasi konvolusi. Kotak-kotak berwarna pada gambar merepresentasikan posisi kernel (sliding window) saat melakukan operasi perkalian titik (dot product). Rincian perhitungan matematis untuk setiap posisi pergeseran kernel tersebut diuraikan secara lengkap pada Tabel 3.7 berikut.

Tabel 3.7 Perhitungan konvolusi

No.	Perhitungan Konvolusi	Hasil
1.	$(0 * 1) + (0 * 0) + (0 * -1) (0 * 1) + (0.980 * 0) + (0.988 * -1) (0 * 1) + (0.984 * 0) + (0.988 * -1)$	-1.976
2.	$(0 * 1) + (0 * 0) + (0 * -1) (0.980 * 1) + (0.988 * 0) + (0.996 * -1) (0.984 * 1) + (0.988 * 0) + (0.996 * -1)$	-0.028
3.	$(0 * 1) + (0.980 * 0) + (0.988 * -1) (0 * 1) + (0.984 * 0) + (0.988 * -1) (0 * 1) + (0.980 * 0) + (0.980 * -1)$	-2.956
4.	$(0.980 * 1) + (0.988 * 0) + (0.996 * -1) (0.984 * 1) + (0.988 * 0) + (0.996 * -1) (0.980 * 1) + (0.980 * 0) + (0.984 * -1)$	-0.032
...	...	...

Setelah dilakukan proses konvolusi pada setiap piksel, diperoleh matriks baru yang disajikan pada Gambar 3.8 berikut.

-1.976	-0.028	-0.016	0.016	0.028	-0.004	-0.024	1.980
-2.956	-0.032	-0.020	0.020	0.032	-0.008	-0.032	2.964
-2.948	-0.020	-0.016	0.024	0.028	-0.012	-0.032	2.952
-2.940	-0.027	-0.016	0.028	0.031	-0.016	-0.047	2.948
-2.929	-0.027	-0.019	0.012	0.007	-0.012	-0.035	2.956
-2.898	0.001	-0.019	-0.031	-0.036	-0.008	-0.004	2.956
-2.840	0.043	-0.007	-0.074	-0.082	-0.004	0.027	2.917
-1.871	0.047	0.000	-0.062	-0.062	-0.004	0.023	1.925

Gambar 3.8 Hasil Konvolusi

3.4.3 Fungsi Aktivasi (*ReLU*)

Setelah operasi konvolusi, peta fitur yang dihasilkan masih bersifat linear dan mengandung nilai negatif. Untuk memperkenalkan *non-linearitas* ke dalam model dan mempercepat proses pelatihan, digunakan fungsi *aktivasi Rectified Linear Unit* (ReLU). Fungsi ini bekerja secara sederhana dengan mengubah semua nilai negatif menjadi 0, sementara nilai positif dibiarkan tetap.

Contoh dipiksel (1,1) dan (2,1):

$$(1,1): \max(0, -1.976) = 0$$

$$(2,1): \max(0, -2.956) = 0$$

Penerapan fungsi ini pada seluruh elemen matriks menghasilkan peta fitur teraktivasi (*Activated Feature Map*) seperti yang disajikan pada Gambar 3.9 berikut. Terlihat bahwa nilai-nilai negatif telah dieliminasi, menyisakan fitur-fitur positif yang signifikan.

0	0	0	0.016	0.028	0	0	1.980
0	0	0	0.020	0.032	0	0	2.964
0	0	0	0.024	0.028	0	0	2.952
0	0	0	0.028	0.031	0	0	2.948
0	0	0	0.012	0.007	0	0	2.956
0	0.001	0	0	0	0	0	2.956
0	0.043	0	0	0	0	0.027	2.917
0	0.047	0	0	0	0	0.023	1.925

Gambar 3.9 Hasil Aktivasi ReLU

3.4.4 Reduksi Dimensi (*Max Pooling*)

Setelah melalui tahap aktivasi, peta fitur berukuran 8×8 piksel diproses oleh lapisan Pooling. Penelitian ini menggunakan metode Max Pooling dengan ukuran filter 2×2 dan stride 2. Metode ini bekerja dengan membagi citra menjadi blok-blok non-overlapping berukuran 2×2 piksel, kemudian mengambil nilai maksimum dari setiap blok tersebut.

Tujuan utama dari tahapan ini adalah untuk mereduksi dimensi spasial data sehingga mengurangi beban komputasi pada lapisan berikutnya, sekaligus mempertahankan informasi fitur yang paling dominan (invarian terhadap pergeseran kecil).

Visualisasi proses pengambilan nilai maksimum pada sampel matriks hasil *ReLU* disajikan sebagai berikut:

1. Sampel Blok 1 (Area Kosong) Mengambil area pojok kiri atas (Baris 1-2, Kolom 1-2) yang berisi nilai 0 (area *padding/background*).

$$\max \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} = 0$$

2. Sampel Blok 2 (Area Fitur Tepi) Mengambil area di mana garis vertikal terdeteksi (Baris 1-2, Kolom 3-4).

$$\max \begin{bmatrix} 0 & 0.016 \\ 0 & 0.020 \end{bmatrix} = 0.020$$

Dengan menerapkan operasi ini pada seluruh bagian matriks 8×8 , dimensi data berkurang setengahnya menjadi 4×4 . Hasil akhir dari proses simulasi manual pada bab ini ditampilkan pada Gambar 3.10 berikut.

0.000	0.020	0.032	2.964
0.000	0.028	0.031	2.952
0.001	0.012	0.007	2.956
0.047	0.000	0.000	2.917

Gambar 3.10 Hasil *Max Pooling*

3.4.5 Perataan Data (*Flattening*)

Setelah proses ekstraksi fitur selesai, langkah selanjutnya adalah mengubah data multidimensi menjadi vektor satu dimensi agar dapat diproses oleh *Fully Connected Layer* (*Dense Layer*). Proses ini disebut dengan *Flattening*. Dalam simulasi ini, matriks hasil Max Pooling berukuran 4×4 diratakan dengan cara menyusun baris-baris matriks secara berurutan menjadi satu vektor panjang.

$$\text{Vector} = [\text{Baris}_1, \text{Baris}_2, \text{Baris}_3, \text{Baris}_4]$$

Berdasarkan data pada tabel sebelumnya, hasil transformasi *flattening* adalah sebagai berikut:

$$[0.000, 0.020, 0.032, 2.964, 0.000, 0.028, 0.031, 2.952, 0.001, 0.012, 0.007, 2.956, 0.047, 0.000, 0.000, 2.917]$$

Vektor V dengan panjang 16 elemen ini merepresentasikan fitur-fitur garis tepi vertikal yang telah diekstraksi dari citra wajah.

3.4.6 Klasifikasi dan *Output* (*Softmax*)

Vektor V tersebut diproses oleh lapisan Dense untuk menghasilkan nilai logit (z) bagi setiap kelas usia. Mengingat kuatnya nilai fitur input (≈ 2.9), neuron yang bertanggung jawab untuk kelas Adult akan menghasilkan skor aktivasi yang tinggi.

Berikut adalah simulasi nilai *logit* yang dihasilkan:

1. $z_{Adult} = 3.8$ (Skor sangat tinggi karena fitur input kuat)
2. $z_{Teenager} = 1.2$
3. $z_{Child} = 0.5$
4. $z_{Old} = 0.1$
5. $z_{MiddleAge} = 0.3$

Selanjutnya, probabilitas prediksi dihitung menggunakan fungsi *Softmax*:

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Hitung Nilai Eksponensial (e^z):

$$e^{3.8} \approx 44.70$$

$$e^{1.2} \approx 3.32$$

$$e^{0.5} \approx 1.65$$

$$e^{0.1} \approx 1.10$$

$$e^{0.3} \approx 1.35$$

Total Penjumlahan (Σ): $44.70 + 3.32 + 1.65 + 1.10 + 1.35 = 52.12$

Hitung Probabilitas Akhir:

$$P(Adult): 44.70 / 52.12 \approx 0.858 \text{ (85.8\%)}$$

$$P(Teenager): 3.32 / 52.12 \approx 0.064 \text{ (6.4\%)}$$

$$P(Child): 1.65 / 52.12 \approx 0.032 \text{ (3.2\%)}$$

$$P(Middle Age): 1.35 / 52.12 \approx 0.026 \text{ (2.6\%)}$$

$$P(Old): 1.10 / 52.12 \approx 0.021 \text{ (2.1\%)}$$

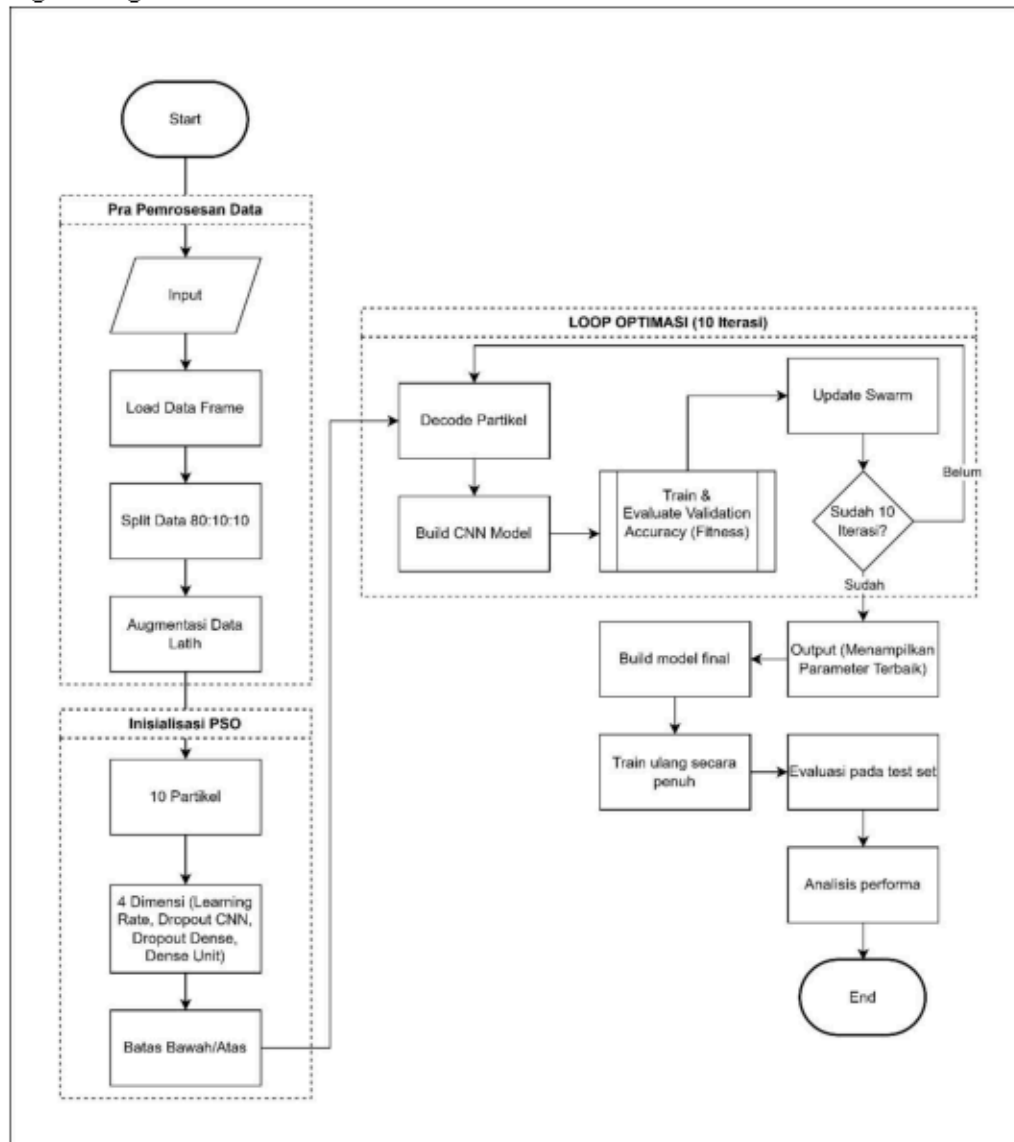
Dengan adanya penguatan sinyal fitur pada tahap *Flattening* (nilai fitur mencapai 2.9), model mampu memprediksi citra masukan sebagai kelas *Adult* dengan tingkat probabilitas yang sangat dominan, yaitu 85.8%. Hal ini menunjukkan bahwa arsitektur CNN mampu menangkap dan memperkuat pola visual penting dari citra wajah untuk menghasilkan keputusan klasifikasi yang akurat.

3.5 Penerapan Arsitektur PSO

Dalam upaya meningkatkan kinerja arsitektur CNN, penelitian ini menerapkan algoritma *Particle Swarm Optimization* (PSO) sebagai metode optimasi otomatis untuk penentuan *hyperparameter*. Mengingat banyaknya kombinasi parameter yang mungkin dalam konfigurasi jaringan saraf tiruan, pencarian parameter secara manual (*trial and error*) dinilai tidak efisien dan berpotensi menghasilkan model yang kurang optimal. Oleh karena itu, PSO digunakan untuk menelusuri ruang pencarian solusi secara adaptif guna menemukan konfigurasi yang dapat memaksimalkan akurasi validasi model. Alur kerja sistem optimasi ini, mulai dari inisialisasi partikel hingga penentuan parameter terbaik, divisualisasikan dalam diagram alir (*flowchart*) pada Gambar 3.11 di bawah.

Sebagaimana diilustrasikan pada Gambar 3.11, proses optimasi dimulai dengan membangkitkan populasi partikel yang masing-masing membawa kandidat konfigurasi parameter. Setiap partikel kemudian dievaluasi kinerjanya melalui siklus pelatihan model

CNN dan bergerak memperbarui posisinya menuju solusi terbaik global (Global Best). Rincian teknis mengenai komponen-komponen utama dalam diagram tersebut diuraikan pada bagian-bagian berikut.



Gambar 3.11 Diagram gambaran umum arsitektur PSO

3.5.1 Definisi Ruang Pencarian (*Search Space*)

Setiap partikel pada algoritma PSO direpresentasikan sebagai vektor solusi berdimensi empat ($D=4$), yang secara matematis dinyatakan sebagai:

$$x_i = [x_{i1}, x_{i2}, x_{i3}, x_{i4}]$$

di mana setiap dimensi merepresentasikan satu hyperparameter CNN yang akan dioptimasi. Tiga dimensi pertama bersifat kontinu. Dimensi pertama merepresentasikan LearningRate, dengan rentang kontinu 1×10^{-3} hingga 5×10^{-3} , learning rate mengontrol besar kecilnya langkah pembaruan bobot pada setiap iterasi pelatihan. Dimensi kedua merepresentasikan Dropout CNN, dengan rentang kontinu 0.3 hingga 0.6. Dimensi ketiga

merepresentasikan dropout dense, dengan rentang kontinu 0.1 hingga 0.4. Dimensi keempat merepresentasikan jumlah unit pada dense layer, dengan rentang nilai 64 hingga 256, tidak seperti yang lain, nilai dense unit akan dibulatkan menjadi angka seperti 64, 96, 128, 160, 192, 224, dan 256.

Batas bawah (LB) dan batas atas (UB) untuk keempat dimensi didefinisikan sebagai:

$$LB = [1 \times 10^{-5}, 0.3, 0.1, 64], \quad UB = [5 \times 10^{-3}, 0.6, 0.4, 256]$$

Pemilihan ruang pencarian ini dilakukan secara terkontrol untuk menjaga keseimbangan antara eksplorasi solusi yang memadai dan efisiensi komputasi, dengan mempertimbangkan ukuran citra masukan serta keterbatasan sumber daya komputasi pada lingkungan eksperimen.

3.5.2 Inisialisasi dan Konfigurasi PSO

Proses optimasi diawali dengan menginisialisasi populasi sebanyak 10 partikel yang disebar secara acak di dalam ruang pencarian. Algoritma dijalankan selama 10 iterasi untuk menyeimbangkan antara eksplorasi ruang parameter dan efisiensi waktu komputasi. Dalam pergerakannya, perilaku partikel dikendalikan oleh tiga parameter koefisien: bobot inersia (w) sebesar 0.7 yang memberikan prioritas pada eksplorasi global di awal iterasi, koefisien kognitif (c_1) sebesar 1.5 yang mengatur pengaruh pengalaman terbaik individu partikel, dan koefisien sosial (c_2) sebesar 1.5 yang mengatur pengaruh pengalaman terbaik dari seluruh kawanan.

3.5.3 Fungsi Objektif (*Fitness Function*)

Evaluasi kualitas setiap partikel dilakukan menggunakan fungsi fitness yang bertujuan untuk memaksimalkan akurasi model pada data validasi. PSO yang diimplementasikan bekerja secara langsung dengan prinsip maksimasi, sehingga nilai akurasi validasi digunakan secara langsung sebagai nilai fitness tanpa konversi apapun.

Mekanisme evaluasi dimulai dengan proses decoding, di mana posisi vektor partikel diterjemahkan menjadi nilai hyperparameter konkret meliputi learning rate, dropout CNN, dropout dense, dan dense units. Sebuah model CNN baru kemudian dibangun berdasarkan konfigurasi tersebut dan dilatih menggunakan data latih selama 10 epoch. Pelatihan singkat ini bertujuan untuk mendapatkan estimasi performa awal model tanpa membebani sumber daya komputasi secara berlebihan. Setelah pelatihan parsial selesai, model dievaluasi menggunakan data validasi untuk mendapatkan nilai akurasi validasi tertinggi sepanjang proses pelatihan (Max Validation Accuracy). Nilai akurasi ini langsung digunakan sebagai nilai fitness partikel. Partikel dengan nilai fitness tertinggi akan memperbarui solusi terbaik personal (pBest) maupun solusi terbaik global (gBest), yang nantinya digunakan sebagai konfigurasi hyperparameter final untuk pelatihan model utama secara penuh selama 50 epoch.

3.6 Simulasi Perhitungan Manual Particle Swarm Optimization (PSO)

Setelah desain mekanisme optimasi PSO dipaparkan pada sub-bab sebelumnya, bagian ini menyajikan simulasi perhitungan manual untuk memvalidasi alur proses optimasi

hiperparameter di dalam sistem. Simulasi dilakukan dengan menelusuri proses pergerakan partikel dari tahap inisialisasi hingga pembaruan posisi menggunakan contoh nilai statis yang disederhanakan.

Pada simulasi ini, setiap partikel merepresentasikan kombinasi hiperparameter CNN yang berada dalam ruang pencarian berdimensi empat. Proses perhitungan dilakukan secara bertahap, mulai dari inisialisasi partikel, penentuan nilai personal best (pBest) dan global best (gBest), hingga pembaruan nilai berdasarkan persamaan matematis PSO. Dengan pendekatan ini, alur kerja algoritma dapat diverifikasi secara sistematis dan memastikan implementasi PSO yang ditulis secara manual telah berjalan sesuai dengan persamaan matematis standar PSO.

3.6.1 Representasi Partikel

Pada algoritma Particle Swarm Optimization (PSO), setiap solusi kandidat direpresentasikan sebagai sebuah partikel dalam ruang pencarian multidimensi. Dalam penelitian ini, partikel digunakan untuk merepresentasikan kombinasi hiperparameter pada model Convolutional Neural Network (CNN) yang akan dioptimalkan.

Ruang pencarian yang digunakan terdiri dari empat dimensi, di mana masing-masing dimensi merepresentasikan satu hiperparameter CNN. Adapun daftar kandidat nilai untuk masing-masing hiperparameter adalah sebagai berikut:

- Learning rate: $1 \times 10^{-5} - 5 \times 10^{-3}$
- Dropout cnn: 0.3 – 0.6
- Dropout dense: 0.1 – 0.4
- Dense unit: 64 – 256

Sebagai contoh, apabila suatu partikel memiliki posisi:

$$X1 = (0.001, 0.3, 0.1, 64)$$

maka interpretasinya adalah sebagai berikut:

Nilai 0.001 pada dimensi pertama digunakan langsung sebagai learning rate

Nilai 0.3 pada dimensi kedua digunakan langsung sebagai dropout cnn

Nilai 0.1 pada dimensi ketiga digunakan langsung sebagai dropout dense

Nilai 64 pada dimensi keempat digunakan langsung sebagai dense unit

Dengan demikian, partikel tersebut merepresentasikan satu konfigurasi arsitektur CNN tertentu yang selanjutnya akan dievaluasi menggunakan data pelatihan untuk memperoleh nilai fitness.

Representasi ini memastikan bahwa proses optimasi tetap mengikuti mekanisme kontinu PSO, namun tetap menghasilkan konfigurasi hiperparameter yang valid sesuai dengan batasan arsitektur yang telah ditentukan.

3.6.2 Inisialisasi Partikel dan Kecepatan Awal

Pada tahap awal algoritma Particle Swarm Optimization (PSO), sejumlah partikel dibangkitkan secara acak di dalam ruang pencarian yang telah ditentukan. Setiap partikel memiliki dua komponen utama, yaitu posisi (position) dan kecepatan (velocity). Posisi merepresentasikan kombinasi hiperparameter CNN, sedangkan kecepatan merepresentasikan besar serta arah perubahan nilai hiperparameter pada iterasi berikutnya.

Dalam penelitian ini, batas bawah (lower bound) dan batas atas (upper bound) untuk setiap dimensi ditentukan sebagai berikut:

$$lb = (1 \times 10^{-5}, 0.3, 0.1, 64)$$

$$ub = (5 \times 10^{-3}, 0.6, 0.4, 256)$$

Dimensi pertama hingga ketiga merepresentasikan indeks kandidat hiperparameter diskrit, sedangkan dimensi keempat dan kelima merepresentasikan nilai kontinu untuk dropout dan learning rate.

Sebagai contoh simulasi, misalkan pada iterasi awal ($t = 0$) terdapat satu partikel dengan posisi:

$$X_1^0 = (0.001, 0.3, 0.1, 64)$$

Selain posisi, partikel juga memiliki kecepatan awal yang diinisialisasi secara acak dalam rentang kecil di sekitar nol, misalnya:

$$V_1^0 = (0.0002, 0.02, -0.01, 10.5)$$

Nilai kecepatan ini menentukan bagaimana posisi partikel diperbarui pada iterasi berikutnya. Kecepatan partikel tidak ditentukan secara acak pada setiap iterasi, melainkan dihitung berdasarkan persamaan dasar Particle Swarm Optimization (PSO) yang telah dijelaskan pada rumus (2.18), sehingga pergerakan partikel mengikuti mekanisme inersia, komponen kognitif, dan komponen sosial dalam proses pencarian solusi optimal.

Pada penelitian ini digunakan parameter:

$$w = 0.7, \quad c_1 = 1.5, \quad c_2 = 1.5$$

Misalkan pada dimensi ke-1 (learning rate):

Posisi saat ini (x^t): **0.001**

Velocity sebelumnya (v^t): **0.0002**

Personal Best ($pBest$): **0.0015**

Global Best ($gBest$): **0.0018**

Nilai Random (r): $r_1 = 0.6, \quad r_2 = 0.4$

Maka perhitungan velocity baru:

Komponen Inersia:

$$0.7 \times 0.0002 = 0.00014$$

Komponen Kognitif:

$$1.5 \times 0.6 \times (0.0015 - 0.001) = 0.9 \times 0.0005 = 0.00045$$

Komponen Sosial:

$$1.5 \times 0.4 \times (0.0018 - 0.001) = 0.6 \times 0.0008 = 0.00048$$

Sehingga velocity baru:

$$v^{t+1} = 0.00014 + 0.00045 + 0.00048$$

$$v^{t+1} = 0.00107$$

Pembaruan posisi pada iterasi berikutnya dihitung menggunakan:

$$x^{t+1} = x^t + v^{t+1}$$

$$x^{t+1} = 0.0010 + 0.00107 = 0.00207$$

Hasil ini menunjukkan bahwa nilai learning rate meningkat secara bertahap menuju kombinasi hiperparameter yang lebih optimal berdasarkan pengalaman individu dan global.

Dengan demikian, tahap inisialisasi dan pembaruan velocity memastikan bahwa partikel tidak bergerak secara acak, melainkan mengikuti mekanisme adaptif yang dipengaruhi oleh pengalaman terbaik individu serta populasi secara keseluruhan. Proses ini memungkinkan pencarian solusi optimal dilakukan secara bertahap hingga tercapai konvergensi.

3.6.3 Evaluasi Fitness dan Penentuan pBest serta gBest

Setelah partikel diinisialisasi dan memiliki posisi awal dalam ruang pencarian, langkah berikutnya adalah melakukan evaluasi fitness untuk setiap partikel. Evaluasi fitness bertujuan untuk mengukur seberapa baik kombinasi hiperparameter yang direpresentasikan oleh partikel dalam meningkatkan performa model CNN.

Pada penelitian ini, nilai fitness diperoleh dari hasil pelatihan model CNN menggunakan kombinasi hyperparameter yang dihasilkan oleh partikel. Kinerja model diukur menggunakan akurasi validasi tertinggi selama pelatihan. Karena algoritma PSO yang digunakan bersifat maksimasi, maka nilai fitness didefinisikan langsung sebagai akurasi validasi:

$$Fitness = Accuracy$$

Dengan demikian, Semakin tinggi nilai akurasi validasi, maka semakin optimal partikel tersebut. Setiap partikel menyimpan posisi terbaik yang pernah dicapai selama proses iterasi sebagai personal best (pBest). pBest diperbarui berdasarkan kondisi:

$$\text{Jika } Fitness(X_i^t) > Fitness(pBest_i) \text{ maka } pBest_i = X_i^t$$

Artinya, apabila posisi partikel pada iterasi saat ini menghasilkan nilai fitness yang lebih baik, maka posisi tersebut akan menggantikan pBest sebelumnya.

Sebagai contoh, misalkan:

Iterasi ke-0:

Akurasi partikel 1 = 0.82

Fitness = 0.82

→ $pBest_1 = 0.82$

Iterasi ke-1:

Akurasi partikel 1 = 0.85

Fitness = 0.85

Karena $0.85 > 0.82$, maka:

$$pBest_1 = 0.85$$

Selain pengalaman individu, PSO juga mempertimbangkan pengalaman terbaik dari seluruh populasi partikel. Posisi terbaik dari semua pBest disebut sebagai global best (gBest). Secara matematis:

$$gBest = \arg \max(Fitness(pBest_i))$$

Artinya, gBest adalah posisi partikel yang menghasilkan nilai akurasi validasi tertinggi di antara seluruh partikel dalam populasi. Sebagai contoh, misalkan terdapat tiga partikel dengan nilai pBest sebagai berikut:

$$pBest_1 = 0.85$$

$$pBest_2 = 0.80$$

$$pBest_3 = 0.88$$

Karena 0.88 adalah nilai paling besar, maka:

$$gBest = pBest_3$$

Posisi partikel ke-3 menjadi referensi global bagi seluruh partikel lainnya pada iterasi berikutnya.

Evaluasi fitness menjadi dasar dalam menentukan arah pergerakan partikel pada iterasi berikutnya. Nilai $pBest$ merepresentasikan pengalaman terbaik individu, sedangkan $gBest$ merepresentasikan pengalaman terbaik populasi secara keseluruhan. Kombinasi antara $pBest$ dan $gBest$ pada persamaan pembaruan $velocity$ menyebabkan partikel bergerak menuju solusi yang semakin optimal secara bertahap. Dengan mekanisme ini, proses pencarian solusi tidak dilakukan secara acak, melainkan dipandu oleh pengalaman historis dan interaksi sosial antar partikel.

Dengan demikian, evaluasi fitness serta pembaruan $pBest$ dan $gBest$ menjadi komponen utama yang menghubungkan algoritma PSO dengan performa model CNN dalam penelitian ini.

3.7 Strategi Pelatihan Model

Proses pelatihan model dalam penelitian ini dirancang secara bertahap untuk menghasilkan model yang memiliki performa baik pada data latih serta kemampuan generalisasi yang baik pada data baru. Strategi pelatihan dilakukan dengan membandingkan model CNN baseline dan model CNN dengan optimasi Particle Swarm Optimization (PSO) pada dua skenario pengelompokan usia, yaitu skenario 4 kelas dan skenario 5 kelas. Dengan demikian, terdapat empat skenario pelatihan yang digunakan dalam penelitian ini.

Skenario pertama adalah pelatihan model CNN baseline pada pengelompokan usia 4 kelas berdasarkan jurnal acuan, yaitu Child, Teenager, Adult, dan Old. Skenario kedua adalah pelatihan model CNN dengan optimasi PSO pada pengelompokan usia 4 kelas. Skenario ketiga adalah pelatihan model CNN baseline pada pengelompokan usia 5 kelas, yaitu Child, Teenager, Adult, Middle Age, dan Old. Skenario keempat adalah pelatihan model CNN dengan optimasi PSO pada pengelompokan usia 5 kelas. Keempat skenario tersebut digunakan untuk menganalisis pengaruh optimasi PSO terhadap performa CNN sekaligus melihat dampak perubahan jumlah kelas terhadap tingkat kesulitan klasifikasi usia.

Pada tahap pelatihan model CNN baseline, model dibangun dan dilatih menggunakan konfigurasi awal yang telah ditetapkan, yaitu learning rate sebesar 0,00092, dropout CNN sebesar 0,50, dropout dense sebesar 0,20, dan dense units sebesar 128. Konfigurasi ini digunakan sebagai pembanding awal sebelum dilakukan optimasi hyperparameter menggunakan PSO. Model baseline dilatih menggunakan data latih dan dipantau menggunakan data validasi untuk memperoleh performa awal pada masing-masing skenario pengelompokan usia.

Dalam proses pelatihan, model menggunakan fungsi kerugian Categorical Cross-entropy karena penelitian ini merupakan permasalahan klasifikasi multi-kelas. Fungsi kerugian ini digunakan untuk mengukur perbedaan antara hasil prediksi model dengan label sebenarnya yang telah direpresentasikan dalam bentuk one-hot encoding. Semakin kecil nilai loss yang diperoleh, maka semakin baik kemampuan model dalam menyesuaikan prediksi terhadap label sebenarnya.

Proses pembaruan bobot pada model dilakukan menggunakan optimizer Adam atau Adaptive Moment Estimation. Optimizer Adam dipilih karena mampu menyesuaikan laju pembelajaran secara adaptif selama proses pelatihan, sehingga proses konvergensi model dapat berjalan lebih stabil dan efisien. Pada penelitian ini, nilai learning rate menjadi salah satu hyperparameter yang dioptimasi menggunakan PSO karena nilai tersebut berpengaruh terhadap kecepatan dan kestabilan proses pembelajaran model. Learning rate yang terlalu kecil dapat menyebabkan proses pelatihan berjalan lambat, sedangkan learning rate yang terlalu besar dapat menyebabkan model sulit mencapai titik konvergensi yang optimal.

Selain penggunaan loss function dan optimizer, proses pelatihan model juga dilengkapi dengan mekanisme kendali pelatihan menggunakan callback. Callback digunakan untuk memantau performa model selama proses pelatihan, khususnya berdasarkan nilai validation accuracy dan validation loss. Mekanisme ini membantu proses pelatihan agar lebih terkontrol, misalnya dengan menyimpan model terbaik selama pelatihan, menghentikan pelatihan apabila performa validasi tidak lagi mengalami peningkatan, atau menyesuaikan laju pembelajaran ketika proses konvergensi mulai melambat. Dengan adanya callback, model yang digunakan tidak hanya bergantung pada hasil epoch terakhir, tetapi dapat dipilih berdasarkan performa validasi terbaik selama proses pelatihan.

Pada tahap optimasi, algoritma PSO digunakan untuk mencari kombinasi hyperparameter terbaik yang terdiri atas learning rate, dropout CNN, dropout dense, dan jumlah unit pada dense layer. Setiap partikel dalam PSO merepresentasikan satu kombinasi hyperparameter yang dievaluasi menggunakan nilai validation accuracy sebagai fungsi fitness. Dalam proses ini, model dilatih secara singkat untuk menilai kualitas masing-masing kandidat solusi. Nilai validation accuracy dipilih sebagai fungsi fitness karena metrik tersebut menunjukkan kemampuan model dalam mengklasifikasikan data validasi secara benar, sehingga dapat digunakan sebagai dasar untuk menentukan kualitas kombinasi hyperparameter yang diuji.

Proses optimasi PSO dilakukan secara terpisah pada skenario 4 kelas dan skenario 5 kelas. Hal ini dilakukan karena perbedaan jumlah kelas dapat memengaruhi tingkat kesulitan klasifikasi dan kombinasi hyperparameter terbaik yang dibutuhkan oleh model. Dengan demikian, konfigurasi hyperparameter terbaik pada skenario 4 kelas tidak langsung diasumsikan sebagai konfigurasi terbaik untuk skenario 5 kelas, begitu pula sebaliknya.

Setelah diperoleh kombinasi hyperparameter terbaik melalui proses optimasi PSO, model CNN kemudian dilatih ulang atau retraining secara penuh menggunakan konfigurasi hasil optimasi tersebut. Proses retraining dilakukan agar model dapat mempelajari pola data secara lebih optimal berdasarkan hyperparameter terbaik yang telah ditemukan. Hasil

pelatihan ulang ini selanjutnya digunakan sebagai model CNN + PSO untuk dibandingkan dengan model CNN baseline pada masing-masing skenario.

Perbandingan dilakukan untuk mengetahui apakah optimasi hyperparameter menggunakan PSO mampu meningkatkan performa model CNN dalam klasifikasi kelompok usia berdasarkan citra wajah. Selain itu, perbandingan antara skenario 4 kelas dan 5 kelas juga digunakan untuk menganalisis pengaruh penambahan kelas Middle Age terhadap performa klasifikasi. Dengan strategi ini, hasil penelitian tidak hanya menunjukkan perbedaan performa antara CNN baseline dan CNN + PSO, tetapi juga menunjukkan bagaimana perubahan struktur kelas dapat memengaruhi tingkat kesulitan model dalam mengenali kelompok usia.

3.8 Prosedur Evaluasi

Tahapan evaluasi dilakukan untuk mengukur performa model CNN hasil optimasi PSO dalam mengenali pola citra wajah pada data baru yang tidak pernah dipelajari sebelumnya. Proses ini melibatkan pengujian model pada himpunan data uji (*test set*) dan penghitungan metrik kuantitatif berdasarkan hasil prediksi.

3.8.1 Skenario Pengujian

Pengujian dilakukan menggunakan 10% dari total dataset yang telah dipisahkan secara *stratified* pada tahap awal. Berbeda dengan fase pelatihan, data uji ini tidak melalui proses augmentasi geometri (seperti rotasi atau *flip*), namun tetap melalui proses normalisasi piksel ($1/255$) agar sesuai dengan format masukan jaringan.

Mekanisme pengujian diawali dengan memasukkan seluruh citra uji ke dalam model untuk mendapatkan probabilitas kelas dari lapisan *Softmax*. Probabilitas tertinggi kemudian dikonversi menjadi label prediksi definitif menggunakan fungsi *Argmax*. Label hasil prediksi ini selanjutnya disandingkan dengan label sebenarnya (*ground truth*) untuk dianalisis ketepatannya.

3.8.2 Analisis Matriks Kebingungan (*Confusion Matrix*)

Confusion matrix disusun sesuai dengan jumlah kelas pada masing-masing skenario. Pada skenario 4 kelas, confusion matrix berukuran 4×4 yang merepresentasikan kelas Child, Teenager, Adult, dan Old. Pada skenario 5 kelas, confusion matrix berukuran 5×5 yang merepresentasikan kelas Child, Teenager, Adult, Middle Age, dan Old.

Matriks ini dihasilkan menggunakan fungsi *confusion_matrix* dari pustaka *Scikit-learn*. Dalam implementasinya, sumbu vertikal matriks merepresentasikan kelas aktual, sedangkan sumbu horizontal merepresentasikan kelas prediksi. Melalui pemetaan ini, analisis tidak hanya berfokus pada jumlah prediksi benar (diagonal utama), tetapi juga dapat mengidentifikasi kecenderungan *misclassification* spesifik, misalnya seberapa sering citra *Adult* keliru diprediksi sebagai *Middle Age*. Hasil matriks kemudian divisualisasikan dalam bentuk *heatmap* untuk mempermudah pembacaan pola kesalahan model.

3.8.3 Perhitungan Metrik Performansi

Berdasarkan nilai *True Positive* (TP), *False Positive* (FP), *True Negative* (TN), dan *False Negative* (FN) yang dihasilkan dari matriks kebingungan tersebut, kinerja model dihitung menggunakan empat parameter evaluasi utama:

1. Akurasi (*Accuracy*): Dihitung untuk mengetahui rasio ketepatan prediksi global terhadap keseluruhan data uji.
2. Presisi (*Precision*): Dihitung untuk mengukur tingkat keandalan model saat memprediksi suatu kelas positif, guna meminimalkan positif palsu.
3. *Recall* (Sensitivitas): Dihitung untuk mengukur kemampuan model dalam menjangkau seluruh sampel yang relevan dalam suatu kelas.
4. *F1-Score*: Dihitung sebagai rata-rata harmonik antara presisi dan *recall*, yang digunakan sebagai acuan utama apabila terdapat ketidakseimbangan jumlah sampel antar kelas usia.

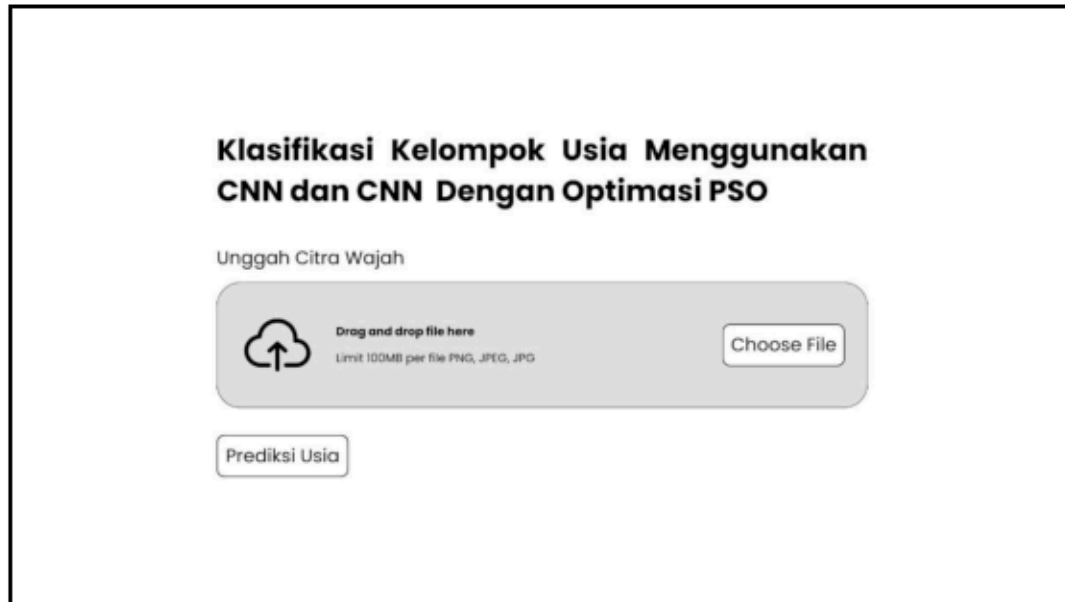
Hasil perhitungan keempat metrik ini kemudian direkapitulasi ke dalam tabel laporan klasifikasi (*Classification Report*) untuk memberikan gambaran performa model secara komprehensif pada setiap kategori usia.

3.9 Implementasi Antarmuka Pengujian Model

Antarmuka pengguna (User Interface) pada penelitian ini dirancang sebagai alat bantu untuk melakukan pengujian model klasifikasi usia yang telah dikembangkan. Antarmuka ini tidak menjadi fokus utama penelitian, melainkan digunakan untuk mempermudah proses validasi hasil prediksi model terhadap citra wajah secara statis.

Antarmuka dibangun dengan pendekatan sederhana yang memisahkan antara tahap masukan data dan tahap penyajian hasil. Secara umum, antarmuka terdiri dari dua tampilan utama, yaitu halaman input dan halaman output.

Halaman input berfungsi sebagai media bagi pengguna untuk memasukkan data citra wajah yang akan diuji. Pada halaman ini, pengguna dapat memilih file citra dari penyimpanan lokal melalui fitur unggah (upload) yang tersedia. Tampilan halaman input dapat dilihat pada Gambar 3.12.



Gambar 3.12 Rancangan Halaman Input Aplikasi

Sebagaimana ditunjukkan pada Gambar 3.12, antarmuka dirancang secara sederhana dengan menampilkan area unggah citra yang mendukung format file seperti PNG, JPG, dan JPEG. Setelah citra dipilih, pengguna dapat menekan tombol "Prediksi Usia" untuk memulai proses klasifikasi. Tombol ini berfungsi untuk mengirimkan citra ke dalam sistem agar diproses oleh model CNN yang telah dilatih.

Setelah proses klasifikasi selesai, sistem akan menampilkan hasil prediksi pada halaman output. Tampilan halaman output dapat dilihat pada Gambar 3.13.

Klasifikasi Kelompok Usia Menggunakan CNN dan CNN Dengan Optimasi PSO

Unggah Citra Wajah

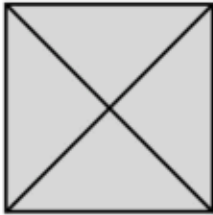


Drag and drop file here

Limit 100MB per file PNG, JPEG, JPG

Choose File

Prediksi Usia



Hasil Deteksi

CNN

Child: 10%
Teenager: 10%
Adult: 70%
Old: 10%
Result: Adult

CNN + PSO

Child: 10%
Teenager: 10%
Adult: 70%
Old: 10%
Result: Adult

Gambar 3.13 Rancangan Halaman Output Aplikasi

Berdasarkan Gambar 3.13, halaman ini menampilkan citra yang telah diproses beserta hasil klasifikasi dari dua model, yaitu model baseline dan model CNN yang telah dioptimasi menggunakan PSO. Hasil yang ditampilkan mencakup label kategori usia serta nilai probabilitas (confidence score) dari masing-masing model.

Penyajian hasil dilakukan secara berdampingan untuk memudahkan proses analisis dan perbandingan kinerja antara kedua model. Selain itu, sistem juga menampilkan hasil akhir prediksi berupa kategori usia dominan yang dihasilkan oleh masing-masing model.

3.10 Implementasi Model dalam Aplikasi Realtime

Implementasi ini bertujuan untuk menguji kemampuan model dalam melakukan prediksi secara langsung menggunakan input dari perangkat kamera. Sistem realtime yang dikembangkan memanfaatkan perangkat kamera (webcam) sebagai sumber input data citra wajah. Alur kerja sistem dimulai dari proses pengambilan frame secara kontinu dari kamera, kemudian dilanjutkan dengan proses deteksi wajah pada setiap frame. Area wajah yang terdeteksi selanjutnya diproses melalui tahap preprocessing sebelum diberikan sebagai input ke model CNN.

Setelah preprocessing, citra wajah akan diklasifikasikan menggunakan model CNN yang telah dilatih, baik model baseline maupun model yang telah dioptimasi menggunakan Particle Swarm Optimization (PSO). Hasil klasifikasi berupa kategori kelompok usia kemudian ditampilkan secara langsung pada layar.

Model CNN hasil pelatihan disimpan dalam format file .keras dan diintegrasikan ke dalam aplikasi menggunakan framework TensorFlow/Keras. Model ini digunakan untuk melakukan proses inferensi terhadap setiap frame citra wajah yang diperoleh dari kamera.

Pada tahap ini, model tidak mengalami proses pelatihan ulang, melainkan hanya digunakan untuk melakukan prediksi berdasarkan bobot terbaik yang telah diperoleh sebelumnya.

Implementasi sistem realtime dikembangkan menggunakan beberapa perangkat lunak pendukung, antara lain:

1. Python sebagai bahasa pemrograman utama
2. OpenCV untuk pengolahan citra dan pengambilan frame dari kamera
3. TensorFlow/Keras untuk proses pemuatan model dan inferensi

Implementasi realtime dalam penelitian ini difokuskan pada pengujian fungsionalitas model dalam kondisi nyata. Oleh karena itu, aspek performa sistem seperti kecepatan pemrosesan (frame rate), latensi, dan efisiensi komputasi tidak dianalisis secara mendalam. Sistem dikembangkan sebagai bentuk demonstrasi penerapan model, bukan sebagai produk siap pakai.

BAB IV

HASIL PENGUJIAN DAN PEMBAHASAN

4.1 Implementasi

Pada tahap ini, dilakukan implementasi sistem berdasarkan perancangan yang telah dijelaskan pada Bab III. Implementasi mencakup seluruh proses mulai dari pengolahan data, pembangunan model Convolutional Neural Network (CNN), penerapan optimisasi menggunakan Particle Swarm Optimization (PSO), hingga implementasi model ke dalam aplikasi klasifikasi usia berbasis realtime. Tahap ini bertujuan untuk merealisasikan rancangan sistem ke dalam bentuk program yang dapat dijalankan serta memastikan bahwa setiap komponen yang telah dirancang dapat berfungsi sesuai dengan tujuan penelitian.

4.1.1 Pengumpulan Data

Pada penelitian ini, data yang digunakan berasal dari dataset UTKFace, yaitu dataset citra wajah yang memiliki informasi atribut berupa usia, jenis kelamin, dan ras. Dataset ini dipilih karena menyediakan label usia yang lengkap dan bervariasi, sehingga sesuai untuk digunakan dalam proses klasifikasi kelompok usia.

Proses pengumpulan data dilakukan dengan memanfaatkan dataset UTKFace yang tersedia dalam bentuk file citra dengan format penamaan tertentu, di mana informasi usia dapat diperoleh langsung dari nama file. Setiap file citra memiliki format penamaan yang mengandung nilai usia, sehingga memungkinkan dilakukan ekstraksi label secara otomatis tanpa perlu proses anotasi manual. Pada penelitian ini digunakan dua skenario pembagian kelompok usia, yaitu skenario pertama dengan 4 kelas yang terdiri dari umur 0–12 tahun sebanyak 3.413 data, umur 13–18 tahun sebanyak 1.082 data, umur 19–40 tahun sebanyak 12.505 data, dan umur lebih dari 40 tahun sebanyak 6.706 data. Sementara itu, skenario kedua menggunakan 5 kelas dengan menambahkan kategori middle age sebagai kelompok usia tersendiri. Pengelompokan ini dilakukan untuk mengubah permasalahan dari regresi usia menjadi klasifikasi kelompok usia. Distribusi data tersebut menunjukkan adanya ketidakseimbangan data (class imbalance), di mana kelas umur 19–40 tahun memiliki jumlah data yang jauh lebih besar dibandingkan kelas lainnya, khususnya kelas umur 13–18 tahun yang memiliki jumlah data paling sedikit.

Ketidakseimbangan ini berpotensi menyebabkan model cenderung bias terhadap kelas mayoritas, sehingga mempengaruhi performa model dalam mengenali kelas dengan jumlah data yang lebih sedikit. Setelah proses pengelompokan, dataset dibagi menjadi tiga bagian, yaitu data latih (training set), data validasi (validation set), dan data uji (test set) dengan rasio pembagian sebesar 80:10:10. Pembagian ini bertujuan untuk melatih model, melakukan validasi selama proses pelatihan, serta mengevaluasi kinerja model secara objektif menggunakan data yang tidak pernah dilihat sebelumnya.

Proses pengumpulan dan pembagian data dilakukan secara otomatis menggunakan program, sehingga memastikan konsistensi dan mengurangi kemungkinan kesalahan manual dalam pengolahan dataset.

4.1.2 Implementasi Preprocessing Data

Tahap preprocessing data dilakukan untuk mempersiapkan citra wajah agar sesuai dengan kebutuhan input model CNN. Proses ini mencakup ekstraksi label usia dari nama file, penyesuaian ukuran citra, normalisasi nilai piksel, serta pembagian dataset menjadi data latih, validasi, dan data uji. Seluruh proses dilakukan secara otomatis menggunakan program untuk menjaga konsistensi dan efisiensi pengolahan data.

1. Input Gambar dan Ekstraksi Label Usia

Dataset yang digunakan berupa kumpulan citra wajah dengan format penamaan file yang mengandung informasi usia. Pada tahap ini, sistem membaca setiap file citra dan melakukan ekstraksi nilai usia dari nama file. Nilai usia tersebut kemudian digunakan untuk menentukan label kelas sesuai dengan rentang usia yang telah ditentukan. Proses ini memungkinkan pelabelan data dilakukan secara otomatis tanpa perlu anotasi manual, sehingga lebih efisien dan meminimalisir kesalahan manusia.

Modul Program 4.1 Input Gambar & Ekstraksi Nilai Umur dari Nama File

```
def get_age_label(age: int) -> str:
    if age <= 12: return "child"
        elif age <= 18: return "teenager"
        elif age <= 40: return "adult"
        elif age <= 60: return "Middle Age"
        else: return "old"

def extract_age_from_filename(filename: str) -> tuple:
    try:
        age = int(filename.split("_")[0])
        label = get_age_label(age)
        return age, label
    except (ValueError, IndexError):
        return None, None
```

2. Resize Citra

Setelah label diperoleh, setiap citra wajah diubah ukurannya (resize) menjadi 150 × 150 piksel. Proses ini dilakukan agar seluruh data memiliki dimensi yang seragam dan sesuai dengan input layer pada model CNN. Penyeragaman ukuran citra sangat penting karena model CNN hanya dapat menerima input dengan ukuran yang konsisten. Selain itu, ukuran yang lebih kecil juga membantu mengurangi beban komputasi selama proses pelatihan.

Modul Program 4.2 Resize Gambar

```
import tensorflow as tf
IMG_SIZE = (150, 150)
def resize_image(image_path: str) -> tf.Tensor:
    raw = tf.io.read_file(image_path)
    image = tf.image.decode_jpeg(raw, channels=3)
    image = tf.image.resize(image, IMG_SIZE)
    return image
```

3. Normalisasi Nilai Piksel

Citra yang telah di-resize kemudian dinormalisasi dengan cara mengubah rentang nilai piksel dari 0–255 menjadi 0–1. Proses ini dilakukan dengan membagi setiap nilai piksel dengan 255. Normalisasi bertujuan untuk mempercepat proses pelatihan dan membantu model dalam mencapai konvergensi yang lebih stabil. Tanpa normalisasi, perbedaan skala nilai piksel dapat menyebabkan proses pembelajaran menjadi kurang optimal.

Modul Program 4.3 Normalisasi Piksel

```
def normalize_pixels(image: tf.Tensor) -> tf.Tensor:
    image = tf.cast(image, tf.float32) / 255.0
    return image
```

4. Pembagian Dataset (Data Splitting)

Dataset yang telah diproses selanjutnya dibagi menjadi tiga bagian, yaitu data latih (training set) sebesar 80%, data validasi (validation set) sebesar 10%, data uji (test set) sebesar 10%. Pembagian ini dilakukan secara acak untuk memastikan distribusi data yang merata pada setiap subset. Data latih digunakan untuk melatih model, data validasi digunakan untuk memantau performa model selama pelatihan, sedangkan data uji digunakan untuk mengevaluasi kinerja model secara objektif.

Modul Program 4.4 Split Data (Train/Validation/Test)

```
import os
import shutil
import random

CLASS_NAMES = ["child", "teenager", "adult", "Middle Age",
               "old"]

def prepare_utkface_dataset(dataset_path: str,
                           train_ratio: float = 0.8,
                           val_ratio: float = 0.1):

    for split in ["train", "val", "test"]:
        for cls in CLASS_NAMES:
            os.makedirs(os.path.join(f"./data/{split}", cls),
                        exist_ok=True)

    all_files = [
        f for f in os.listdir(dataset_path)
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ]

    grouped = {cls: [] for cls in CLASS_NAMES}

    for fname in all_files:
        try:
            age = int(fname.split("_")[0])
            label = get_age_label(age)
```

```

        grouped[label].append(os.path.join(dataset_path,
fname))
    except (ValueError, IndexError):
        continue

    for cls, files in grouped.items():
        random.shuffle(files)

        n = len(files)
        n_train = int(n * train_ratio)
        n_val = int(n * val_ratio)

        splits = {
            "train": files[:n_train],
            "val": files[n_train:n_train + n_val],
            "test": files[n_train + n_val:],
        }

        for split, split_files in splits.items():
            dest_dir = os.path.join(f"./data/{split}", cls)
            for src in split_files:
                shutil.copy2(src, os.path.join(dest_dir,
os.path.basename(src)))

    return "./data/train", "./data/val", "./data/test"

```

4.1.3 Implementasi CNN

Pada tahap ini, dilakukan implementasi model Convolutional Neural Network (CNN) sebagai model baseline untuk klasifikasi kelompok usia. Arsitektur model dibangun menggunakan framework TensorFlow Keras dengan menyesuaikan rancangan yang telah dijelaskan pada Bab III.

1. Arsitektur Model CNN

Model CNN yang digunakan terdiri dari beberapa tahapan utama, yaitu augmentasi data, ekstraksi fitur menggunakan convolutional layer, serta klasifikasi menggunakan fully connected layer. Pada bagian awal, model menerima input citra berukuran $150 \times 150 \times 3$. Sebelum masuk ke proses ekstraksi fitur, dilakukan augmentasi data secara online menggunakan layer bawaan Keras, yaitu:

- RandomRotation untuk rotasi citra
- RandomZoom untuk variasi skala
- RandomTranslation untuk pergeseran posisi

Augmentasi ini dilakukan secara langsung pada saat proses training, sehingga setiap epoch model menerima variasi citra yang berbeda tanpa perlu menyimpan data hasil augmentasi secara terpisah.

Selanjutnya, proses ekstraksi fitur dilakukan menggunakan empat buah convolutional layer dengan jumlah filter berturut-turut 32, 64, 64, dan 128, masing-masing menggunakan kernel berukuran 3×3 , fungsi aktivasi ReLU, serta padding

same. Pada setiap layer konvolusi diterapkan regularisasi L2 untuk mengurangi risiko overfitting.

Setiap convolutional layer diikuti oleh max pooling layer berukuran 2×2 yang berfungsi untuk mengurangi dimensi fitur serta mempertahankan informasi penting dari citra.

Hasil ekstraksi fitur kemudian diubah menjadi vektor satu dimensi menggunakan flatten layer, yang selanjutnya diproses pada bagian fully connected layer. Pada bagian ini, digunakan tiga buah dense layer dengan masing-masing 256 neuron dan fungsi aktivasi ReLU. Untuk mengurangi overfitting, ditambahkan dropout sebesar 0,50 pada bagian CNN dan 0,20 pada setiap dense layer.

Pada bagian akhir, digunakan output layer dengan jumlah neuron sesuai jumlah kelas, yaitu lima kelas usia, serta fungsi aktivasi softmax untuk menghasilkan probabilitas pada masing-masing kelas.

Modul Program 4.5 Arsitektur Model CNN

```
def build_cnn_model(input_shape: tuple = (*IMG_SIZE, 3),
                    num_classes: int    = NUM_CLASSES) ->
Model:
    inp = Input(shape=input_shape, name="InputLayer")

    x = tf.keras.layers.RandomRotation(
        factor=10/360, fill_mode="nearest",
name="Aug_Rotation")(inp)
    x = tf.keras.layers.RandomZoom(
        height_factor=(-0.1, 0.1), width_factor=(-0.1, 0.1),
        fill_mode="nearest", name="Aug_Zoom")(x)
    x = tf.keras.layers.RandomTranslation(
        height_factor=0.1, width_factor=0.1,
        fill_mode="nearest", name="Aug_Translation")(x)

    x = Conv2D(32, (3,3), activation="relu", padding="same",
        kernel_regularizer=l2(0.001),
name="Conv2D_1")(x)
    x = MaxPooling2D((2,2), padding="same",
name="MaxPool_1")(x)

    x = Conv2D(64, (3,3), activation="relu", padding="same",
        kernel_regularizer=l2(0.001),
name="Conv2D_2")(x)
    x = MaxPooling2D((2,2), padding="same",
name="MaxPool_2")(x)

    x = Conv2D(64, (3,3), activation="relu", padding="same",
        kernel_regularizer=l2(0.001),
name="Conv2D_3")(x)
    x = MaxPooling2D((2,2), padding="same",
name="MaxPool_3")(x)

    x = Conv2D(128, (3,3), activation="relu", padding="same",
        kernel_regularizer=l2(0.001),
name="Conv2D_4")(x)
```

```

x = MaxPooling2D((2,2), padding="same",
name="MaxPool_4")(x)

x = Flatten(name="Flatten")(x)
x = Dropout(0.50, name="Dropout_CNN")(x)

for i in range(1, 4):
    x = Dense(128, activation="relu",
              kernel_regularizer=l2(0.0001),
name=f"Dense_{i}")(x)
    x = Dropout(0.20, name=f"Dropout_Dense_{i}")(x)

output = Dense(num_classes, activation="softmax",
               dtype="float32", name="Dense_age")(x)

return Model(inputs=inp, outputs=output,
name="CNN_AgeClassification")

```

2. Augmentasi CNN

Augmentasi di dalam arsitektur CNN digunakan untuk menambah variasi citra wajah selama proses pelatihan. Pada bagian ini, augmentasi dilakukan menggunakan tiga layer, yaitu RandomRotation, RandomZoom, dan RandomTranslation.

Augmentasi ini bekerja secara acak saat training. Citra yang masuk ke model dapat mengalami perubahan ringan, seperti diputar sedikit, diperbesar atau diperkecil, serta digeser secara horizontal maupun vertikal. Dengan cara ini, model dapat belajar mengenali wajah dalam berbagai kondisi, misalnya wajah yang sedikit miring, tidak berada tepat di tengah gambar, atau memiliki ukuran yang berbeda.

Pada implementasi program, augmentasi diletakkan sebelum layer convolution pertama. Artinya, citra terlebih dahulu melewati layer augmentasi, kemudian hasilnya diproses oleh CNN untuk ekstraksi fitur. Augmentasi ini hanya aktif saat training, sedangkan pada validasi, evaluasi, dan prediksi augmentasi otomatis tidak aktif. Hal ini dilakukan agar data uji tetap dalam bentuk asli, sehingga hasil evaluasi lebih objektif.

Augmentasi dibagi menjadi dua bagian karena masing-masing jenis augmentasi lebih efisien dijalankan di tempat yang berbeda. Augmentasi sederhana seperti flip, brightness, contrast, dan saturation dilakukan pada pipeline tf.data karena dapat diproses secara paralel saat data dibaca. Sementara itu, augmentasi seperti rotation, zoom, dan translation diletakkan di dalam model CNN karena lebih efisien dijalankan sebagai layer Keras dan dapat diproses langsung oleh GPU. Dengan pembagian ini, proses training menjadi lebih cepat, penggunaan memori lebih efisien, dan GPU tidak banyak menunggu proses preprocessing dari CPU.

Selain itu, augmentasi dilakukan secara dinamis, bukan dengan menyimpan hasil augmentasi sebagai file baru. Jadi satu citra tidak menghasilkan semua kombinasi augmentasi secara permanen, tetapi setiap kali citra masuk ke proses training, sistem dapat memberikan variasi acak yang berbeda. Dengan begitu, variasi data bertambah tanpa menambah jumlah file pada penyimpanan.

Modul Program 4.6 Augmentasi di Dalam CNN

```
# Keras Augmentation Layers, aktif hanya saat training
x = tf.keras.layers.RandomRotation(
    factor=10/360,
    fill_mode="nearest",
    name="Aug_Rotation"
)(inp)

x = tf.keras.layers.RandomZoom(
    height_factor=(-0.1, 0.1),
    width_factor=(-0.1, 0.1),
    fill_mode="nearest",
    name="Aug_Zoom"
)(x)

x = tf.keras.layers.RandomTranslation(
    height_factor=0.1,
    width_factor=0.1,
    fill_mode="nearest",
    name="Aug_Translation"
)(x)
```

3. Kompilasi dan Training Model

Setelah arsitektur model dibangun, tahap selanjutnya adalah kompilasi dan pelatihan model. Pada tahap ini, model dikonfigurasi dengan menentukan optimizer, fungsi loss, serta metrik evaluasi yang digunakan selama proses training.

Model menggunakan optimizer Adam dengan learning rate sebesar 0.00092, yang dipilih karena kemampuannya dalam melakukan penyesuaian bobot secara adaptif sehingga mempercepat proses konvergensi. Fungsi loss yang digunakan adalah categorical crossentropy, sesuai dengan permasalahan klasifikasi multi-kelas, sedangkan metrik evaluasi yang digunakan adalah accuracy.

Selanjutnya, dilakukan proses training model menggunakan data latih dan data validasi yang telah dipersiapkan sebelumnya. Jumlah langkah pelatihan (steps per epoch) ditentukan berdasarkan jumlah data dan ukuran batch, sehingga seluruh data dapat digunakan secara optimal dalam setiap epoch.

Untuk meningkatkan kualitas pelatihan dan mencegah overfitting, digunakan beberapa mekanisme callback, yaitu:

- EarlyStopping, yang berfungsi menghentikan proses training secara otomatis apabila tidak terjadi peningkatan pada akurasi validasi dalam sejumlah epoch tertentu
- ReduceLROnPlateau, yang menurunkan nilai learning rate secara adaptif ketika performa model tidak mengalami peningkatan
- ModelCheckpoint, yang menyimpan model terbaik berdasarkan performa pada data validasi

Proses training dilakukan selama jumlah epoch yang telah ditentukan dengan memanfaatkan tf.data pipeline, sehingga proses pembacaan data, augmentasi, dan

pengolahan input dapat dilakukan secara efisien dan paralel. Hasil dari proses training disimpan dalam bentuk model terbaik yang akan digunakan pada tahap evaluasi.

Modul Program 4.7 Kompilasi dan Training Model

```
def compile_and_train(model: Model,
                      train_ds: tf.data.Dataset, train_n: int,
                      val_ds:   tf.data.Dataset, val_n:
int):
    optimizer = Adam(learning_rate=0.00092)

    model.compile(
        optimizer = optimizer,
        loss      = "categorical_crossentropy",
        metrics   = ["accuracy"],
    )

    steps_train = int(np.ceil(train_n / BATCH_SIZE))
    steps_val   = int(np.ceil(val_n   / BATCH_SIZE))

    callbacks = [
        EarlyStopping(
            monitor="val_accuracy", patience=10,
            restore_best_weights=True, verbose=1,
        ),
        ReduceLROnPlateau(
            monitor="val_loss", factor=0.5, patience=5,
            verbose=1, min_lr=1e-6,
        ),
        ModelCheckpoint(
            filepath=MODEL_SAVE_PATH, monitor="val_accuracy",
            save_best_only=True, verbose=1,
        ),
    ]

    history = model.fit(
        train_ds,
        validation_data = val_ds,
        epochs          = EPOCHS,
        steps_per_epoch = steps_train,
        validation_steps = steps_val,
        callbacks        = callbacks,
        verbose          = 1,
    )

    return history
```

4.1.4 Implementasi PSO

Pada tahap ini, dilakukan implementasi algoritma Particle Swarm Optimization (PSO) untuk mengoptimasi hyperparameter model CNN. Tujuan dari penggunaan PSO adalah untuk mencari kombinasi hyperparameter terbaik yang mampu meningkatkan performa model dalam klasifikasi kelompok usia. Hyperparameter yang dioptimasi dalam penelitian ini meliputi:

- Learning rate
- Dropout CNN
- Dropout Dense
- Jumlah unit pada dense layer

1. Fungsi Fitness

Fungsi fitness digunakan untuk mengevaluasi kualitas setiap partikel dalam proses optimasi. Pada penelitian ini, fitness didefinisikan sebagai nilai validation accuracy tertinggi yang diperoleh selama proses pelatihan model. Setiap partikel merepresentasikan satu kombinasi hyperparameter, yang kemudian digunakan untuk membangun dan melatih model CNN.

Proses evaluasi dimulai dengan membangun model berdasarkan hyperparameter yang dihasilkan partikel, kemudian dilakukan kompilasi menggunakan optimizer Adam dengan nilai learning rate yang sesuai. Model selanjutnya dilatih menggunakan data training dan validasi dalam jumlah epoch terbatas. Untuk meningkatkan efisiensi komputasi, digunakan mekanisme EarlyStopping sehingga proses pelatihan dapat dihentikan lebih awal apabila tidak terjadi peningkatan performa.

Nilai fitness diambil dari nilai validation accuracy terbaik selama proses pelatihan. Pendekatan ini memungkinkan proses optimasi berjalan lebih efisien karena tidak memerlukan pelatihan penuh pada setiap kombinasi hyperparameter yang diuji.

Modul Program 4.8 Fungsi Fitness PSO

```
def evaluate_fitness(hp: dict, train_ds, val_ds, train_n, val_n,
                    pso_epochs: int = 10) -> float:
    tf.keras.backend.clear_session()
    gc.collect()

    model = build_cnn_with_params(hp)

    model.compile(optimizer=Adam(learning_rate=hp["learning_rate"]),
                  loss="categorical_crossentropy",
                  metrics=["accuracy"])

    early_stop = EarlyStopping(monitor="val_accuracy",
                               patience=3, ...)

    history = model.fit(train_ds, validation_data=val_ds,
                        epochs=pso_epochs,
                        callbacks=[early_stop], ...)

    fitness = float(max(history.history.get("val_accuracy",
                                             [0.0])))

    del model
    tf.keras.backend.clear_session()
    gc.collect()

    return fitness
```

2. Kelas PSOptimizer

Implementasi PSO dilakukan melalui kelas PSOptimizer yang bertanggung jawab dalam mengelola proses optimasi. Pada tahap awal, sejumlah partikel diinisialisasi secara acak dalam ruang pencarian hyperparameter. Setiap partikel memiliki posisi yang merepresentasikan nilai hyperparameter serta kecepatan yang menentukan arah pergerakan dalam ruang pencarian.

Selama proses optimasi, setiap partikel akan dievaluasi menggunakan fungsi fitness untuk menentukan kualitas solusi yang dihasilkan. Setiap partikel menyimpan posisi terbaik yang pernah dicapai (personal best), sementara seluruh partikel berbagi informasi mengenai solusi terbaik secara global (global best).

Pergerakan partikel diatur melalui pembaruan kecepatan yang dipengaruhi oleh tiga komponen, yaitu inersia, kecenderungan menuju posisi terbaik individu, dan kecenderungan menuju posisi terbaik global. Setelah kecepatan diperbarui, posisi partikel akan disesuaikan dan dibatasi agar tetap berada dalam rentang nilai yang valid.

Proses ini dilakukan secara iteratif hingga mencapai jumlah iterasi yang telah ditentukan, sehingga diperoleh kombinasi hyperparameter terbaik berdasarkan nilai fitness yang dihasilkan.

Modul Program 4.9 Kelas PSO Optimizer

```
class PSOptimizer:
    def __init__(self, n_particles=10, n_iterations=10,
                 w=0.7, c1=1.5, c2=1.5, pso_epochs=10,
                 seed=42):
        # Inisialisasi posisi & kecepatan partikel
        self.positions = np.random.uniform(LOWER, UPPER,
size=(n_particles, dim))
        self.velocities = np.random.uniform(-v_range, v_range,
size=(n_particles, dim))
        self.pbest_pos = self.positions.copy()
        self.gbest_pos = self.positions[0].copy()

    def _update_velocity(self):
        r1 = np.random.rand(self.n_particles, self.dim)
        r2 = np.random.rand(self.n_particles, self.dim)
        cognitive = self.c1 * r1 * (self.pbest_pos -
self.positions)
        social = self.c2 * r2 * (self.gbest_pos -
self.positions)
        self.velocities = self.w * self.velocities + cognitive
+ social

    def _update_position(self):
        self.positions += self.velocities
        self._position_clamp()

    def optimize(self, train_ds, val_ds, train_n, val_n) ->
dict:
        for it in range(self.n_iterations):
            for p in range(self.n_particles):
                hp = decode_particle(self.positions[p])
                fitness = evaluate_fitness(hp, ...)
```

```

        if fitness > self.pbest_fit[p]:    # Update
pbest
            self.pbest_fit[p] = fitness
            self.pbest_pos[p] =
self.positions[p].copy()

        if fitness > self.gbest_fit:      # Update
gbest
            self.gbest_fit = fitness
            self.gbest_pos = self.positions[p].copy()

    self._update_velocity()
    self._update_position()

```

3. Training Baseline dan Retraining Model Terbaik

Sebelum proses optimasi dilakukan, model CNN baseline terlebih dahulu dilatih menggunakan hyperparameter awal yang mengacu pada penelitian sebelumnya. Pelatihan dilakukan secara penuh dengan jumlah epoch yang telah ditentukan serta menggunakan beberapa mekanisme callback seperti EarlyStopping, ReduceLROnPlateau, dan ModelCheckpoint untuk menjaga stabilitas proses training.

Setelah proses optimasi PSO selesai, kombinasi hyperparameter terbaik yang diperoleh kemudian digunakan untuk melatih ulang model CNN. Pada tahap ini, model dilatih kembali menggunakan jumlah epoch penuh dengan tujuan untuk memaksimalkan performa model berdasarkan konfigurasi hyperparameter hasil optimasi.

Model hasil retraining selanjutnya dievaluasi menggunakan data uji untuk mengetahui performa akhirnya, yang kemudian dibandingkan dengan model baseline guna menganalisis pengaruh penggunaan PSO terhadap peningkatan kinerja model.

Modul Program 4.10 Training Baseline & Retrain Model Terbaik

```

def train_baseline(...):
    baseline_hp = {
        "learning_rate" : 0.00092,
        "dropout_cnn"   : 0.50,
        "dropout_dense" : 0.20,
        "dense_units"   : 128,
    }
    # Training 50 epoch dengan EarlyStopping +
    ReduceLROnPlateau + ModelCheckpoint

def retrain_best_model(hp, ..., full_epochs=50):
    # Gunakan hyperparameter terbaik hasil PSO
    # Training ulang 50 epoch penuh
    callbacks = [EarlyStopping(...), ReduceLROnPlateau(...),
ModelCheckpoint(...)]
    history = model.fit(train_ds, ...)
    test_loss, test_acc = model.evaluate(test_ds, ...)

```

4.1.5 Evaluasi Model

Tahap evaluasi model dilakukan untuk mengukur kinerja model dalam melakukan klasifikasi kelompok usia berdasarkan citra wajah. Evaluasi dilakukan menggunakan data

uji (test set) yang tidak pernah digunakan selama proses training, sehingga hasil yang diperoleh mencerminkan kemampuan generalisasi model terhadap data baru.

Proses evaluasi dimulai dengan menghitung nilai loss dan accuracy menggunakan fungsi `model.evaluate()`. Nilai loss menunjukkan tingkat kesalahan model dalam melakukan prediksi, sedangkan accuracy menunjukkan persentase prediksi yang benar dari seluruh data uji.

Jumlah langkah evaluasi (steps) ditentukan berdasarkan jumlah data uji dan ukuran batch, sehingga seluruh data dapat diproses secara optimal. Hasil evaluasi ini memberikan gambaran awal mengenai performa model secara keseluruhan. Setelah proses evaluasi, model digunakan untuk melakukan prediksi terhadap data uji menggunakan fungsi `model.predict()`. Output dari model berupa probabilitas untuk setiap kelas, yang kemudian dikonversi menjadi label kelas menggunakan fungsi `argmax`.

Selanjutnya, label asli (ground truth) dikumpulkan dari dataset uji dengan cara mengekstrak nilai label dari setiap batch. Proses ini memastikan bahwa hasil prediksi dapat dibandingkan secara langsung dengan label sebenarnya.

Untuk memperoleh evaluasi yang lebih komprehensif, digunakan classification report yang mencakup metrik:

- Precision, untuk mengukur ketepatan prediksi pada setiap kelas
- Recall, untuk mengukur kemampuan model dalam mendeteksi seluruh data pada suatu kelas
- *F1-Score*, sebagai kombinasi antara precision dan recall

Penggunaan metrik ini sangat penting karena dataset yang digunakan memiliki ketidakseimbangan jumlah data antar kelas (class imbalance), sehingga accuracy saja tidak cukup untuk menggambarkan performa model secara menyeluruh.

Modul Program 4.11 Evaluasi Model

```
def evaluate_model(model: Model, test_ds: tf.data.Dataset,
                  test_n: int, history):
    print("\n" + "=" * 60)
    print("TAHAP 5: Evaluasi Model pada Test Set")
    print("=" * 60)

    steps_test = int(np.ceil(test_n / BATCH_SIZE))
    test_loss, test_acc = model.evaluate(test_ds,
    steps=steps_test, verbose=1)
    print(f"\nTest Loss      : {test_loss:.4f}")
    print(f"Test Accuracy: {test_acc:.4f}  ({test_acc *
    100:.2f}%)")

    y_pred_prob = model.predict(test_ds, steps=steps_test,
    verbose=1)
    y_pred      = np.argmax(y_pred_prob, axis=1)

    y_true_list = []
    for _, labels_batch in test_ds:
        y_true_list.append(np.argmax(labels_batch.numpy(),
    axis=1))
```

```

y_true = np.concatenate(y_true_list)[:len(y_pred)]

print("\nClassification Report:")
print(classification_report(y_true, y_pred,
target_names=CLASS_NAMES))

```

4.1.6 Visualisasi Hasil

Visualisasi hasil digunakan untuk memberikan gambaran yang lebih intuitif terhadap proses pelatihan dan performa model dalam melakukan klasifikasi. Pada penelitian ini, visualisasi yang digunakan meliputi training curves dan confusion matrix.

1. Training Curve

Training curves digunakan untuk memvisualisasikan perkembangan performa model selama proses training, baik pada data latih maupun data validasi. Visualisasi ini mencakup dua metrik utama, yaitu accuracy dan loss terhadap epoch.

Grafik accuracy menunjukkan peningkatan kemampuan model dalam melakukan prediksi yang benar seiring bertambahnya epoch, sedangkan grafik loss menunjukkan penurunan tingkat kesalahan model selama proses pelatihan. Berdasarkan grafik yang dihasilkan:

- Kurva training accuracy dan validation accuracy digunakan untuk melihat kemampuan generalisasi model
- Kurva training loss dan validation loss digunakan untuk mendeteksi potensi overfitting atau underfitting

Apabila kurva training dan validation menunjukkan pola yang sejalan, maka model dapat dikatakan memiliki kemampuan generalisasi yang baik. Sebaliknya, jika terdapat perbedaan yang signifikan antara keduanya, maka terdapat indikasi overfitting.

Selain itu, grafik ini juga digunakan sebagai dasar dalam menentukan jumlah epoch optimal, dimana peningkatan performa yang signifikan umumnya terjadi pada epoch awal, kemudian cenderung melandai pada epoch berikutnya.

Modul Program 4.12 Training Curves

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("Training & Validation - CNN Age
Classification\n(Negara et al., 2024)",
            fontsize=13, fontweight="bold")

axes[0].plot(history.history["accuracy"], label="Train
Accuracy", linewidth=2)
axes[0].plot(history.history["val_accuracy"], label="Val
Accuracy", linewidth=2, linestyle="--")
axes[0].set_title("Model Accuracy")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

```

```

    axes[1].plot(history.history["loss"], label="Train
Loss", linewidth=2, color="tomato")
    axes[1].plot(history.history["val_loss"], label="Val
Loss", linewidth=2, color="darkorange", linestyle="--")
    axes[1].set_title("Model Loss")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Loss")
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("training_curves.png", dpi=150,
bbox_inches="tight")
plt.show()

```

2. Confusion Matrix

Confusion matrix digunakan untuk menganalisis performa model secara lebih detail pada masing-masing kelas. Visualisasi ini menunjukkan perbandingan antara label aktual (true label) dan hasil prediksi model (predicted label).

Nilai diagonal pada confusion matrix menunjukkan jumlah prediksi yang benar pada setiap kelas, sedangkan nilai di luar diagonal menunjukkan kesalahan klasifikasi yang dilakukan oleh model. Melalui confusion matrix, dapat dilakukan analisis sebagai berikut:

- Mengidentifikasi kelas yang memiliki tingkat akurasi tinggi
- Mengetahui kelas yang sering mengalami kesalahan klasifikasi
- Melihat pola kesalahan antar kelas, misalnya kecenderungan model mengklasifikasikan kelas teenager sebagai adult

Analisis ini sangat penting terutama pada dataset yang tidak seimbang, karena dapat mengungkap kelemahan model yang tidak terlihat hanya dari nilai accuracy.

Modul Program 4.13 Confusion Matrix

```

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(7, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=CLASS_NAMES,
            yticklabels=CLASS_NAMES,
            linewidths=0.5)
plt.title("Age Confusion Matrix\n(CNN Age Classification -
Negara et al., 2024)",
        fontsize=12, fontweight="bold")
plt.xlabel("Predicted", fontsize=11)
plt.ylabel("True", fontsize=11)
plt.tight_layout()
plt.savefig("confusion_matrix_age.png", dpi=150,
bbox_inches="tight")
plt.show()

```

4.1.7 Implementasi Antarmuka Pengujian Model

Pada tahap ini, model CNN dan CNN dengan optimasi Particle Swarm Optimization (PSO) yang telah dilatih diimplementasikan ke dalam sebuah antarmuka berbasis web menggunakan framework Streamlit. Antarmuka ini bertujuan untuk mempermudah proses pengujian model secara interaktif dengan menggunakan citra wajah yang diunggah oleh pengguna, serta membandingkan hasil prediksi dari kedua model.

1. Konfigurasi Parameter dan Model

Tahap awal dilakukan dengan memuat model yang telah dilatih, yaitu model CNN baseline dan model CNN hasil optimasi PSO. Selain itu, ditentukan pula ukuran citra input serta label kelas yang digunakan dalam proses klasifikasi. Ukuran citra disesuaikan dengan proses training, yaitu 150×150 piksel, untuk menjaga konsistensi input model.

Modul Program 4.14 Konfigurasi Model dan Parameter

```
model_cnn = tf.keras.models.load_model("model_baseline.keras")
model_pso = tf.keras.models.load_model("model_pso_best.keras")

labels = ["child", "teenager", "adult", "Middle Age", "old"]
IMG_SIZE = 150
```

2. Inisialisasi Antarmuka Pengguna

Selanjutnya, sistem membangun antarmuka pengguna menggunakan Streamlit. Antarmuka ini menampilkan judul aplikasi serta menyediakan fitur unggah citra yang memungkinkan pengguna memasukkan gambar wajah yang akan diuji.

Modul Program 4.15 Antarmuka Unggah Citra

```
st.markdown("<h2 style='text-align: center;'>Klasifikasi  
Kelompok Usia Menggunakan<br>CNN dan CNN Dengan Optimasi  
PSO</h2>", unsafe_allow_html=True)

uploaded_file = st.file_uploader("Upload gambar", type=["png",  
"jpg", "jpeg"])
```

3. Deteksi Wajah pada Citra

Sebelum dilakukan proses prediksi, sistem terlebih dahulu melakukan deteksi wajah menggunakan metode Haar Cascade dari library OpenCV. Deteksi ini bertujuan untuk memastikan bahwa hanya area wajah yang diproses oleh model, sehingga meningkatkan akurasi prediksi.

Citra yang diunggah akan dikonversi ke dalam format grayscale untuk mempermudah proses deteksi, kemudian dilakukan pencarian area wajah menggunakan fungsi `detectMultiScale`.

Modul Program 4.16 Deteksi Wajah

```
gray = cv2.cvtColor(image_np, cv2.COLOR_RGB2GRAY)

faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.3,
    minNeighbors=5
)
```

4. Preprocessing Data

Setelah wajah terdeteksi, dilakukan proses preprocessing yang meliputi pemotongan area wajah, perubahan ukuran citra menjadi 150×150 piksel, serta normalisasi nilai piksel ke rentang 0 hingga 1. Data kemudian diubah ke dalam bentuk yang sesuai dengan input model.

Modul Program 4.17 Preprocessing Citra

```
face_resized = cv2.resize(face, (IMG_SIZE, IMG_SIZE))
face_norm = face_resized / 255.0
face_input = np.expand_dims(face_norm, axis=0)
```

5. Proses Prediksi Model

Citra wajah yang telah diproses kemudian digunakan sebagai input untuk kedua model, yaitu model CNN dan model CNN dengan optimasi PSO. Masing-masing model menghasilkan probabilitas untuk setiap kelas usia, yang kemudian dikonversi menjadi label kelas menggunakan fungsi `argmax`.

Modul Program 4.18 Proses Prediksi

```
pred_cnn = model_cnn.predict(face_input)
pred_pso = model_pso.predict(face_input)

class_cnn = labels[np.argmax(pred_cnn)]
class_pso = labels[np.argmax(pred_pso)]
```

6. Visualisasi Hasil Prediksi

Hasil prediksi ditampilkan dalam bentuk persentase probabilitas untuk setiap kelas usia, serta label hasil klasifikasi akhir. Antarmuka dibagi menjadi dua bagian untuk menampilkan hasil dari model CNN dan CNN dengan optimasi PSO secara berdampingan, sehingga memudahkan perbandingan performa kedua model.

Modul Program 4.19 Visualisasi Hasil

```
for i, prob in enumerate(pred_cnn[0]):
    st.progress(float(prob))
    st.write(f"{labels[i]}: {prob*100:.2f}%")
st.success(f"Hasil: {class_cnn}")
```

7. Visualisasi Hasil Training

Selain menampilkan hasil prediksi, antarmuka juga menyediakan visualisasi hasil training berupa confusion matrix dan grafik training history. Visualisasi ini membantu pengguna dalam memahami performa model secara keseluruhan.

Modul Program 4.20 Visualisasi Training

```
st.image("confusion_matrix_age.png", caption="Confusion Matrix")
st.image("training_history_age.png", caption="Training History")
```

4.1.8 Implementasi Aplikasi Realtime

Pada tahap ini, model CNN yang telah dilatih diimplementasikan ke dalam sebuah aplikasi klasifikasi usia berbasis realtime. Implementasi ini bertujuan untuk menguji kemampuan model dalam melakukan prediksi secara langsung terhadap citra wajah yang diperoleh dari kamera.

1. Konfigurasi Parameter

Tahap awal dilakukan dengan menentukan parameter yang digunakan dalam proses prediksi. Ukuran citra input disesuaikan dengan ukuran yang digunakan saat training, yaitu 150×150 piksel, sehingga konsistensi data tetap terjaga. Selain itu, ditentukan pula label kelas yang digunakan dalam proses klasifikasi, yaitu child, teenager, adult, middle Age, dan old.

Modul Program 4.21 Konfigurasi Parameter

```
IMG_SIZE = 150
```

```
age_labels = ["child", "teenager", "adult", "Middle Age",  
"old"]
```

2. Inisialisasi Face Detector & Kamera

Selanjutnya, sistem menginisialisasi face detector menggunakan metode Haar Cascade yang tersedia pada library OpenCV. Metode ini digunakan untuk mendeteksi area wajah pada setiap frame yang ditangkap oleh kamera. Kamera diaktifkan menggunakan fungsi VideoCapture, yang memungkinkan sistem untuk mengambil citra secara terus-menerus dalam bentuk video.

Modul Program 4.22 Inisialisasi Face Detector dan Kamera

```
face_cascade = cv2.CascadeClassifier(  
    cv2.data.harcascades +  
    'haarcascade_frontalface_default.xml'  
)
```

```
cap = cv2.VideoCapture(0)
```

```
if not cap.isOpened():  
    print("Tidak bisa membuka kamera")  
    exit()
```

3. Loop Real-Time & Deteksi Wajah

Proses utama aplikasi berjalan dalam sebuah loop yang terus menerus membaca frame dari kamera. Setiap frame yang diperoleh akan dikonversi ke dalam format grayscale untuk meningkatkan efisiensi proses deteksi wajah. Setelah itu, dilakukan deteksi wajah menggunakan metode detectMultiScale, yang menghasilkan koordinat area wajah pada citra.

Modul Program 4.23 Loop Realtime dan Deteksi Wajah

```
while True:  
    ret, frame = cap.read()  
    if not ret:  
        break  
  
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)  
    faces = face_cascade.detectMultiScale(  
        gray,  
        scaleFactor=1.3,  
        minNeighbors=5  
    )
```

4. Preprocessing & Prediksi

Untuk setiap wajah yang terdeteksi, dilakukan proses preprocessing yang meliputi pemotongan area wajah, resize ke ukuran 150×150 piksel, serta normalisasi nilai piksel ke rentang 0 hingga 1. Data citra kemudian diubah ke dalam bentuk yang sesuai dengan input model sebelum dilakukan proses prediksi.

Modul Program 4.24 Preprocessing dan prediksi

```
for (x, y, w, h) in faces:
    face = frame[y:y+h, x:x+w]

    face_resized = cv2.resize(face, (IMG_SIZE, IMG_SIZE))
    face_norm     = face_resized / 255.0
    face_input    = np.reshape(face_norm, (1, IMG_SIZE,
IMG_SIZE, 3))

    prediction = model.predict(face_input, verbose=0)

    age_class = np.argmax(prediction)
    label = age_labels[age_class]
```

5. Tampilan Bounding Box & Label

Model CNN selanjutnya digunakan untuk memprediksi kelas usia berdasarkan citra wajah yang telah diproses. Output model berupa probabilitas untuk setiap kelas, yang kemudian dikonversi menjadi label kelas menggunakan fungsi `argmax`.

Modul Program 4.25 Tampilan Bounding Box dan Label

```
cv2.rectangle(frame, (x, y), (x+w, y+h), (0,255,0), 2)
    cv2.putText(frame, label, (x, y-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0,255,0), 2)

cv2.imshow("Real-Time Age Detection", frame)

if cv2.waitKey(1) & 0xFF == 27: # ESC untuk keluar
    break
```

6. Cleanup & Penutupan Sumber Daya

Hasil prediksi ditampilkan secara langsung pada frame dengan menambahkan bounding box di sekitar wajah yang terdeteksi, serta label kelas usia di bagian atasnya. Proses ini berlangsung secara real-time, sehingga pengguna dapat melihat hasil klasifikasi secara langsung melalui tampilan video.

Modul Program 4.26 Cleanup dan Penutupan Sumber Daya

```
cap.release()
cv2.destroyAllWindows()
```

4.2 Hasil

Pada bagian ini disajikan hasil dari seluruh tahapan pengujian yang telah dilakukan dalam penelitian. Hasil yang dibahas meliputi preprocessing data, pembagian kelas usia, performa model CNN baseline, hasil optimasi hyperparameter menggunakan Particle Swarm Optimization (PSO), serta performa model CNN setelah dilakukan optimasi.

Pengujian dilakukan dalam empat skenario, yaitu model CNN baseline dengan pembagian kelas berdasarkan jurnal acuan, model CNN baseline jurnal dengan optimasi PSO, model CNN baseline penelitian dengan penambahan kelas Middle Age, serta model CNN penelitian dengan optimasi PSO. Penyajian hasil ini bertujuan untuk membandingkan pengaruh optimasi PSO terhadap peningkatan kinerja CNN pada masing-masing skenario.

4.2.1 Hasil Preprocessing Data

Pada tahap preprocessing, data citra wajah yang diperoleh dari dataset UTKFace telah melalui beberapa proses, yaitu ekstraksi label usia dari nama file, pengelompokan label berdasarkan rentang usia, resize citra, normalisasi nilai piksel, serta pembagian data ke dalam data latih (training set), data validasi (validation set), dan data uji (test set). Tahap preprocessing dilakukan untuk memastikan bahwa data yang digunakan telah memiliki format, ukuran, dan label yang sesuai sebelum digunakan dalam proses pelatihan model.

Pada penelitian ini, preprocessing dilakukan terhadap dua skenario pengelompokan data. Skenario pertama merupakan preprocessing untuk model baseline jurnal yang menggunakan empat kelas usia, yaitu Child, Teenager, Adult, dan Old. Skenario kedua merupakan preprocessing untuk model baseline penelitian yang menggunakan lima kelas usia, yaitu dengan menambahkan kelas Middle Age sebagai pemisah antara kelompok usia dewasa dan lanjut usia. Perbedaan utama dari kedua skenario preprocessing tersebut terletak pada proses pengelompokan label usia, sedangkan tahapan lain seperti resize citra, normalisasi, pembagian data, dan augmentasi tetap dilakukan dengan prosedur yang sama.

1. Distribusi Data Model Baseline Jurnal 4 Kelas

Pada skenario baseline jurnal, data usia dikelompokkan ke dalam empat kelas, yaitu Child untuk usia 0-12 tahun, Teenager untuk usia 13-18 tahun, Adult untuk usia 19-40 tahun, dan Old untuk usia lebih dari 40 tahun. Pada skenario ini, data usia 41-60 tahun dan usia lebih dari 60 tahun masih digabung ke dalam satu kelas Old.

Tabel 4.1 Distribusi Data Model Baseline Jurnal 4 Kelas

Kelas Usia	Train	Val	Test	Jumlah Data
Child	2730	341	342	3413
Teenager	865	108	109	1082
Adult	10004	1250	1251	12505
Old	5365	670	673	6708
Total	18964	2369	2375	23708

Berdasarkan Tabel 4.1, dataset pada skenario baseline jurnal terdiri dari 23.708 citra wajah yang dibagi ke dalam empat kelas usia. Kelas Adult memiliki jumlah data paling besar, yaitu 12.505 citra, sedangkan kelas Teenager memiliki jumlah data paling

sedikit, yaitu 1.082 citra. Kelas Old pada skenario ini berjumlah 6.708 citra karena merupakan gabungan dari data usia 41-60 tahun dan usia lebih dari 60 tahun.

Distribusi data tersebut menunjukkan bahwa dataset masih bersifat tidak seimbang. Kelas Adult tetap menjadi kelas mayoritas, sedangkan kelas Teenager menjadi kelas minoritas. Kondisi ini dapat menyebabkan model lebih mudah mengenali kelas dengan jumlah data besar, tetapi berpotensi mengalami kesulitan dalam mengenali kelas dengan jumlah data lebih sedikit. Oleh karena itu, evaluasi tidak hanya dilakukan berdasarkan accuracy, tetapi juga menggunakan precision, recall, F1-Score, dan confusion matrix.

2. Distribusi Data Model Baseline Penelitian 5 Kelas

Pada skenario baseline penelitian, dilakukan penambahan kelas Middle Age untuk memisahkan kelompok usia 41-60 tahun dari kelas Old. Dengan demikian, kelas Old tidak lagi mencakup seluruh usia di atas 40 tahun, tetapi hanya difokuskan pada usia lebih dari 60 tahun. Pembagian ini bertujuan agar kategori usia menjadi lebih spesifik dan representatif terhadap perbedaan karakteristik wajah pada setiap kelompok usia.

Tabel 4.2 Distribusi Data Model Baseline Penelitian 5 Kelas

Kelas Usia	Train	Val	Test	Jumlah Data
Child	2730	341	342	3413
Teenager	865	108	109	1082
Adult	10004	1250	1251	12505
Middle Age	3448	431	432	4311
Old	1917	239	241	2397
Total	18964	2369	2375	23708

Berdasarkan Tabel 4.2, dataset pada skenario baseline penelitian juga terdiri dari 23.708 citra wajah dengan pembagian data train, validation, dan test menggunakan rasio 80:10:10. Perbedaan utama dibandingkan skenario baseline jurnal terletak pada pemisahan kelas Old menjadi dua kelas, yaitu Middle Age dan Old. Kelas Middle Age memiliki 4.311 citra, sedangkan kelas Old memiliki 2.397 citra.

Penambahan kelas Middle Age membuat pembagian kategori usia menjadi lebih detail, tetapi juga meningkatkan kompleksitas klasifikasi. Pada model 4 kelas, seluruh usia di atas 40 tahun digabung menjadi satu kelas Old, sedangkan pada model 5 kelas, model harus membedakan karakteristik wajah usia 41-60 tahun dan usia di atas 60 tahun. Dengan demikian, skenario 5 kelas memiliki tantangan klasifikasi yang lebih tinggi dibandingkan skenario 4 kelas.

3. Hasil Pembagian Data Train, Validation, dan Test

Dataset pada kedua skenario dibagi menjadi tiga bagian, yaitu data latih sebesar 80%, data validasi sebesar 10%, dan data uji sebesar 10%. Data latih digunakan untuk proses pembelajaran model, data validasi digunakan untuk memantau performa model selama pelatihan dan proses optimasi PSO, sedangkan data uji digunakan untuk mengevaluasi performa akhir model.

Pembagian data dilakukan secara terpisah untuk setiap kelas agar distribusi data pada train, validation, dan test tetap mengikuti proporsi data awal. Dengan demikian, baik pada skenario 4 kelas maupun 5 kelas, pola ketidakseimbangan data tetap dipertahankan. Hal ini penting agar hasil evaluasi mencerminkan kondisi distribusi data yang sebenarnya.

4. Analisis Dampak Preprocessing terhadap Data

Hasil preprocessing menunjukkan bahwa data telah siap digunakan dalam proses pelatihan model CNN baseline maupun CNN dengan optimasi PSO. Namun, terdapat permasalahan utama berupa ketidakseimbangan distribusi data antar kelas. Pada kedua skenario, kelas Adult memiliki jumlah data paling besar, sedangkan kelas Teenager memiliki jumlah data paling sedikit.

Pada skenario 4 kelas, jumlah kelas lebih sedikit sehingga proses klasifikasi relatif lebih sederhana. Hal ini karena data usia di atas 40 tahun masih digabung ke dalam satu kelas Old. Sebaliknya, pada skenario 5 kelas, penambahan kelas Middle Age membuat klasifikasi menjadi lebih spesifik, tetapi juga lebih sulit karena model harus membedakan dua kelompok usia yang sebelumnya digabung menjadi satu.

Implikasi dari kondisi tersebut adalah model dapat memperoleh accuracy yang tinggi secara keseluruhan, tetapi belum tentu memiliki performa yang merata pada setiap kelas. Kelas dengan jumlah data besar seperti Adult cenderung lebih mudah dikenali oleh model, sedangkan kelas minoritas seperti Teenager berpotensi memiliki nilai recall dan F1-Score yang lebih rendah. Oleh karena itu, evaluasi performa pada penelitian ini tidak hanya dilihat dari accuracy, tetapi juga dari precision, recall, F1-Score, serta confusion matrix untuk mengetahui kemampuan model dalam mengenali setiap kelas usia.

4.2.2 Hasil Model CNN Baseline Jurnal 4 Kelas

Pada tahap ini dilakukan pengujian terhadap model CNN baseline dengan pembagian kelas berdasarkan jurnal acuan. Model ini menggunakan empat kelas usia, yaitu child, teenager, adult, dan old. Berbeda dengan model penelitian lima kelas, pada skenario ini kelas old mencakup seluruh data usia di atas 40 tahun, sehingga data usia 41-60 tahun dan lebih dari 60 tahun masih digabung ke dalam satu kelas.

Untuk melihat proses pembelajaran model CNN baseline secara rinci, hasil training dari epoch 1 hingga epoch 50 disajikan pada Tabel 4.3. Tabel ini memuat nilai accuracy, loss, validation accuracy, dan validation loss pada setiap epoch, sehingga perkembangan performa model selama pelatihan dapat diamati secara lebih jelas.

Tabel 4.3 Hasil Training Model CNN Baseline Jurnal 4 Kelas per Epoch

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
1	0.5231	1.2642	0.5821	1.1234
2	0.5863	1.0806	0.6935	0.9528
3	0.6543	0.9513	0.7045	0.8335
4	0.6831	0.8943	0.7286	0.7737
5	0.6963	0.8543	0.7560	0.7166

Tabel 4.4 Hasil Training Model CNN Baseline Jurnal 4 Kelas per Epoch

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
6	0.7196	0.8105	0.7142	0.8293
7	0.7242	0.7927	0.7699	0.6845
8	0.7379	0.7603	0.7856	0.6452
9	0.7376	0.7562	0.7978	0.6336
10	0.7557	0.7245	0.7792	0.6780
11	0.7590	0.7130	0.7999	0.6214
12	0.7665	0.6995	0.7965	0.6148
13	0.7720	0.6926	0.8122	0.5996
14	0.7774	0.6740	0.8164	0.5943
15	0.7810	0.6734	0.8223	0.5795
16	0.7819	0.6649	0.8143	0.5857
17	0.7839	0.6585	0.7602	0.7430
18	0.7803	0.6781	0.8172	0.5938
19	0.7917	0.6447	0.8185	0.5850
20	0.7925	0.6428	0.7919	0.6202
21	0.7979	0.6296	0.8282	0.5387
22	0.8036	0.6145	0.8278	0.5524
23	0.8083	0.6054	0.8290	0.5339
24	0.8092	0.5984	0.8164	0.5588
25	0.8078	0.6042	0.8240	0.5549
26	0.8084	0.6002	0.8320	0.5348
27	0.8095	0.5909	0.8295	0.5222
28	0.8117	0.5897	0.8345	0.5140
29	0.8142	0.5836	0.8084	0.5855
30	0.8143	0.5803	0.8316	0.5139
31	0.8139	0.5761	0.8337	0.5319
32	0.8171	0.5722	0.8413	0.5138
33	0.8138	0.5774	0.8295	0.5230
34	0.8188	0.5669	0.8362	0.5047
35	0.8186	0.5653	0.8358	0.5096
36	0.8197	0.5638	0.8409	0.5216
37	0.8235	0.5608	0.8392	0.5030
38	0.8224	0.5555	0.8413	0.5055
39	0.8226	0.5571	0.8345	0.5120
40	0.8241	0.5568	0.8350	0.5283
41	0.8204	0.5631	0.8409	0.5179
42	0.8236	0.5498	0.8438	0.4887
43	0.8218	0.5541	0.8333	0.5182
44	0.8225	0.5507	0.8468	0.4977
45	0.8259	0.5445	0.8447	0.4813
46	0.8249	0.5437	0.8447	0.4730
47	0.8286	0.5432	0.8514	0.4923
48	0.8274	0.5399	0.8489	0.4854
49	0.8263	0.5410	0.8413	0.4840
50	0.8277	0.5396	0.8421	0.4813

Berdasarkan Tabel 4.3, performa model CNN baseline jurnal menunjukkan peningkatan yang cukup konsisten selama proses pelatihan. Nilai accuracy meningkat dari 0,5231 pada epoch pertama menjadi 0,8277 pada epoch ke-50. Sementara itu, nilai loss menurun dari 1,2642 menjadi 0,5396. Hal ini menunjukkan bahwa model mampu mempelajari pola data secara bertahap selama proses training.

Pada data validasi, nilai validation accuracy juga menunjukkan peningkatan dari 0,5821 pada epoch pertama hingga mencapai nilai terbaik sebesar 0,8514 pada epoch ke-47. Nilai validation loss terendah diperoleh pada epoch ke-46, yaitu sebesar 0,4730. Kondisi ini menunjukkan bahwa model memperoleh performa validasi terbaik pada rentang akhir pelatihan, khususnya sekitar epoch ke-46 hingga epoch ke-48.

Setelah proses training selesai, model diuji menggunakan data uji untuk mengevaluasi performa akhir dalam klasifikasi kelompok usia. Ringkasan hasil evaluasi model CNN baseline jurnal 4 kelas disajikan pada Tabel 4.5.

Tabel 4.5 Metrik Evaluasi Model CNN Baseline Jurnal 4 Kelas

Metrik	Nilai
Accuracy	83,87%
Precision	78,49%
Recall	72,06%
F1-Score	74,14%

Nilai accuracy sebesar 83,87% menunjukkan bahwa model CNN baseline jurnal 4 kelas mampu melakukan klasifikasi dengan cukup baik secara keseluruhan. Namun, nilai recall dan F1-Score yang lebih rendah dibandingkan accuracy menunjukkan bahwa performa model belum sepenuhnya merata pada seluruh kelas. Hal ini terutama terlihat pada kelas teenager, yang memiliki jumlah data lebih sedikit dibandingkan kelas lainnya.

1. Evaluasi Perkelas

Tabel 4.6 Hasil Evaluasi Perkelas (4 kelas)

Kelas	Precision	Recall	F1-Score
child	95.4%	91.8%	93.6%
teenager	54.5%	27.5%	36.6%
adult	84.4%	88.4%	86.4%
old	79.6%	80.5%	80.0%

Berdasarkan Tabel 4.6, performa model CNN baseline jurnal 4 kelas menunjukkan hasil yang berbeda pada setiap kelas. Kelas child memperoleh performa paling baik dengan precision sebesar 95,4%, recall sebesar 91,8%, dan F1-Score sebesar 93,6%. Hal ini menunjukkan bahwa model mampu mengenali kelas child dengan sangat baik dan memiliki tingkat kesalahan prediksi yang relatif rendah.

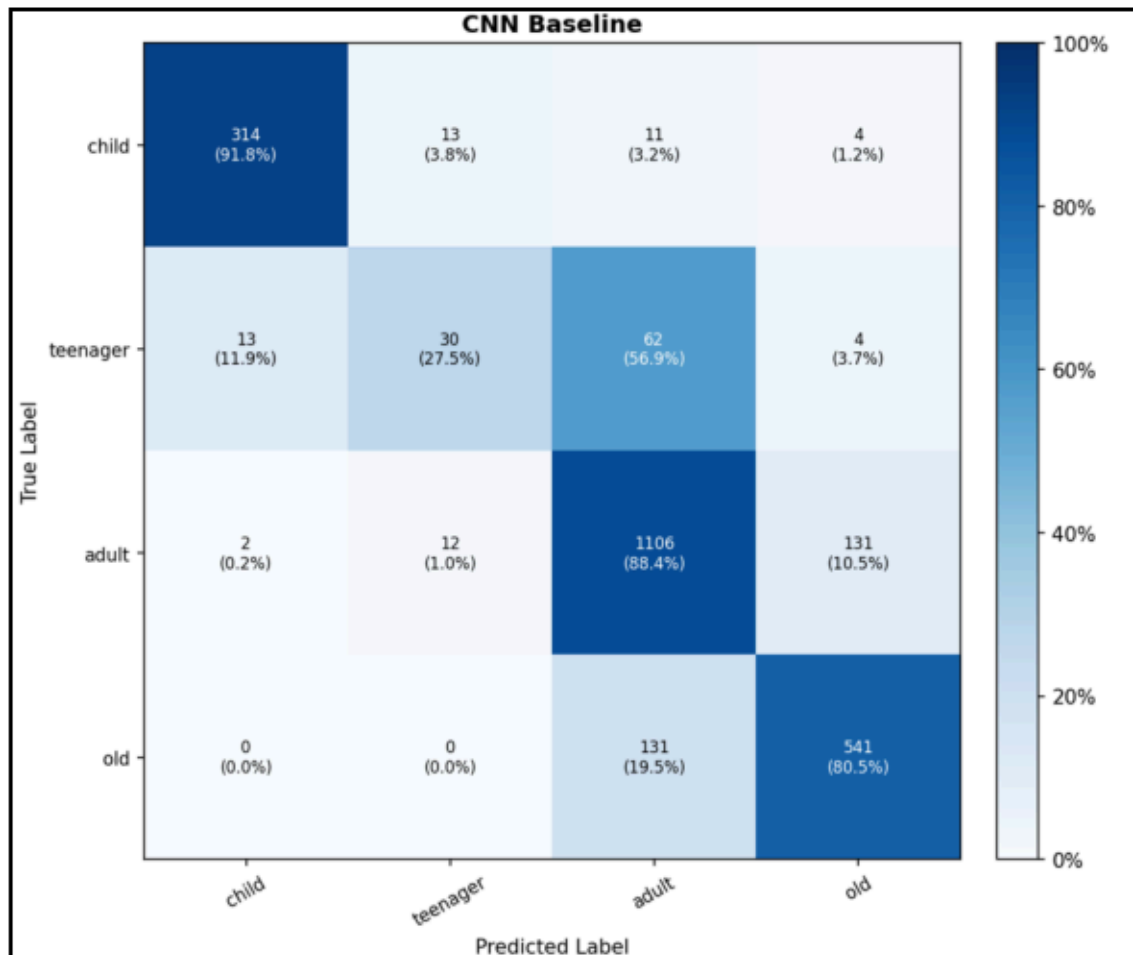
Kelas adult juga menunjukkan performa yang baik dengan precision sebesar 84,4%, recall sebesar 88,4%, dan F1-Score sebesar 86,4%. Nilai recall yang tinggi menunjukkan bahwa sebagian besar data adult berhasil dikenali dengan benar oleh model. Kondisi ini dapat dipengaruhi oleh jumlah data adult yang paling besar dibandingkan kelas lainnya, sehingga model memperoleh lebih banyak variasi fitur selama proses pelatihan.

Kelas old memperoleh precision sebesar 79,6%, recall sebesar 80,5%, dan F1-Score sebesar 80,0%. Hasil ini menunjukkan bahwa model cukup mampu mengenali kelas old, meskipun masih terdapat sebagian data old yang salah diklasifikasikan sebagai adult. Hal ini dapat terjadi karena pada skenario 4 kelas, kelas old mencakup seluruh usia di atas 40 tahun, sehingga karakteristik visualnya masih memiliki kedekatan dengan kelas adult, terutama pada usia sekitar awal 40 tahun.

Sebaliknya, kelas teenager memiliki performa paling rendah dibandingkan kelas lainnya, dengan precision sebesar 54,5%, recall sebesar 27,5%, dan F1-Score sebesar 36,6%. Nilai recall yang rendah menunjukkan bahwa sebagian besar data teenager gagal dikenali sebagai kelas teenager. Berdasarkan confusion matrix, banyak data teenager yang salah diklasifikasikan sebagai adult. Hal ini menunjukkan bahwa model mengalami kesulitan dalam membedakan karakteristik wajah remaja dan dewasa, yang kemungkinan disebabkan oleh kemiripan fitur visual serta jumlah data teenager yang jauh lebih sedikit dibandingkan kelas adult.

Secara keseluruhan, hasil evaluasi per kelas menunjukkan bahwa model CNN baseline jurnal 4 kelas memiliki performa yang baik pada kelas child, adult, dan old, tetapi masih lemah pada kelas teenager. Oleh karena itu, meskipun nilai accuracy keseluruhan cukup tinggi, analisis per kelas tetap penting karena dapat menunjukkan kelemahan model pada kelas minoritas yang tidak terlihat dari nilai accuracy saja.

2. Analisis Confusion Matrix



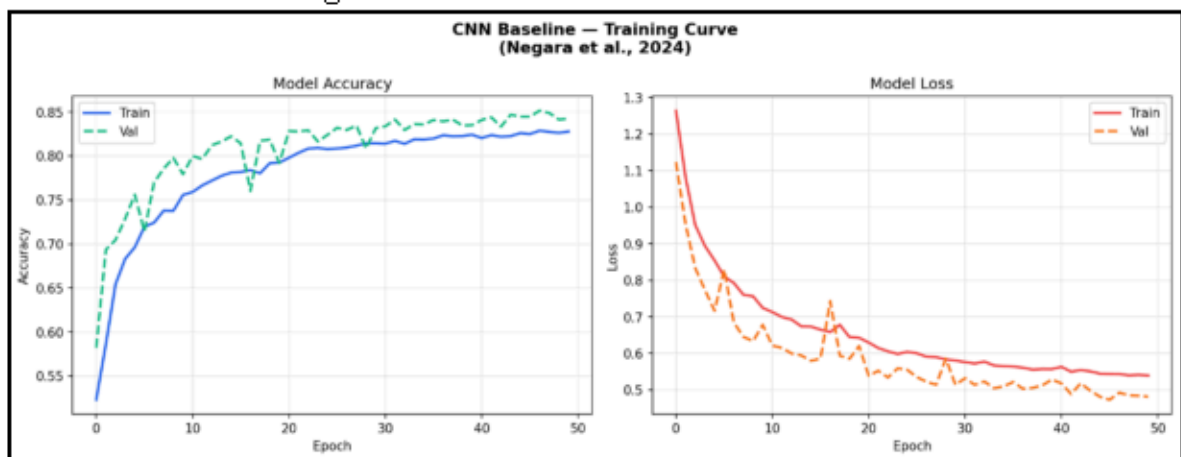
Berdasarkan confusion matrix model CNN baseline jurnal 4 kelas, kelas child menunjukkan performa yang sangat baik. Dari 342 data child, sebanyak 314 data berhasil diklasifikasikan dengan benar, atau sekitar 91,8%. Kesalahan pada kelas ini relatif kecil, yaitu sebagian kecil diprediksi sebagai teenager, adult, dan old.

Pada kelas adult, model juga menunjukkan performa yang tinggi. Dari 1.251 data adult, sebanyak 1.106 data berhasil diklasifikasikan dengan benar, atau sekitar 88,4%. Namun, masih terdapat 131 data adult yang salah diklasifikasikan sebagai old. Hal ini menunjukkan adanya kemiripan karakteristik visual antara usia dewasa akhir dan usia tua, terutama karena kelas old pada skenario 4 kelas mencakup seluruh usia di atas 40 tahun.

Kelas old juga memiliki performa yang cukup baik, dengan 541 data berhasil diklasifikasikan dengan benar, atau sekitar 80,5%. Namun, terdapat 131 data old yang salah diklasifikasikan sebagai adult. Kesalahan ini menunjukkan adanya tumpang tindih karakteristik wajah antara kelas adult dan old, terutama pada batas usia sekitar 40 tahun ke atas.

Kelas teenager menjadi kelas dengan performa terendah. Dari 109 data teenager, hanya 30 data yang berhasil diklasifikasikan dengan benar, atau sekitar 27,5%. Sebagian besar data teenager, yaitu 62 data, salah diklasifikasikan sebagai adult. Hal ini menunjukkan bahwa model masih mengalami kesulitan dalam membedakan karakteristik wajah remaja dan dewasa, yang kemungkinan disebabkan oleh kemiripan fitur visual serta jumlah data teenager yang jauh lebih sedikit dibandingkan kelas adult.

3. Analisis Training Curve



Berdasarkan grafik training curve, terlihat bahwa model mengalami peningkatan performa yang cukup signifikan pada epoch awal, khususnya pada rentang epoch 1 hingga sekitar epoch 15. Setelah itu, peningkatan accuracy mulai melambat dan memasuki fase konvergensi. Kurva training accuracy meningkat secara bertahap hingga mencapai sekitar 0,82, sedangkan validation accuracy berada pada kisaran 0,84 hingga 0,85 pada akhir pelatihan.

Pada grafik loss, nilai training loss dan validation loss sama-sama menunjukkan tren menurun. Training loss menurun dari sekitar 1,26 menjadi 0,54, sedangkan validation loss menurun dari sekitar 1,12 menjadi sekitar 0,48. Hal ini menunjukkan bahwa proses

pelatihan berjalan cukup stabil dan model mampu mengurangi kesalahan prediksi secara bertahap.

Selain itu, jarak antara kurva training dan validation tidak terlalu besar. Bahkan, nilai validation accuracy cenderung lebih tinggi dibandingkan training accuracy. Kondisi ini dapat terjadi karena data training menerima augmentasi selama pelatihan, sehingga data latih menjadi lebih bervariasi dan lebih sulit dipelajari dibandingkan data validasi. Dengan demikian, grafik training curve tidak menunjukkan indikasi overfitting yang kuat pada model baseline jurnal 4 kelas.

4.2.3 Hasil Optimasi PSO Jurnal 4 Kelas

Pada tahap ini, algoritma Particle Swarm Optimization (PSO) diterapkan untuk mencari kombinasi hyperparameter terbaik pada model CNN baseline jurnal 4 kelas. Optimasi dilakukan terhadap empat hyperparameter, yaitu learning rate, dropout CNN, dropout dense, dan dense units. Setiap partikel merepresentasikan satu kombinasi hyperparameter yang dievaluasi menggunakan nilai validation accuracy sebagai fungsi fitness.

Berdasarkan hasil optimasi, PSO berhasil memperoleh konfigurasi hyperparameter terbaik dengan nilai gbest validation accuracy sebesar 0,8168. Konfigurasi tersebut kemudian digunakan pada tahap pelatihan ulang model CNN untuk membentuk model CNN 4 kelas dengan optimasi PSO. Hasil konfigurasi PSO yang kamu lampirkan menunjukkan 10 partikel, 10 iterasi, total 100 evaluasi, dan setiap evaluasi dilakukan selama 10 epoch.

1. Parameter Optimasi PSO

Tabel 4.7 Parameter Particle Swarm Optimization

Parameter	Nilai
Jumlah Partikel (<i>Swarm Size</i>)	10 Partikel
Jumlah Iterasi	10 Iterasi
Bobot Inersia (w)	0.7
Koefisien Kognitif ($c1$)	1.5
Koefisien Sosial ($c2$)	1.5

Berdasarkan Tabel 4.7, proses optimasi dilakukan menggunakan 10 partikel selama 10 iterasi, sehingga terdapat total 100 evaluasi kandidat solusi. Setiap kandidat solusi dievaluasi melalui proses training singkat selama 10 epoch untuk memperoleh nilai validation accuracy sebagai nilai fitness.

Penggunaan bobot inersia sebesar 0,7 bertujuan untuk menjaga pergerakan partikel agar tetap memiliki kemampuan eksplorasi terhadap ruang pencarian. Sementara itu, nilai koefisien kognitif dan sosial masing-masing sebesar 1,5 digunakan untuk menyeimbangkan pengaruh pengalaman terbaik individu partikel (pBest) dan pengalaman terbaik seluruh populasi (gBest).

2. Hasil Evaluasi Partikel pada Setiap Iterasi PSO

Tabel 4.8 Iterasi Ke-1

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000103	0.5850	0.3200	192	0.7408
2	P2	0.000026	0.3470	0.1170	224	0.7024
3	P3	0.000420	0.5120	0.1060	256	0.7940
4	P4	0.001770	0.3640	0.1550	96	0.7759
5	P5	0.000066	0.4570	0.2300	128	0.7282
6	P6	0.000449	0.3420	0.1880	128	0.7868
7	P7	0.000170	0.5360	0.1600	160	0.7687
8	P8	0.000398	0.3140	0.2820	96	0.7709
9	P9	0.000015	0.5850	0.3900	224	0.5276
10	P10	0.000066	0.3290	0.3050	160	0.7358

Nilai partikel pada iterasi pertama dibangkitkan secara acak untuk menginisiasi posisi awal dalam ruang pencarian hyperparameter. Berdasarkan hasil evaluasi, global best (gBest) awal dicapai oleh Partikel 3 dengan Val Accuracy tertinggi sebesar 0.7940 (Learning Rate 0.000420, Dropout CNN 0.5120, Dropout Dense 0.1060, Dense Units 256).

Tabel 4.9 Iterasi Ke-2

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000407	0.4950	0.2300	192	0.7835
2	P2	0.000055	0.4370	0.1000	256	0.7404
3	P3	0.000437	0.4990	0.1260	256	0.7868
4	P4	0.002000	0.4540	0.1000	160	0.7560
5	P5	0.000195	0.5030	0.1430	192	0.7708
6	P6	0.000370	0.3970	0.1410	192	0.7944
7	P7	0.000207	0.5360	0.1000	224	0.7830
8	P8	0.000276	0.4040	0.1920	128	0.7864
9	P9	0.000020	0.5450	0.3000	224	0.6256
10	P10	0.000429	0.4190	0.2280	192	0.7881

Nilai partikel pada iterasi kedua bukan dibangkitkan ulang secara acak, tetapi diperoleh dari hasil pergerakan partikel pada iterasi pertama menggunakan rumus PSO dengan memperbarui velocity dan posisi berdasarkan pBest dan gBest. Pada iterasi ini, posisi terbaik ditempati oleh Partikel 6 dengan Val Accuracy sebesar 0.7944 (Learning Rate 0.000370, Dropout CNN 0.3970, Dropout Dense 0.1410, Dense Units 192).

Tabel 4.10 Iterasi Ke-3

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000973	0.4200	0.1450	192	0.7889
2	P2	0.000356	0.4990	0.1000	192	0.7750
3	P3	0.000448	0.4770	0.1320	192	0.7927
4	P4	0.000419	0.4980	0.1000	192	0.7927
5	P5	0.000567	0.4160	0.1000	224	0.8122
6	P6	0.000323	0.4350	0.1080	224	0.7965
7	P7	0.000299	0.4850	0.1140	224	0.7864
8	P8	0.000316	0.4600	0.1020	192	0.7535
9	P9	0.000130	0.4550	0.2100	192	0.7611
10	P10	0.001490	0.4810	0.1380	256	0.7949

Pergerakan partikel berlanjut pada iterasi ketiga berdasarkan hasil pembaruan velocity dan posisi dari iterasi sebelumnya. Hasil optimasi menunjukkan bahwa Partikel 5 berhasil mencapai konfigurasi yang lebih baik dengan Val Accuracy sebesar 0.8122 (Learning Rate 0.000567, Dropout CNN 0.4160, Dropout Dense 0.1000, Dense Units 224).

Tabel 4.11 Iterasi Ke-4

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000871	0.3650	0.1000	192	0.8024
2	P2	0.001970	0.5380	0.1000	192	0.7830
3	P3	0.000459	0.4520	0.1230	192	0.8037
4	P4	0.000203	0.4840	0.1000	256	0.7813
5	P5	0.001200	0.3550	0.1000	256	0.7906
6	P6	0.000307	0.4460	0.1000	256	0.7881
7	P7	0.000776	0.3950	0.1130	256	0.8088
8	P8	0.000648	0.4570	0.1470	160	0.7978
9	P9	0.000507	0.3650	0.1200	192	0.7856
10	P10	0.001960	0.5080	0.1000	256	0.7628

Memasuki iterasi keempat, pembaruan posisi kawanan partikel (swarm) kembali dilakukan. Hasil evaluasi pada tahap ini mencatat bahwa performa terbaik dipimpin oleh Partikel 7 yang menghasilkan Val Accuracy sebesar 0.8088 (Learning Rate 0.000776, Dropout CNN 0.3950, Dropout Dense 0.1130, Dense Units 256).

Tabel 4.12 Iterasi Ke-5

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000627	0.3750	0.1000	224	0.7965
2	P2	0.001120	0.4950	0.1000	224	0.8168
3	P3	0.000496	0.4310	0.1140	192	0.7915
4	P4	0.000144	0.4070	0.1000	256	0.7645
5	P5	0.000300	0.3780	0.1000	256	0.7881
6	P6	0.000343	0.4140	0.1000	224	0.7974
7	P7	0.001070	0.3570	0.1070	256	0.8079
8	P8	0.000921	0.4050	0.1080	192	0.7927
9	P9	0.001400	0.3610	0.1000	224	0.7851
10	P10	0.000476	0.4440	0.1040	224	0.7877

Berdasarkan pergerakan partikel dari iterasi keempat, nilai hyperparameter pada iterasi kelima kembali dievaluasi. Pada tahap ini, Partikel 2 menempati posisi tertinggi dengan Val Accuracy sebesar 0.8168 (Learning Rate 0.001120, Dropout CNN 0.4950, Dropout Dense 0.1000, Dense Units 224).

Tabel 4.13 Iterasi Ke-6

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000565	0.4650	0.1000	192	0.8016
2	P2	0.000759	0.4640	0.1000	256	0.8067
3	P3	0.000747	0.4280	0.1000	224	0.7940
4	P4	0.000930	0.4970	0.1000	224	0.7957
5	P5	0.000154	0.4680	0.1000	224	0.7780
6	P6	0.000457	0.4700	0.1000	192	0.7982
7	P7	0.001400	0.4390	0.1040	224	0.7860
8	P8	0.001050	0.4950	0.1000	224	0.8050
9	P9	0.001090	0.4510	0.1000	192	0.7877
10	P10	0.000220	0.4880	0.1380	192	0.7737

Nilai partikel pada iterasi keenam terus disesuaikan melalui mekanisme PSO untuk mendekati area yang lebih optimal. Hasil evaluasi menunjukkan bahwa Partikel 2 kembali memberikan performa tertinggi dengan nilai Val Accuracy sebesar 0.8067 (Learning Rate 0.000759, Dropout CNN 0.4640, Dropout Dense 0.1000, Dense Units 256).

Tabel 4.14 Iterasi Ke-7

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.001460	0.4400	0.1000	224	0.7210
2	P2	0.001330	0.5060	0.1000	256	0.7970
3	P3	0.000514	0.5180	0.1030	224	0.8029
4	P4	0.004550	0.5580	0.1000	192	0.7003
5	P5	0.000995	0.5350	0.1000	192	0.7885
6	P6	0.002020	0.5320	0.1000	224	0.7607
7	P7	0.000752	0.5160	0.1140	224	0.7999
8	P8	0.001210	0.5580	0.1000	256	0.7822
9	P9	0.000942	0.5410	0.1000	192	0.7944
10	P10	0.000523	0.5110	0.1200	224	0.8033

Pembaruan parameter pada iterasi ketujuh mengarahkan pencarian ke konfigurasi baru. Evaluasi pada iterasi ini menunjukkan bahwa Partikel 10 meraih posisi terbaik dengan Val Accuracy sebesar 0.8033 (Learning Rate 0.000523, Dropout CNN 0.5110, Dropout Dense 0.1200, Dense Units 224).

Tabel 4.15 Iterasi Ke-8

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.001540	0.4210	0.1000	224	0.7974
2	P2	0.001750	0.5180	0.1000	224	0.7797
3	P3	0.000612	0.5230	0.1210	192	0.8071
4	P4	0.001440	0.5360	0.1000	256	0.7750
5	P5	0.003280	0.4450	0.1000	224	0.7332
6	P6	0.001750	0.4730	0.1000	224	0.7725
7	P7	0.000728	0.5150	0.1110	224	0.7999
8	P8	0.001300	0.5640	0.1000	224	0.7894
9	P9	0.000970	0.5670	0.1000	192	0.7995
10	P10	0.001770	0.5220	0.1000	224	0.7526

Hasil pembaruan dari iterasi ketujuh diaplikasikan pada iterasi kedelapan. Berdasarkan evaluasi iterasi ini, Partikel 3 mencatatkan hasil paling optimal dengan Val Accuracy sebesar 0.8071 (Learning Rate 0.000612, Dropout CNN 0.5230, Dropout Dense 0.1210, Dense Units 192).

Tabel 4.16 Iterasi Ke-9

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.001070	0.4090	0.1000	224	0.7885
2	P2	0.001050	0.4760	0.1000	192	0.7957
3	P3	0.001390	0.5190	0.1230	192	0.7805
4	P4	0.000363	0.4620	0.1000	256	0.8008
5	P5	0.002250	0.4090	0.1000	256	0.7771
6	P6	0.000541	0.4290	0.1000	224	0.8029
7	P7	0.000749	0.4570	0.1050	224	0.8012
8	P8	0.001210	0.4740	0.1000	224	0.7902
9	P9	0.001100	0.5410	0.1000	256	0.7885
10	P10	0.001920	0.4840	0.1150	224	0.7754

Pada iterasi kesembilan, partikel terus mengeksplorasi ruang pencarian hyperparameter. Hasil evaluasi mencatat bahwa performa terbaik pada iterasi ini dicapai oleh Partikel 6 dengan Val Accuracy sebesar 0.8029 (Learning Rate 0.000541, Dropout CNN 0.4290, Dropout Dense 0.1000, Dense Units 224).

Tabel 4.17 Iterasi Ke-10

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000628	0.4290	0.1000	192	0.7826
2	P2	0.000784	0.4860	0.1000	256	0.7898
3	P3	0.001730	0.4870	0.1200	224	0.7712
4	P4	0.000172	0.4290	0.1000	256	0.7835
5	P5	0.000604	0.4850	0.1000	224	0.7915
6	P6	0.000509	0.4520	0.1000	224	0.8067
7	P7	0.000864	0.3850	0.1010	224	0.7881
8	P8	0.000898	0.4600	0.1000	256	0.7978
9	P9	0.001080	0.4930	0.1000	192	0.7847
10	P10	0.000297	0.4890	0.1170	224	0.7885

Pembaruan velocity dan posisi pada iterasi kesepuluh kembali menunjukkan keunggulan Partikel 6 yang mengamankan posisi global best dengan capaian Val Accuracy sebesar 0.8067 (Learning Rate 0.000509, Dropout CNN 0.4520, Dropout Dense 0.1000, Dense Units 224).

3. Hasil Hyperparameter Terbaik

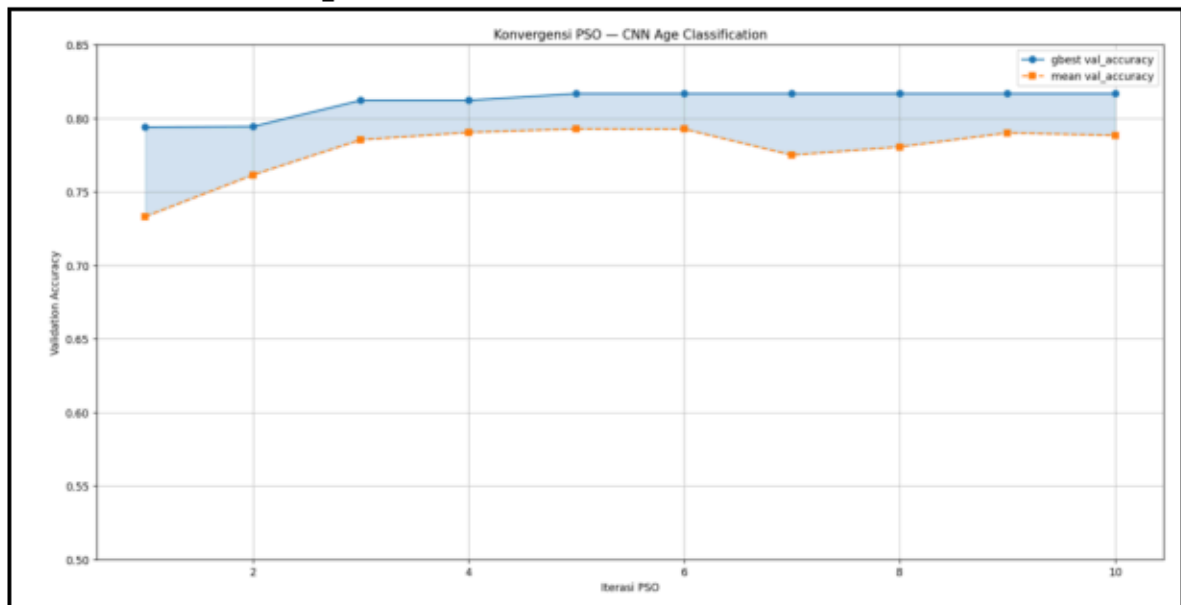
Hyperparameter	CNN Baseline	CNN + PSO (Best)
Learning Rate	0.00092	0,001120
Dropout CNN	0.5	0,4950
Dropout Dense	0.2	0,1000
Dense Units	128	224

Berdasarkan hasil optimasi PSO, diperoleh konfigurasi hyperparameter terbaik berupa learning rate sebesar 0,001120, dropout CNN sebesar 0,4950, dropout dense sebesar

0,1000, dan dense units sebesar 224. Konfigurasi ini diperoleh pada iterasi kelima dan menghasilkan nilai validation accuracy tertinggi sebesar 0,8168.

Jika dibandingkan dengan konfigurasi awal model CNN baseline, hasil PSO menunjukkan adanya perubahan pada beberapa hyperparameter. Nilai dense units meningkat menjadi 224, yang menunjukkan bahwa model membutuhkan kapasitas representasi fitur yang lebih besar dibandingkan konfigurasi baseline. Nilai dropout dense turun hingga 0,1000, yang menunjukkan bahwa regularisasi pada bagian dense layer dibuat lebih rendah agar informasi fitur yang dipelajari tidak terlalu banyak dinonaktifkan selama proses training. Sementara itu, nilai dropout CNN tetap relatif tinggi, yaitu 0,4950, sehingga model masih mempertahankan regularisasi pada bagian ekstraksi fitur.

4. Analisis Konvergensi PSO



Berdasarkan grafik konvergensi PSO pada Gambar 4.X, terlihat bahwa nilai gbest validation accuracy meningkat pada iterasi awal dan kemudian cenderung stabil setelah iterasi kelima. Pada iterasi pertama, nilai gbest berada pada 0,7940, kemudian meningkat menjadi 0,8122 pada iterasi ketiga, dan mencapai nilai terbaik sebesar 0,8168 pada iterasi kelima. Setelah itu, nilai gbest tidak berubah hingga iterasi kesepuluh.

Pola tersebut menunjukkan bahwa PSO mampu menemukan solusi yang lebih baik pada tahap awal proses optimasi. Kenaikan nilai gbest pada iterasi pertama hingga kelima memperlihatkan bahwa partikel berhasil bergerak menuju area solusi yang lebih optimal. Namun, setelah iterasi kelima, kurva gbest menjadi relatif datar. Hal ini menandakan bahwa algoritma telah mencapai kondisi konvergen, karena tidak ditemukan kombinasi hyperparameter baru yang mampu melampaui nilai fitness terbaik sebelumnya.

Selain itu, nilai mean validation accuracy juga menunjukkan tren peningkatan pada beberapa iterasi awal. Hal ini mengindikasikan bahwa secara umum populasi partikel bergerak menuju area solusi yang lebih baik, bukan hanya satu partikel tertentu saja. Meskipun nilai rata-rata sempat mengalami fluktuasi pada iterasi pertengahan, pola akhirnya tetap stabil pada kisaran validation accuracy yang tinggi. Kondisi ini menunjukkan bahwa proses eksplorasi dan eksploitasi pada PSO berjalan cukup seimbang.

Secara keseluruhan, hasil konvergensi menunjukkan bahwa PSO berhasil menemukan kombinasi hyperparameter terbaik dalam 10 iterasi. Akan tetapi, karena nilai gbest sudah tidak berubah sejak iterasi kelima, dapat dikatakan bahwa proses optimasi telah mencapai titik stabil sebelum seluruh iterasi selesai. Dengan demikian, PSO mampu mengarahkan pencarian hyperparameter menuju konfigurasi yang lebih baik, tetapi peningkatan performa setelah titik konvergensi menjadi sangat terbatas.

4.2.4 Hasil Model CNN + PSO 4 Kelas

Untuk melihat proses pembelajaran model CNN hasil optimasi Particle Swarm Optimization (PSO) secara lebih rinci, hasil training model dari epoch 1 hingga epoch 50 disajikan pada Tabel 4.18. Tabel ini memuat nilai accuracy, loss, validation accuracy, dan validation loss pada setiap epoch, sehingga perkembangan performa model selama proses pelatihan dapat diamati secara lebih jelas.

Tabel 4.18 Hasil Training Model CNN + PSO Jurnal 4 Kelas per Epoch

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
1	0.5384	1.2365	0.6442	1.0060
2	0.6287	1.0102	0.6927	0.8888
3	0.6668	0.9198	0.6788	0.9274
4	0.6896	0.8642	0.7016	0.8979
5	0.7069	0.8317	0.7345	0.7591
6	0.7239	0.7949	0.7661	0.7246
7	0.7346	0.7649	0.7488	0.7584
8	0.7412	0.7527	0.7493	0.7259
9	0.7490	0.7345	0.7889	0.6376
10	0.7627	0.7057	0.7678	0.6702
11	0.7687	0.6953	0.7961	0.6172
12	0.7677	0.6901	0.7889	0.6250
13	0.7771	0.6754	0.8029	0.5977
14	0.7797	0.6663	0.8147	0.6012
15	0.7881	0.6553	0.8024	0.6005
16	0.7890	0.6582	0.8143	0.5767
17	0.7957	0.6352	0.8155	0.5717
18	0.7922	0.6413	0.8155	0.5834
19	0.7925	0.6424	0.8181	0.5763
20	0.7992	0.6265	0.8164	0.5576
21	0.7960	0.6355	0.8227	0.5567
22	0.8038	0.6192	0.8122	0.5908
23	0.8062	0.6181	0.8214	0.5659
24	0.8082	0.6148	0.8202	0.5495
25	0.8053	0.6158	0.8151	0.5659
26	0.8005	0.6282	0.8307	0.5534
27	0.8101	0.6022	0.8282	0.5476
28	0.8111	0.6008	0.8320	0.5456
29	0.8124	0.6030	0.8324	0.5369
30	0.8128	0.5944	0.7775	0.6559

Tabel 4.19 Hasil Training Model CNN + PSO Jurnal 4 Kelas per Epoch

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
31	0.8125	0.6015	0.8244	0.5479
32	0.8156	0.5924	0.8392	0.5258
33	0.8199	0.5879	0.8088	0.5606
34	0.8140	0.5954	0.8371	0.5393
35	0.8185	0.5886	0.8328	0.5327
36	0.8184	0.5831	0.8143	0.5870
37	0.8175	0.5836	0.8362	0.5486
38	0.8285	0.5565	0.8451	0.5161
39	0.8313	0.5448	0.8468	0.5071
40	0.8327	0.5398	0.8442	0.4981
41	0.8316	0.5419	0.8379	0.5203
42	0.8317	0.5427	0.8312	0.5191
43	0.8331	0.5367	0.8455	0.5101
44	0.8346	0.5344	0.8425	0.5212
45	0.8333	0.5403	0.8472	0.5001
46	0.8379	0.5173	0.8455	0.4938
47	0.8427	0.5107	0.8510	0.4843
48	0.8421	0.5107	0.8523	0.4893
49	0.8434	0.5090	0.8468	0.4842
50	0.8417	0.5108	0.8468	0.5069

Berdasarkan Tabel 4.18, performa model CNN + PSO jurnal 4 kelas menunjukkan peningkatan yang cukup konsisten selama proses training. Nilai accuracy pada data latih meningkat dari 0,5384 pada epoch pertama menjadi 0,8417 pada epoch ke-50. Sementara itu, nilai loss mengalami penurunan dari 1,2365 menjadi 0,5108. Hal ini menunjukkan bahwa model mampu mempelajari pola data secara bertahap selama proses pelatihan.

Pada data validasi, nilai validation accuracy juga menunjukkan peningkatan dari 0,6442 pada epoch pertama hingga mencapai nilai terbaik sebesar 0,8523 pada epoch ke-48. Nilai validation loss terendah diperoleh pada epoch ke-49, yaitu sebesar 0,4842. Hasil ini menunjukkan bahwa model hasil optimasi PSO mampu menghasilkan performa validasi yang stabil pada akhir proses pelatihan.

Setelah proses training selesai, model CNN yang telah dioptimasi menggunakan Particle Swarm Optimization kemudian diuji menggunakan data uji untuk mengevaluasi performa klasifikasi kelompok usia secara menyeluruh. Ringkasan hasil evaluasi model disajikan pada Tabel 4.20.

Tabel 4.20 Metrik Evaluasi Model CNN + PSO Jurnal 4 Kelas

Metrik	Nilai
Accuracy	83,99%
Precision	77,85%
Recall	72,94%
F1-Score	74,88%

Nilai accuracy sebesar 83,99% menunjukkan bahwa model CNN + PSO jurnal 4 kelas mampu melakukan klasifikasi kelompok usia dengan cukup baik secara keseluruhan. Namun, nilai precision, recall, dan F1-Score yang lebih rendah dibandingkan accuracy menunjukkan bahwa performa model belum sepenuhnya merata pada seluruh kelas. Hal ini terutama dipengaruhi oleh rendahnya performa pada kelas teenager, yang memiliki jumlah data paling sedikit dibandingkan kelas lainnya.

1. Evaluasi Per Kelas

Tabel 4.21 Evaluasi Per Kelas Model CNN + PSO Jurnal 4 Kelas

Kelas	Precision	Recall	F1-Score
child	96,3%	90,9%	93,5%
teenager	50,0%	32,1%	39,1%
adult	84,5%	88,8%	86,6%
old	80,6%	79,9%	80,3%

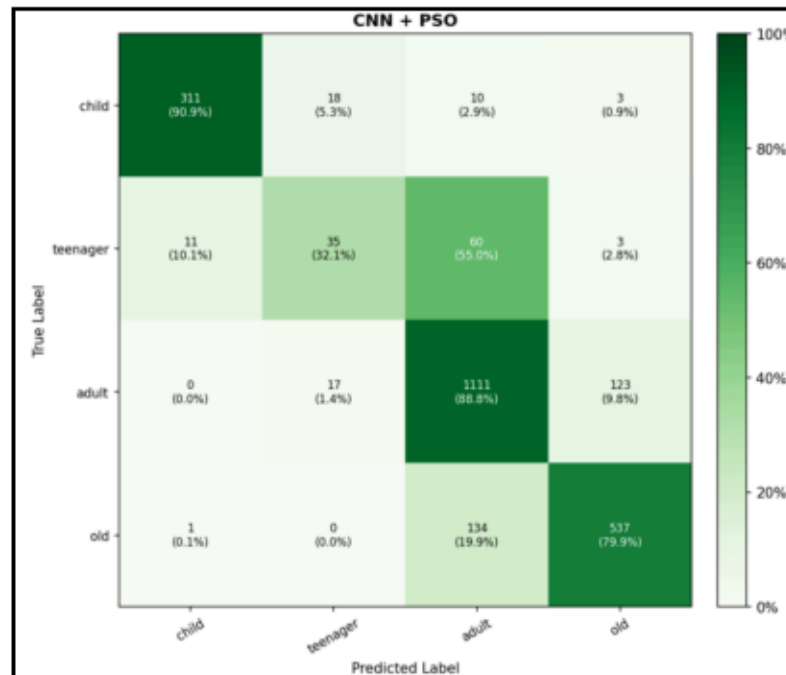
Berdasarkan hasil evaluasi per kelas yang terlihat pada Tabel 4.21, terlihat bahwa performa model CNN + PSO jurnal 4 kelas masih dipengaruhi oleh distribusi data pada masing-masing kelas. Kelas child memperoleh performa terbaik dengan precision sebesar 96,3%, recall sebesar 90,9%, dan F1-Score sebesar 93,5%. Hal ini menunjukkan bahwa model mampu mengenali karakteristik wajah pada kelas child dengan sangat baik.

Kelas adult juga menunjukkan performa yang tinggi dengan precision sebesar 84,5%, recall sebesar 88,8%, dan F1-Score sebesar 86,6%. Nilai recall yang tinggi menunjukkan bahwa sebagian besar data adult berhasil dikenali dengan benar oleh model. Hal ini dapat dipengaruhi oleh jumlah data adult yang paling besar, sehingga model memperoleh representasi fitur yang lebih banyak selama proses training.

Kelas old memperoleh precision sebesar 80,6%, recall sebesar 79,9%, dan F1-Score sebesar 80,3%. Hasil ini menunjukkan bahwa model cukup mampu mengenali kelas old, meskipun masih terdapat kesalahan klasifikasi antara kelas old dan adult. Kesalahan ini dapat terjadi karena pada skenario 4 kelas, kelas old mencakup seluruh data usia di atas 40 tahun, sehingga terdapat kemungkinan kemiripan fitur wajah dengan kelas adult, terutama pada batas usia yang berdekatan.

Sebaliknya, kelas teenager masih menjadi kelas dengan performa terendah dibandingkan kelas lainnya. Kelas ini memperoleh precision sebesar 50,0%, recall sebesar 32,1%, dan F1-Score sebesar 39,1%. Nilai recall yang rendah menunjukkan bahwa sebagian besar data teenager belum berhasil dikenali dengan baik oleh model. Berdasarkan confusion matrix, sebagian besar data teenager masih salah diklasifikasikan sebagai adult. Hal ini menunjukkan bahwa meskipun PSO meningkatkan performa model secara umum, tantangan pada kelas minoritas masih tetap ada.

2. Analisis Confusion Matrix



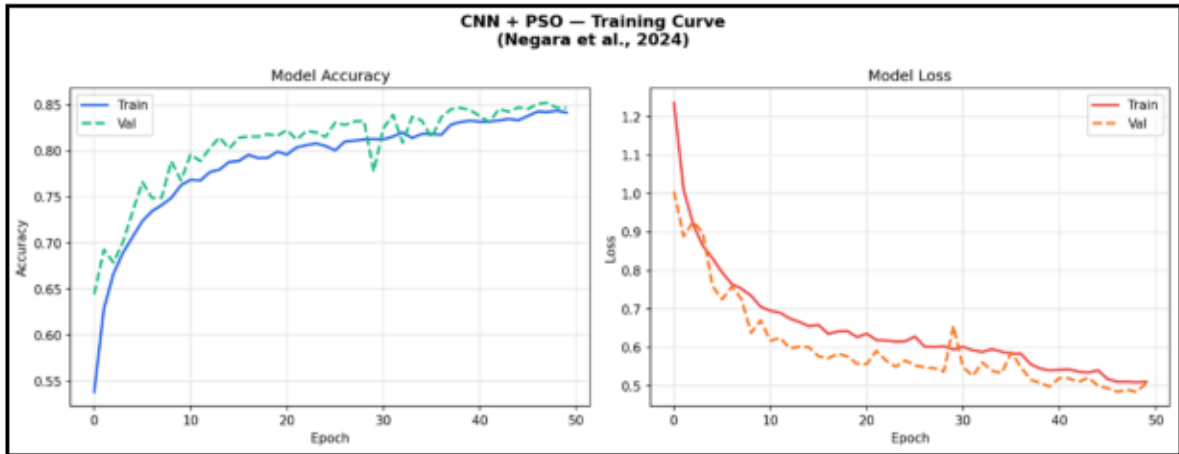
Gambar 4.1 Confusion Matrix CNN + PSO

Berdasarkan confusion matrix model CNN + PSO jurnal 4 kelas, terlihat bahwa prediksi benar paling dominan terdapat pada kelas child, adult, dan old. Pada kelas child, sebanyak 311 data berhasil diklasifikasikan dengan benar dari total 342 data, atau sekitar 90,9%. Pada kelas adult, sebanyak 1.111 data berhasil diklasifikasikan dengan benar dari total 1.251 data, atau sekitar 88,8%. Sementara itu, pada kelas old, sebanyak 537 data berhasil diklasifikasikan dengan benar, atau sekitar 79,9%.

Kesalahan klasifikasi paling dominan masih terjadi pada kelas teenager. Dari total 109 data teenager, hanya 35 data yang berhasil diklasifikasikan dengan benar, sedangkan 60 data salah diklasifikasikan sebagai adult. Pola ini menunjukkan bahwa model masih mengalami kesulitan dalam membedakan karakteristik wajah remaja dan dewasa. Hal ini dapat disebabkan oleh kemiripan fitur visual antara kedua kelas tersebut serta jumlah data teenager yang jauh lebih sedikit dibandingkan kelas adult.

Selain itu, kesalahan klasifikasi juga terlihat antara kelas adult dan old. Sebanyak 123 data adult diklasifikasikan sebagai old, sedangkan 134 data old diklasifikasikan sebagai adult. Hal ini menunjukkan adanya tumpang tindih karakteristik visual antara usia dewasa akhir dan usia tua, khususnya karena pada skenario 4 kelas, kelas old mencakup seluruh usia di atas 40 tahun.

3. Analisis Training Curve



Gambar 4.2 Training Curve CNN + PSO

Berdasarkan grafik training curve model CNN + PSO jurnal 4 kelas, terlihat bahwa nilai accuracy pada data latih dan data validasi mengalami peningkatan seiring bertambahnya epoch. Peningkatan paling signifikan terjadi pada epoch awal, khususnya dari epoch 1 hingga sekitar epoch 15. Setelah itu, kurva mulai melandai dan cenderung stabil hingga akhir pelatihan. Kondisi ini menunjukkan bahwa model berhasil mempelajari pola data dengan baik dan secara bertahap mencapai kondisi konvergen.

Dari sisi loss, baik training loss maupun validation loss menunjukkan tren penurunan hingga akhir pelatihan. Training loss menurun dari 1,2365 menjadi 0,5108, sedangkan validation loss menurun dari 1,0060 hingga nilai terendah 0,4842 pada epoch ke-49. Walaupun terdapat fluktuasi pada beberapa epoch, terutama pada validation accuracy dan validation loss, pola umumnya menunjukkan proses pelatihan yang stabil.

Selain itu, jarak antara kurva training dan validation tidak terlalu jauh. Bahkan, pada beberapa epoch, nilai validation accuracy lebih tinggi dibandingkan training accuracy. Kondisi ini dapat terjadi karena data training menerima augmentasi selama proses pelatihan, sehingga data latih menjadi lebih bervariasi dan lebih sulit dibandingkan data validasi. Dengan demikian, training curve tidak menunjukkan indikasi overfitting yang kuat.

Secara keseluruhan, hasil pengujian menunjukkan bahwa model CNN + PSO jurnal 4 kelas mampu memberikan performa klasifikasi yang baik dengan accuracy sebesar 83,99%. Namun, kelemahan utama model masih terdapat pada kelas teenager, yang tetap menjadi kelas paling sulit dikenali akibat jumlah data yang sedikit dan kemiripan karakteristik wajah dengan kelas adult.

4.2.5 Hasil Model CNN Baseline 5 Kelas

Untuk melihat proses pembelajaran model CNN baseline secara rinci, hasil training dari epoch 1 hingga epoch 50 disajikan pada Tabel 4.22. Tabel ini memuat nilai accuracy, loss, validation accuracy, dan validation loss pada setiap epoch, sehingga perkembangan performa model selama pelatihan dapat diamati secara lebih jelas.

Tabel 4.22 Hasil Training Model CNN Baseline per Epoch

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
1	0.5235	1.4304	0.5618	1.3490
2	0.5552	1.2796	0.5969	1.1087
3	0.6013	1.1370	0.6678	0.9461
4	0.6315	1.0619	0.6881	0.8894
5	0.6545	1.0000	0.7003	0.8761
6	0.6646	0.9646	0.7113	0.8368
7	0.6751	0.9386	0.7315	0.7946
8	0.6862	0.9108	0.7358	0.7946
9	0.6930	0.8941	0.7104	0.8497
10	0.6970	0.8801	0.7298	0.7736
11	0.7071	0.8585	0.7429	0.7473
12	0.7073	0.8548	0.7585	0.7567
13	0.7132	0.8402	0.7602	0.7274
14	0.7225	0.8156	0.7041	0.8426
15	0.7237	0.8242	0.7509	0.7345
16	0.7254	0.8082	0.7704	0.6988
17	0.7304	0.8074	0.7708	0.7026
18	0.7338	0.7975	0.7607	0.7318
19	0.7252	0.8103	0.7809	0.6869
20	0.7320	0.7964	0.7640	0.6989
21	0.7397	0.7735	0.7653	0.6931
22	0.7399	0.7692	0.7843	0.6641
23	0.7421	0.7711	0.7801	0.6664
24	0.7449	0.7656	0.7830	0.6635
25	0.7477	0.7574	0.7763	0.6714
26	0.7445	0.7688	0.7873	0.6615
27	0.7522	0.7464	0.7531	0.7275
28	0.7487	0.7530	0.7961	0.6423
29	0.7509	0.7456	0.7809	0.6604
30	0.7523	0.7529	0.7847	0.6555
31	0.7550	0.7405	0.7336	0.7517
32	0.7540	0.7398	0.7902	0.6471
33	0.7573	0.7374	0.7911	0.6494
34	0.7641	0.7165	0.7957	0.6257
35	0.7689	0.7064	0.7974	0.6269
36	0.7674	0.7019	0.7949	0.6239
37	0.7673	0.7029	0.8012	0.6221
38	0.7728	0.6900	0.8016	0.6262
39	0.7750	0.6887	0.8008	0.6126
40	0.7706	0.6859	0.7970	0.6200
41	0.7728	0.6863	0.8054	0.6211
42	0.7766	0.6783	0.8016	0.6262
43	0.7752	0.6814	0.7927	0.6241
44	0.7739	0.6798	0.7716	0.7011
45	0.7758	0.6751	0.7995	0.6006

Tabel 4.23 Hasil Training Model CNN Baseline per Epoch

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
46	0.7822	0.6600	0.8062	0.5938
47	0.7821	0.6532	0.8058	0.6029
48	0.7832	0.6555	0.8054	0.5913
49	0.7825	0.6587	0.7991	0.5950
50	0.7823	0.6529	0.8062	0.5948

Berdasarkan Tabel 4.22, performa model CNN baseline menunjukkan peningkatan yang cukup konsisten selama proses training. Nilai accuracy cenderung meningkat dan loss menurun seiring bertambahnya epoch. Pada data validasi, validation accuracy juga mengalami peningkatan hingga mencapai nilai terbaik sebesar 0.8062 pada epoch 46. Hal ini menunjukkan bahwa model telah mempelajari pola data dengan cukup baik dan mencapai performa optimal pada epoch tersebut.

Setelah proses training selesai, model kemudian diuji menggunakan data uji untuk mengevaluasi performa akhir dalam klasifikasi kelompok usia. Ringkasan hasil evaluasi model CNN baseline disajikan pada Tabel 4.24.

Tabel 4.24 Metrik Evaluasi Model CNN Baseline (Macro Average)

Metrik	Nilai
Accuracy	79.20%
Precision	72.85%
Recall	65.31%
<i>F1-Score</i>	67.22%

Nilai accuracy sebesar 79.20% menunjukkan bahwa model secara umum mampu melakukan klasifikasi dengan cukup baik. Namun, nilai recall dan *F1-Score* yang relatif lebih rendah mengindikasikan bahwa performa model belum merata pada seluruh kelas.

1. Evaluasi Per Kelas

Tabel 4.25 Evaluasi per Kelas

Kelas	Precision	Recall	<i>F1-Score</i>
child	91.7%	93.6%	92.6%
teenager	54.1%	18.3%	27.4%
adult	82.8%	91.2%	86.8%
Middle age	59.3%	53.7%	56.4%
old	76.4%	69.7%	72.9%

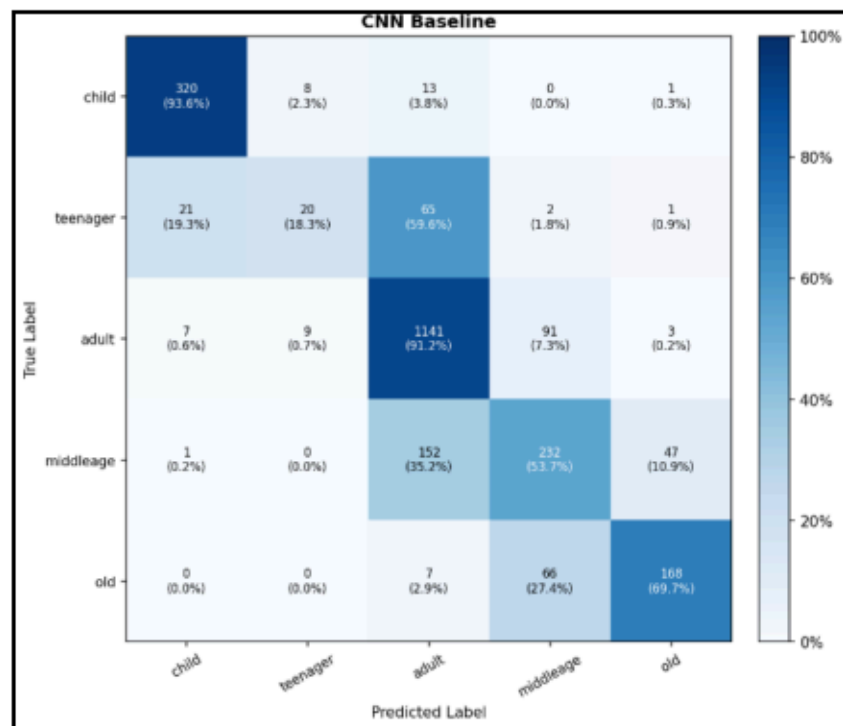
Berdasarkan hasil evaluasi, terlihat bahwa performa model sangat dipengaruhi oleh distribusi data pada masing-masing kelas. Kelas child dan adult menunjukkan performa yang sangat baik dengan nilai precision, recall, dan *F1-Score* yang tinggi. Hal

ini menunjukkan bahwa model mampu mengenali pola pada kelas mayoritas dengan baik.

Sebaliknya, kelas teenager menunjukkan performa yang paling rendah dibandingkan kelas lainnya. Nilai recall yang sangat rendah, yaitu hanya 18.3%, menunjukkan bahwa sebagian besar data teenager gagal dikenali oleh model. Hal ini berdampak langsung pada rendahnya nilai *F1-Score* sebesar 27.4%, yang mengindikasikan ketidakseimbangan antara precision dan recall pada kelas tersebut.

Kondisi ini mengindikasikan bahwa model cenderung bias terhadap kelas mayoritas dan mengalami kesulitan dalam mempelajari karakteristik kelas minoritas. Hal ini konsisten dengan hasil analisis preprocessing yang menunjukkan bahwa jumlah data teenager jauh lebih sedikit dibandingkan kelas lainnya.

2. Analisis Confusion Matrix



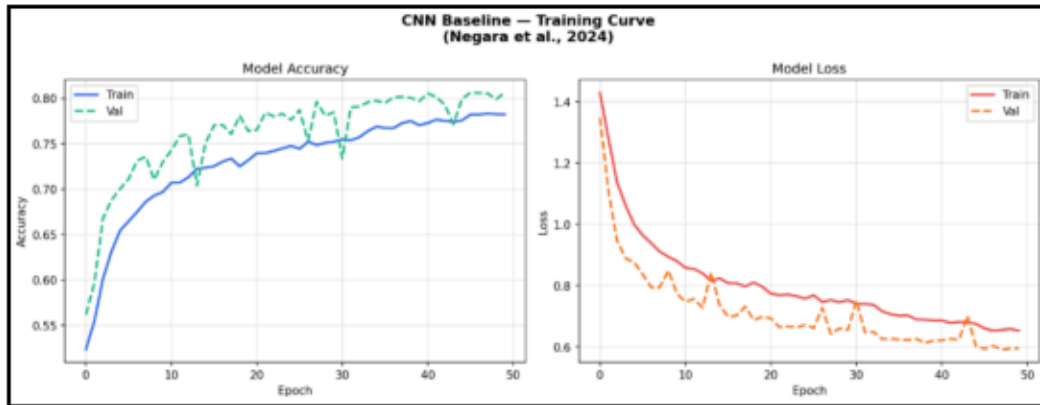
Gambar 4.3 Confusion Matrix Baseline CNN

Berdasarkan confusion matrix pada Gambar 4.3, terlihat bahwa kesalahan klasifikasi paling dominan terjadi pada kelas teenager, di mana sebagian besar data diklasifikasikan sebagai adult. Hal ini menunjukkan bahwa model mengalami kesulitan dalam membedakan karakteristik wajah antara remaja dan dewasa yang memiliki kemiripan fitur visual.

Selain itu, kesalahan juga terlihat pada kelas Middle Age, di mana sejumlah data diklasifikasikan sebagai adult. Hal ini menunjukkan bahwa batas antar kelas usia yang berdekatan menjadi tantangan bagi model dalam proses klasifikasi.

Pada kelas old, meskipun performanya cukup baik, masih terdapat kesalahan klasifikasi ke kelas Middle Age, yang menunjukkan adanya overlap karakteristik antara kedua kelompok usia tersebut.

3. Analisis Training Curve



Gambar 4.4 Training Curve Baseline CNN

Berdasarkan grafik training curve, terlihat bahwa model mengalami peningkatan performa yang signifikan pada epoch awal, khususnya pada rentang epoch 1 hingga 10. Setelah itu, peningkatan accuracy cenderung melambat dan memasuki fase konvergensi.

Selain itu, tidak terdapat perbedaan yang signifikan antara kurva training dan validation, yang menunjukkan bahwa model tidak mengalami overfitting secara signifikan. Namun, stagnasi performa setelah epoch tertentu mengindikasikan bahwa model telah mencapai batas kemampuannya dengan konfigurasi hyperparameter yang digunakan.

4.2.6 Hasil Optimasi PSO 5 Kelas

Pada tahap ini, algoritma Particle Swarm Optimization (PSO) diterapkan untuk mencari kombinasi hyperparameter terbaik pada model CNN baseline. Optimasi dilakukan terhadap empat hyperparameter, yaitu learning rate, dropout CNN, dropout dense, dan jumlah unit pada dense layer. Setiap partikel merepresentasikan satu kombinasi hyperparameter yang kemudian dievaluasi menggunakan nilai validation accuracy sebagai fungsi fitness. Berdasarkan hasil optimasi, PSO berhasil menghasilkan konfigurasi hyperparameter terbaik yang berbeda dari model baseline dan selanjutnya digunakan untuk proses retraining model CNN.

1. Parameter Optimasi PSO

Tabel 4.26 Parameter Particle Swarm Optimization (PSO)

Parameter	Nilai
Jumlah Partikel (<i>Swarm Size</i>)	10 Partikel
Jumlah Iterasi	10 Iterasi
Bobot Inersia (w)	0.7
Koefisien Kognitif ($c1$)	1.5
Koefisien Sosial ($c2$)	1.5

Berdasarkan parameter tersebut, proses optimasi dilakukan menggunakan 10 partikel selama 10 iterasi. Konfigurasi ini menghasilkan total 100 evaluasi kandidat

solusi selama proses pencarian. Penggunaan bobot inersia sebesar 0.7 serta koefisien kognitif dan sosial masing-masing sebesar 1.5 bertujuan untuk menjaga keseimbangan antara eksplorasi ruang pencarian dan eksploitasi solusi terbaik yang telah ditemukan selama iterasi berlangsung.

2. Hasil Evaluasi Partikel pada Setiap Iterasi PSO

Proses optimasi menggunakan algoritma Particle Swarm Optimization (PSO) dilakukan melalui 10 iterasi dengan melibatkan 10 partikel pada setiap iterasinya. Setiap partikel membawa kombinasi nilai empat hyperparameter utama, yaitu learning rate, dropout CNN, dropout dense, dan dense units, yang kemudian dievaluasi kinerjanya berdasarkan nilai validation accuracy sebagai fungsi fitness.

Tabel 4.27 Tabel Iterasi ke-1

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000103	0.585	0.320	192	0.6965
2	P2	0.000026	0.347	0.117	224	0.6518
3	P3	0.000420	0.512	0.106	256	0.7450
4	P4	0.001770	0.364	0.155	96	0.7166
5	P5	0.000066	0.457	0.230	128	0.6872
6	P6	0.000449	0.342	0.188	128	0.7480
7	P7	0.000170	0.536	0.160	160	0.7231
8	P8	0.000398	0.314	0.282	96	0.7213
9	P9	0.000015	0.585	0.390	224	0.5276
10	P10	0.000066	0.329	0.305	160	0.6475

Nilai partikel P1 sampai P10 pada iterasi pertama diperoleh dari proses inisialisasi awal partikel secara acak menggunakan distribusi uniform pada ruang pencarian hyperparameter yang telah ditentukan. Dengan demikian, iterasi pertama merupakan populasi awal PSO yang menjadi titik awal pencarian solusi, sedangkan iterasi berikutnya diperoleh melalui proses pembaruan posisi dan kecepatan berdasarkan rumus PSO.

Setelah nilai Partikel 1 pada iterasi ke-1 diperoleh, langkah berikutnya adalah menghitung nilai Partikel 1 pada iterasi ke-2 menggunakan persamaan pembaruan PSO. Sebagai contoh, pada dimensi ke-3 yang merepresentasikan dropout_dense, nilai posisi Partikel 1 pada iterasi ke-1 adalah $x_{1,3}^{(1)} = 0.31959818$. Karena iterasi pertama merupakan evaluasi awal, maka nilai personal best untuk Partikel 1 pada dimensi ini masih sama dengan posisi awalnya, yaitu $pBest_{1,3} = 0.31959818$. Sementara itu, berdasarkan hasil evaluasi fitness pada iterasi ke-1, partikel terbaik adalah P6, sehingga nilai global best pada dimensi ke-3 diambil dari Partikel 6, yaitu $gBest_3 = 0.18764339$. Kecepatan awal Partikel 1 pada dimensi ke-3 adalah $v_{1,3}^{(1)} = -0.02793669$. Parameter PSO yang digunakan adalah $w = 0.7$, $c_1 = 1.5$, dan $c_2 =$

1.5, dengan bilangan acak pada dimensi ke-3 sebesar $r_{1,3} = 0.33089802$ dan $r_{2,3} = 0.31800347$.

Berdasarkan nilai-nilai tersebut, pembaruan kecepatan untuk dimensi ke-3 dihitung menggunakan persamaan 2.20, Setelah disubstitusikan, diperoleh:

$$v_{1,3}^{(2)} = 0.7 \cdot (-0.02793669) + 1.5 \cdot 0.33089802 \cdot (0.31959818 - 0.31959818) + 1.5 \cdot 0.31800347 \cdot (0.18764339 - 0.31959818)$$

$$v_{1,3}^{(2)} = -0.01955568 + 1.5 \cdot 0.33089802 \cdot 0 + 1.5 \cdot 0.31800347 \cdot (-0.13195479)$$

$$v_{1,3}^{(2)} = -0.01955568 + 0 - 0.06294244$$

$$v_{1,3}^{(2)} = -0.08249812$$

Namun, pada kode program diterapkan velocity clamp untuk membatasi kecepatan partikel. Untuk dimensi ke-3, batas kecepatan maksimum dihitung dari rentang domain dropout_dense, yaitu:

$$v_{\max,3} = (0.4 - 0.1) \times 0.3 = 0.09$$

Karena hasil perhitungan -0.08249812 masih berada dalam rentang $[-0.09, 0.09]$, maka nilai velocity tidak berubah, sehingga:

$$v_{1,3}^{(2)} = -0.08249812$$

Selanjutnya, posisi baru Partikel 1 pada dimensi ke-3 dihitung dengan persamaan:

$$x_{1,3}^{(2)} = x_{1,3}^{(1)} + v_{1,3}^{(2)}$$

$$x_{1,3}^{(2)} = 0.31959818 + (-0.08249812)$$

$$x_{1,3}^{(2)} = 0.23710006$$

Setelah dibulatkan tiga angka di belakang koma, diperoleh nilai:

$$x_{1,3}^{(2)} \approx 0.237$$

Dengan demikian, nilai dropout_dense untuk Partikel 1 pada iterasi ke-2 adalah 0,237. Nilai ini sesuai dengan hasil yang ditampilkan pada tabel iterasi ke-2, sehingga dapat disimpulkan bahwa perubahan nilai tersebut diperoleh dari proses

pembaruan PSO berdasarkan posisi sebelumnya, personal best, global best, dan pembatasan kecepatan partikel.

Tabel 4.28 Tabel Iterasi ke-2

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000441	0.495	0.237	192	0.7518
2	P2	0.000056	0.350	0.196	160	0.6868
3	P3	0.000437	0.422	0.177	224	0.7387
4	P4	0.002030	0.369	0.205	128	0.7429
5	P5	0.000205	0.367	0.188	128	0.7239
6	P6	0.000407	0.332	0.201	128	0.7404
7	P7	0.000213	0.527	0.170	160	0.7222
8	P8	0.000277	0.346	0.192	96	0.7137
9	P9	0.000020	0.495	0.300	192	0.5276
10	P10	0.000429	0.326	0.257	128	0.7244

Nilai partikel pada iterasi kedua bukan dibangkitkan ulang secara acak, tetapi diperoleh dari hasil pergerakan partikel pada iterasi pertama menggunakan rumus PSO. Untuk menunjukkan mekanisme tersebut secara lebih jelas, berikut diberikan contoh perhitungan perubahan Partikel 1 dari iterasi ke-1 ke iterasi ke-2. Contoh ini mencakup posisi awal partikel, penentuan pBest dan gBest, pembaruan velocity, pembaruan posisi, hingga proses decoding menjadi hyperparameter aktual.

Tabel 4.29 Tabel Iterasi ke-3

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.00122	0.432	0.179	192	0.7493
2	P2	0.000364	0.355	0.257	128	0.7480
3	P3	0.000449	0.457	0.204	160	0.7341
4	P4	0.000502	0.415	0.275	160	0.6775
5	P5	0.000658	0.448	0.206	192	0.7556
6	P6	0.000433	0.422	0.197	160	0.6973
7	P7	0.000305	0.518	0.260	160	0.7235
8	P8	0.000534	0.436	0.195	160	0.7328
9	P9	0.000132	0.518	0.264	192	0.7045
10	P10	0.0016	0.329	0.204	128	0.7159

Pada iterasi ketiga, global best kembali mengalami peningkatan menjadi 0,7556 melalui partikel P5. Sebagian besar partikel sudah berada pada rentang validation accuracy sekitar 0,70 hingga 0,75, yang menunjukkan bahwa proses pencarian mulai lebih terarah dibandingkan iterasi awal. Hal ini mengindikasikan bahwa PSO mulai mampu mengidentifikasi kombinasi hyperparameter yang efektif untuk meningkatkan performa model.

Tabel 4.30 Tabel Iterasi ke-4

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000259	0.486	0.233	192	0.7455
2	P2	0.00224	0.363	0.264	160	0.7201
3	P3	0.000489	0.544	0.180	160	0.7358
4	P4	0.00157	0.443	0.293	192	0.7383
5	P5	0.00149	0.504	0.220	224	0.7239
6	P6	0.000482	0.386	0.200	192	0.7552
7	P7	0.000906	0.428	0.282	192	0.7446
8	P8	0.00108	0.504	0.205	192	0.7256
9	P9	0.000517	0.432	0.174	192	0.7429
10	P10	0.000482	0.359	0.224	160	0.7573

Pada iterasi keempat, nilai validation accuracy terbaik kembali naik tipis menjadi 0,7573 yang diperoleh partikel P10. Peningkatan ini menunjukkan bahwa PSO masih mampu menemukan perbaikan solusi, meskipun laju peningkatannya mulai melambat. Variasi nilai dropout yang semakin beragam pada iterasi ini menunjukkan bahwa algoritma masih menyeimbangkan proses eksplorasi dan eksploitasi untuk memperoleh konfigurasi yang lebih optimal.

Tabel 4.31 Tabel Iterasi ke-5

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000194	0.411	0.268	160	0.7066
2	P2	0.000347	0.358	0.204	128	0.7459
3	P3	0.000469	0.574	0.130	224	0.7509
4	P4	0.00296	0.364	0.292	160	0.6745
5	P5	0.00023	0.479	0.224	224	0.7387
6	P6	0.00052	0.333	0.224	160	0.7358
7	P7	0.000967	0.338	0.272	192	0.7400
8	P8	0.00048	0.414	0.232	160	0.7455
9	P9	0.00129	0.342	0.136	128	0.7442
10	P10	0.000208	0.380	0.237	160	0.7168

Pada iterasi kelima, nilai validation accuracy partikel cenderung berada pada kisaran yang relatif stabil di sekitar 0,74 hingga 0,75, meskipun masih terdapat beberapa partikel dengan performa lebih rendah. Kondisi ini menunjukkan bahwa PSO mulai memasuki fase pencarian yang lebih sempit, di mana sebagian besar partikel telah bergerak mendekati area solusi yang baik. Fluktuasi yang masih muncul pada beberapa partikel menunjukkan bahwa eksplorasi tetap berlangsung untuk menghindari terjebak pada local optimum.

Tabel 4.32 Tabel Iterasi ke-6

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000203	0.396	0.244	160	0.7260
2	P2	0.000143	0.352	0.221	128	0.6919
3	P3	0.000463	0.568	0.204	192	0.7307
4	P4	0.000848	0.307	0.202	128	0.7438
5	P5	0.0001	0.389	0.209	160	0.7020
6	P6	0.000494	0.384	0.237	160	0.7121
7	P7	0.000368	0.361	0.251	160	0.7353
8	P8	0.000272	0.324	0.251	96	0.7048
9	P9	0.00231	0.302	0.179	128	0.7265
10	P10	0.000139	0.352	0.213	160	0.7075

Pada iterasi keenam, nilai validation accuracy tidak menunjukkan peningkatan global best yang berarti, namun distribusi performa partikel masih berada pada kisaran yang kompetitif. Hal ini menunjukkan bahwa PSO sedang berada dalam fase penyesuaian posisi partikel di sekitar ruang solusi yang telah ditemukan sebelumnya. Dengan kata lain, proses optimasi masih berjalan, tetapi lebih berfokus pada penyempurnaan solusi dibandingkan pencarian area baru secara luas.

Tabel 4.33 Tabel Iterasi ke-7

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000941	0.468	0.193	160	0.7505
2	P2	0.000801	0.357	0.237	128	0.7480
3	P3	0.000469	0.478	0.229	160	0.7349
4	P4	0.000152	0.311	0.164	128	0.7003
5	P5	0.000647	0.322	0.204	160	0.7358
6	P6	0.000448	0.399	0.224	192	0.7277
7	P7	0.000606	0.393	0.273	160	0.7476
8	P8	0.000641	0.400	0.225	160	0.7472
9	P9	0.000421	0.374	0.264	128	0.7218
10	P10	0.000271	0.341	0.224	128	0.7324

Pada iterasi ketujuh, beberapa partikel kembali mencapai nilai validation accuracy yang tinggi, seperti P1 sebesar 0,7505, P2 sebesar 0,7480, dan P7 sebesar 0,7476. Meskipun belum melampaui global best sebelumnya, hasil ini menunjukkan bahwa kualitas solusi yang ditemukan PSO tetap konsisten. Kondisi tersebut menandakan bahwa algoritma telah berhasil mempertahankan partikel-partikel pada wilayah solusi yang baik dan semakin mendekati fase konvergensi.

Tabel 4.34 Tabel Iterasi ke-8

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.00104	0.482	0.174	160	0.7244
2	P2	0.00161	0.360	0.255	160	0.7239
3	P3	0.000483	0.388	0.170	192	0.7434
4	P4	0.000376	0.316	0.254	160	0.7374
5	P5	0.00233	0.412	0.226	160	0.7180
6	P6	0.000448	0.378	0.197	192	0.7493
7	P7	0.000694	0.396	0.255	160	0.7509
8	P8	0.00115	0.438	0.205	192	0.7472
9	P9	0.000193	0.396	0.174	160	0.7155
10	P10	0.00122	0.345	0.232	128	0.7387

Pada iterasi kedelapan, performa partikel masih berada pada kisaran yang relatif baik, dengan beberapa partikel kembali menghasilkan validation accuracy mendekati 0,75, seperti P6 sebesar 0,7493 dan P7 sebesar 0,7509. Hasil ini menunjukkan bahwa PSO masih mampu menjaga kestabilan proses pencarian, meskipun peningkatan nilai terbaik belum terlalu signifikan. Dengan demikian, iterasi ini memperlihatkan bahwa proses optimasi mulai cenderung stabil sambil tetap membuka peluang ditemukannya solusi yang lebih baik.

Tabel 4.35 Tabel Iterasi ke-9

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000424	0.451	0.194	160	0.7455
2	P2	0.000374	0.355	0.242	160	0.7328
3	P3	0.000475	0.456	0.149	224	0.7560
4	P4	0.00175	0.325	0.278	160	0.7218
5	P5	0.00169	0.448	0.221	192	0.7206
6	P6	0.000494	0.374	0.193	192	0.7446
7	P7	0.000737	0.344	0.226	128	0.7210
8	P8	0.00106	0.368	0.217	128	0.7163
9	P9	0.000268	0.346	0.114	160	0.7273
10	P10	0.000939	0.376	0.224	160	0.7434

Pada iterasi kesembilan, nilai validation accuracy terbaik meningkat kembali menjadi 0,7560 yang dicapai oleh partikel P3. Meskipun peningkatannya kecil, hasil ini menunjukkan bahwa PSO masih mampu menemukan perbaikan solusi pada tahap akhir iterasi. Namun secara umum, sebagian besar partikel tetap berada di sekitar kisaran akurasi yang sama, sehingga dapat dikatakan bahwa proses optimasi mulai mendekati titik konvergen.

Tabel 4.36 Tabel Iterasi ke-10

No	Partikel	Learning Rate	Dropout CNN	Dropout Dense	Dense Units	Val Accuracy
1	P1	0.000245	0.398	0.223	160	0.7244
2	P2	0.000133	0.355	0.215	128	0.6940
3	P3	0.000472	0.384	0.147	160	0.7429
4	P4	0.00195	0.338	0.189	96	0.7095
5	P5	0.000502	0.376	0.211	160	0.7417
6	P6	0.000506	0.365	0.210	160	0.7590
7	P7	0.000671	0.399	0.220	160	0.7526
8	P8	0.000293	0.337	0.228	160	0.7328
9	P9	0.00173	0.321	0.100	128	0.7315
10	P10	0.000165	0.370	0.218	160	0.7142

Iterasi yang dilakukan menunjukkan bahwa proses optimasi hyperparameter dengan Particle Swarm Optimization (PSO) dilakukan melalui mekanisme pergerakan partikel dalam ruang pencarian solusi. Setiap partikel merepresentasikan satu kombinasi hyperparameter CNN dan dievaluasi menggunakan validation accuracy sebagai fitness. Berdasarkan nilai fitness tersebut, algoritma memperbarui personal best (pBest) dan global best (gBest), kemudian mengarahkan pergerakan partikel menggunakan rumus PSO yang tertera pada Persamaan 2.20. Dengan demikian, gBest diperoleh secara adaptif dari solusi terbaik yang ditemukan selama iterasi, bukan melalui pengujian seluruh kombinasi parameter secara brute force.

Pada iterasi kesepuluh, partikel P6 memperoleh nilai validation accuracy tertinggi sebesar 0,7590, yang sekaligus menjadi hasil terbaik pada keseluruhan proses optimasi. Nilai ini menunjukkan bahwa PSO berhasil menemukan kombinasi hyperparameter terbaik, yaitu learning rate 0,000506, dropout CNN 0,365, dropout dense 0,210, dan dense units 160. Dengan demikian, iterasi terakhir menegaskan bahwa proses optimasi telah mencapai solusi terbaik yang kemudian digunakan pada tahap retraining model CNN + PSO.

Secara keseluruhan, hasil evaluasi partikel pada iterasi 1 hingga 10 menunjukkan bahwa algoritma Particle Swarm Optimization (PSO) mampu meningkatkan kualitas solusi secara bertahap. Nilai global best validation accuracy mengalami kenaikan dari iterasi awal hingga mencapai nilai terbaik sebesar 0,7590 pada iterasi ke-10. Meskipun nilai rata-rata partikel masih mengalami fluktuasi, tren ini menunjukkan bahwa proses optimasi berjalan menuju konvergensi dan berhasil menemukan kombinasi hyperparameter yang lebih baik dibandingkan konfigurasi baseline.

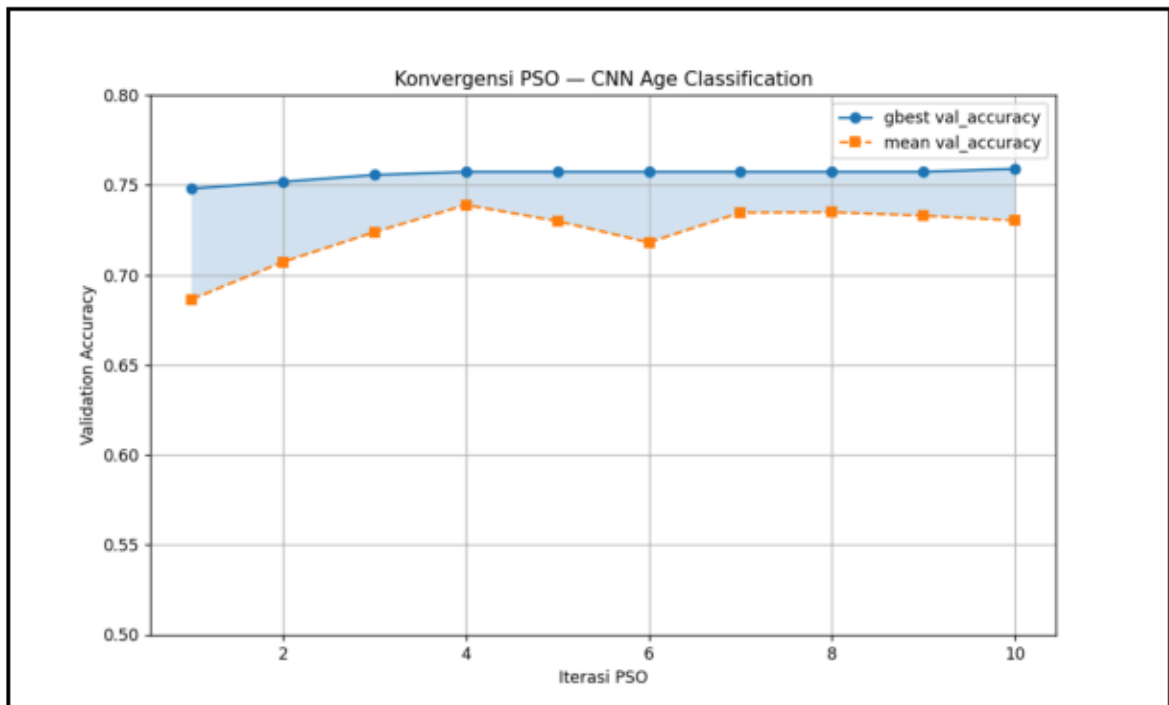
3. Hasil Hyperparameter Terbaik

Tabel 4.37 Perbandingan Nilai Hyperparameter Baseline dan Hasil Optimasi PSO

Hyperparameter	CNN Baseline	CNN + PSO (Best)
Learning Rate	0.00092	0.000506
Dropout CNN	0.5	0.365
Dropout Dense	0.2	0.210
Dense Units	128	160

Berdasarkan hasil optimasi, diperoleh kombinasi hyperparameter terbaik yaitu learning rate sebesar 0.000506, dropout CNN sebesar 0.365, dropout dense sebesar 0.210, dan dense units sebesar 160. Jika dibandingkan dengan konfigurasi baseline, PSO menghasilkan learning rate yang lebih kecil, penurunan nilai dropout pada bagian CNN, sedikit peningkatan dropout pada dense layer, serta penambahan jumlah unit pada dense layer. Perubahan ini menunjukkan bahwa konfigurasi awal pada model baseline masih dapat ditingkatkan agar proses pembelajaran model menjadi lebih stabil dan kemampuan representasinya menjadi lebih baik.

4. Analisis Konvergensi PSO



Gambar 4.5 Konvergensi PSO

Berdasarkan grafik konvergensi PSO, terlihat bahwa nilai gbest validation accuracy mengalami peningkatan pada iterasi awal, kemudian cenderung stabil hingga iterasi ke-10. Pada iterasi pertama, nilai gbest berada di kisaran 0,748, lalu meningkat secara bertahap hingga mencapai sekitar 0,756–0,759. Pola ini menunjukkan bahwa pada tahap awal proses optimasi, PSO masih aktif mencari kombinasi hyperparameter terbaik dan mampu menemukan solusi yang lebih baik dari iterasi ke iterasi.

Di sisi lain, nilai mean validation accuracy juga menunjukkan tren peningkatan yang cukup jelas dari iterasi pertama hingga sekitar iterasi keempat. Hal ini menandakan bahwa secara keseluruhan populasi partikel tidak bergerak secara acak, tetapi mulai berkumpul pada area solusi yang lebih baik. Meskipun sempat mengalami sedikit penurunan pada iterasi pertengahan, nilai rata-rata partikel kembali stabil pada iterasi-iterasi akhir. Kondisi ini menunjukkan bahwa proses eksplorasi dan eksploitasi dalam PSO berjalan seimbang, di mana partikel-partikel masih menyesuaikan posisi untuk menghindari solusi lokal yang kurang optimal.

Secara keseluruhan, grafik ini menunjukkan bahwa PSO berhasil mencapai konvergensi dalam 10 iterasi. Hal ini terlihat dari kurva gbest yang relatif datar pada iterasi akhir, yang menandakan bahwa solusi terbaik telah ditemukan dan tidak mengalami perubahan yang signifikan lagi. Dengan demikian, dapat disimpulkan bahwa penggunaan PSO dalam optimasi hyperparameter CNN mampu mengarahkan proses pencarian menuju konfigurasi yang semakin optimal, serta menghasilkan performa validasi yang stabil pada akhir iterasi.

4.2.7 Hasil Model CNN + PSO 5 Kelas

Untuk melihat proses pembelajaran model CNN hasil optimasi Particle Swarm Optimization (PSO) secara lebih rinci, hasil training model dari epoch 1 hingga epoch 50 disajikan pada Tabel 4.38. Tabel ini memuat nilai accuracy, loss, validation accuracy, dan validation loss pada setiap epoch, sehingga perkembangan performa model selama proses pelatihan dapat diamati secara lebih jelas.

Tabel 4.38 Hasil Training Model CNN + PSO per Epoch

Epoch	Accuracy	Loss	ValAccuracy	Val Loss
1	0.5241	1.4810	0.5665	1.3298
2	0.5587	1.3185	0.6019	1.1672
3	0.5826	1.2092	0.6260	1.0921
4	0.6266	1.1001	0.6834	0.9308
5	0.6481	1.0360	0.7016	0.9155
6	0.6619	1.0031	0.6429	0.9879
7	0.6707	0.9755	0.7345	0.8283
8	0.6877	0.9293	0.7125	0.8690
9	0.6946	0.9110	0.7277	0.8296
10	0.7001	0.8916	0.6796	0.9109
11	0.7053	0.8798	0.6501	0.9354
12	0.7101	0.8588	0.7425	0.7668
13	0.7211	0.8274	0.7636	0.7310
14	0.7257	0.8152	0.7201	0.8201
15	0.7271	0.8194	0.7611	0.7232
16	0.7342	0.7940	0.7298	0.8140
17	0.7331	0.7998	0.7450	0.7344
18	0.7406	0.7815	0.7387	0.7606
19	0.7437	0.7716	0.7733	0.6919
20	0.7470	0.7599	0.7835	0.6767

Tabel 4.39 Hasil Training Model CNN + PSO per Epoch

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
21	0.7509	0.7466	0.7552	0.7431
22	0.7457	0.7714	0.7830	0.6752
23	0.7541	0.7509	0.7851	0.6729
24	0.7543	0.7394	0.7712	0.6803
25	0.7564	0.7337	0.7775	0.6688
26	0.7623	0.7223	0.7873	0.6636
27	0.7607	0.7208	0.7560	0.7295
28	0.7624	0.7164	0.7961	0.6367
29	0.7624	0.7152	0.7657	0.6934
30	0.7660	0.7111	0.7826	0.6592
31	0.7659	0.7036	0.7847	0.6625
32	0.7718	0.7004	0.7847	0.6426
33	0.7687	0.6958	0.7898	0.6539
34	0.7765	0.6754	0.7986	0.6199
35	0.7803	0.6725	0.7957	0.6392
36	0.7814	0.6662	0.7940	0.6404
37	0.7812	0.6631	0.7965	0.6187
38	0.7814	0.6589	0.8050	0.6135
39	0.7883	0.6497	0.8054	0.6122
40	0.7865	0.6501	0.8050	0.6092
41	0.7858	0.6497	0.7978	0.6182
42	0.7841	0.6502	0.8067	0.6165
43	0.7854	0.6493	0.7982	0.6195
44	0.7844	0.6472	0.8050	0.6086
45	0.7909	0.6398	0.7991	0.6167
46	0.7855	0.6478	0.8003	0.6129
47	0.7923	0.6363	0.7986	0.6102
48	0.7893	0.6336	0.7986	0.6051
49	0.7900	0.6323	0.8130	0.5992
50	0.7932	0.6246	0.8088	0.5917

Berdasarkan Tabel 4.38, performa model CNN + PSO menunjukkan peningkatan yang cukup konsisten selama proses training. Nilai accuracy pada data latih cenderung meningkat dari epoch awal hingga akhir pelatihan, sedangkan loss menurun secara bertahap. Pada data validasi, validation accuracy meningkat hingga mencapai nilai terbaik sebesar 0,8130 pada epoch ke-49, sementara validation loss terendah diperoleh pada epoch ke-50 sebesar 0,5917. Hasil ini menunjukkan bahwa model hasil optimasi PSO mampu mempelajari pola data dengan baik serta memiliki performa validasi yang stabil pada akhir proses pelatihan.

Setelah proses training selesai, model CNN yang telah dioptimasi menggunakan Particle Swarm Optimization (PSO) kemudian diuji menggunakan data uji (test set) untuk

mengevaluasi performa klasifikasi kelompok usia secara menyeluruh. Ringkasan hasil evaluasi model disajikan pada Tabel 4.40.

Tabel 4.40 Metrik Evaluasi Model CNN + PSO

Metrik	Nilai
Accuracy	80.34%
Precision	73.42%
Recall	67.88%
<i>F1-Score</i>	69.51%

Nilai accuracy sebesar 80.34% menunjukkan bahwa model CNN + PSO mampu melakukan klasifikasi kelompok usia dengan baik secara keseluruhan. Meskipun demikian, perbedaan nilai precision, recall, dan *F1-Score* menunjukkan bahwa performa model masih belum merata pada seluruh kelas usia.

4. Evaluasi Per Kelas

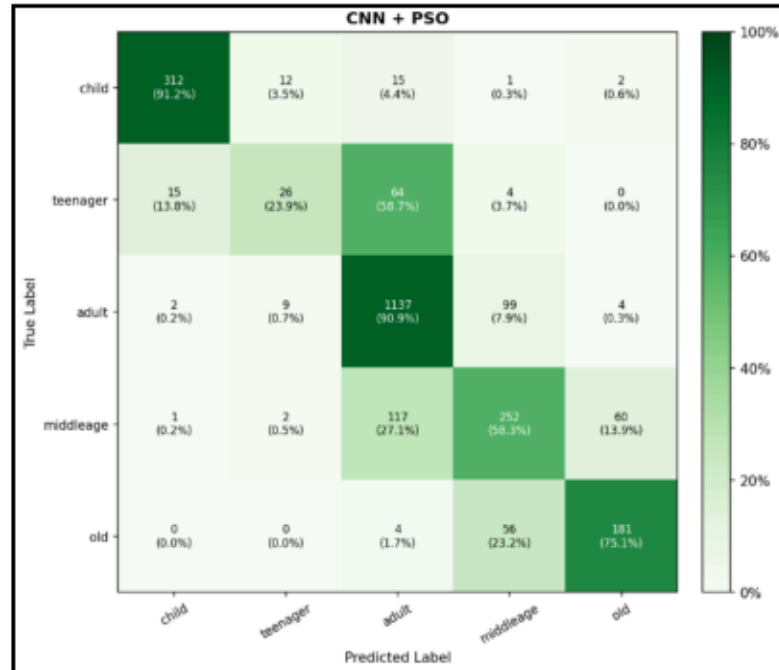
Tabel 4.41 Evaluasi Per Kelas

Kelas	Precision	Recall	<i>F1-Score</i>
child	94.5%	91.2%	92.9%
teenager	53.1%	23.9%	32.9%
adult	85.0%	90.9%	87.9%
Middle Age	61.2%	58.3%	59.7%
old	73.3%	75.1%	74.2%

Berdasarkan hasil evaluasi per kelas, terlihat bahwa performa model CNN + PSO masih dipengaruhi oleh distribusi data pada masing-masing kelas. Kelas child dan adult menunjukkan performa yang sangat baik dengan nilai precision, recall, dan *F1-Score* yang tinggi. Hal ini menunjukkan bahwa model mampu mengenali pola wajah pada kedua kelas tersebut dengan cukup konsisten.

Sebaliknya, kelas teenager masih menjadi kelas dengan performa terendah dibandingkan kelas lainnya. Nilai recall sebesar 23.9% menunjukkan bahwa sebagian besar data teenager belum berhasil dikenali dengan baik oleh model. Kondisi ini menunjukkan bahwa meskipun optimasi PSO telah meningkatkan performa model secara umum, tantangan pada kelas minoritas masih tetap ada. Selain itu, kelas Middle Age dan old menunjukkan performa yang cukup baik, namun masih terdapat kesalahan klasifikasi pada kelompok usia yang berdekatan.

5. Analisis Confusion Matrix

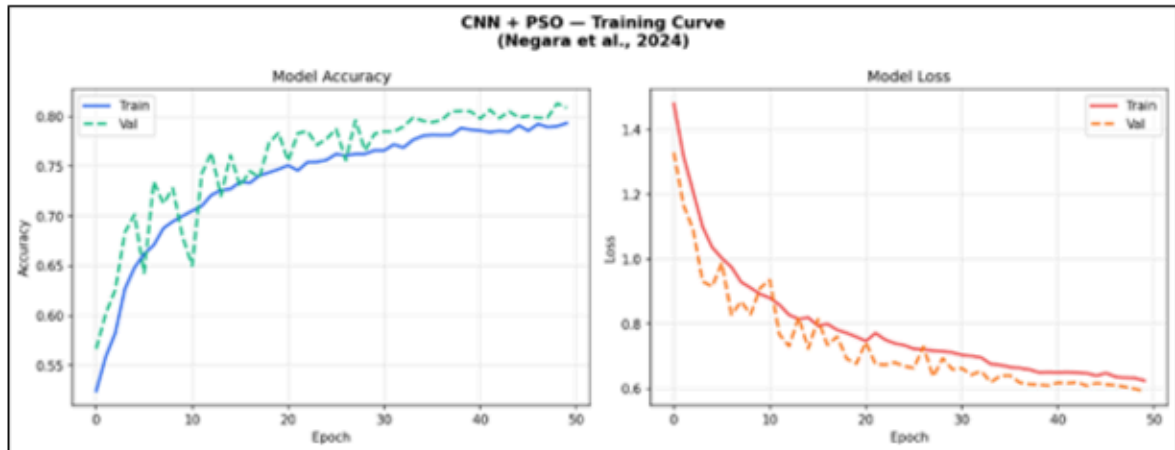


Gambar 4.6 Confusion Matrix CNN + PSO

Berdasarkan confusion matrix model CNN + PSO, terlihat bahwa prediksi yang benar paling dominan berada pada kelas child, adult, dan old. Kelas child memiliki 312 prediksi benar atau sekitar 91.2%, sedangkan kelas adult memiliki 1137 prediksi benar atau 90.9%. Pada kelas old, model mampu mengklasifikasikan 181 data dengan benar atau sekitar 75.1%. Hasil ini menunjukkan bahwa model cukup baik dalam mengenali kelas dengan ciri visual yang relatif lebih jelas.

Kesalahan klasifikasi paling dominan masih terjadi pada kelas teenager, di mana sebagian besar data diprediksi sebagai adult. Pola ini menunjukkan bahwa model masih mengalami kesulitan dalam membedakan karakteristik wajah remaja dan dewasa karena keduanya memiliki kemiripan fitur visual. Selain itu, pada kelas Middle Age juga terlihat adanya kecenderungan salah klasifikasi ke kelas adult dan old, yang menandakan adanya overlap karakteristik pada kelompok usia yang berdekatan. Pada kelas old, sebagian data juga masih diprediksi sebagai Middle Age, sehingga batas antara kedua kelompok usia tersebut masih menjadi tantangan dalam proses klasifikasi.

6. Analisis Training Curve



Gambar 4.7 Training Curve CNN + PSO

Berdasarkan grafik training curve model CNN + PSO, terlihat bahwa nilai accuracy pada data latih dan data validasi mengalami peningkatan seiring bertambahnya epoch. Peningkatan yang cukup signifikan terjadi pada epoch awal, kemudian kurva mulai melandai dan cenderung stabil pada epoch berikutnya. Hal ini menunjukkan bahwa model berhasil mempelajari pola data dengan baik dan secara bertahap mencapai kondisi konvergen.

Dari sisi loss, baik training loss maupun validation loss menunjukkan tren penurunan hingga akhir pelatihan. Meskipun pada beberapa epoch awal validation accuracy dan validation loss masih mengalami fluktuasi, secara umum kedua kurva tidak menunjukkan selisih yang terlalu jauh terhadap kurva training. Kondisi ini menunjukkan bahwa model CNN + PSO tidak memperlihatkan indikasi overfitting yang kuat dan memiliki kemampuan generalisasi yang cukup baik.

Secara keseluruhan, hasil pengujian menunjukkan bahwa model CNN yang telah dioptimasi menggunakan PSO mampu memberikan performa klasifikasi yang baik. Meski demikian, kelemahan utama model masih terletak pada kelas teenager, yang tetap menjadi kelas paling sulit dikenali akibat ketidakseimbangan distribusi data dan kemiripan fitur wajah dengan kelas adult.

4.2.8 Hasil Implementasi Antarmuka Pengujian Model

Pada tahap ini, antarmuka pengujian model yang telah dibangun menggunakan Streamlit diuji untuk mengetahui kemampuan model dalam melakukan klasifikasi kelompok usia berdasarkan citra wajah yang diunggah oleh pengguna. Pengujian dilakukan dengan membandingkan hasil prediksi dari dua model, yaitu model CNN baseline 4 kelas dan model CNN hasil optimasi Particle Swarm Optimization (PSO) 4 kelas.

1. Hasil Tampilan Antarmuka Pengujian

Berdasarkan hasil implementasi, antarmuka pengujian berhasil menampilkan seluruh alur proses secara fungsional, mulai dari unggah citra, deteksi wajah, hingga hasil klasifikasi usia. Sistem mampu mendeteksi area wajah pada citra yang diunggah

menggunakan metode Haar Cascade, kemudian menampilkan hasil potongan wajah (crop) yang digunakan sebagai input model.

Selanjutnya, sistem menampilkan hasil prediksi dalam bentuk probabilitas untuk setiap kelas usia, yaitu child, teenager, adult, dan old. Hasil prediksi dari model CNN dan CNN + PSO ditampilkan secara berdampingan, sehingga memudahkan pengguna dalam melakukan perbandingan secara langsung.

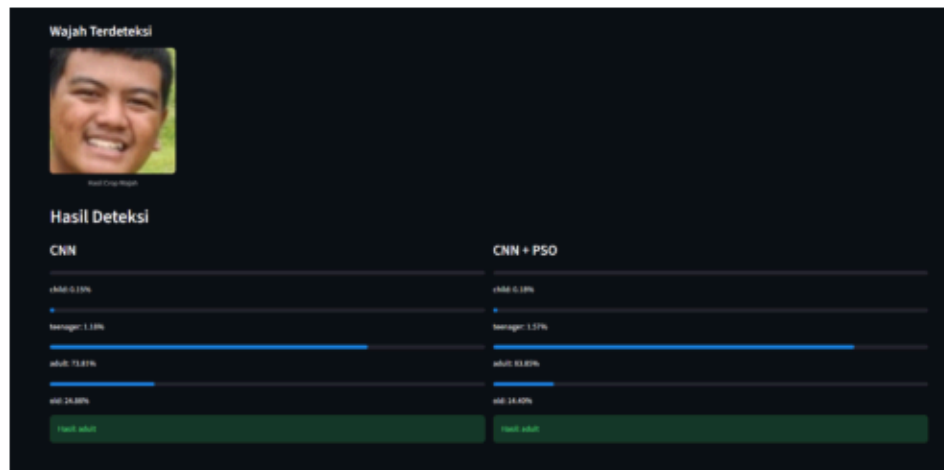
2. Hasil Uji Coba pada 5 Sampel Citra

Untuk mengevaluasi performa antarmuka pengujian, dilakukan uji coba terhadap 5 citra wajah yang berbeda. Setiap citra diuji menggunakan model CNN baseline dan model CNN + PSO, kemudian hasil prediksi dibandingkan dengan perkiraan kelompok usia sebenarnya. Berdasarkan hasil pengujian yang dilakukan, kedua model mampu memberikan hasil prediksi yang sesuai pada seluruh sampel uji coba, di mana seluruh citra berhasil diklasifikasikan dengan benar oleh masing-masing model. Hasil tersebut menunjukkan bahwa baik model CNN maupun CNN + PSO telah mampu diimplementasikan pada antarmuka pengujian dan dapat melakukan klasifikasi kelompok usia dengan baik pada sampel citra yang diuji.

Meskipun pada pengujian antarmuka ini tidak terlihat perbedaan hasil prediksi antara kedua model, model CNN + PSO tetap menunjukkan performa evaluasi yang sedikit lebih baik pada tahap pengujian dataset secara keseluruhan berdasarkan nilai akurasi, precision, recall, dan F1-Score yang telah diperoleh sebelumnya. Namun, peningkatan akurasi yang diperoleh dari model baseline ke model hasil optimasi PSO relatif kecil, yaitu sekitar 0,12, sehingga perbedaan performa antara kedua model pada implementasi antarmuka pengujian tidak terlihat secara signifikan. Hal ini menunjukkan bahwa optimasi PSO memberikan peningkatan performa yang bersifat moderat, terutama pada aspek stabilitas dan kualitas hasil evaluasi model secara keseluruhan.

Tabel 4.42 Hasil Uji Coba Antarmuka Pengujian Model

No	Sampel	Hasil CNN	Hasil CNN + PSO	Kesesuaian CNN	Kesesuaian CNN + PSO
1	Citra 1	Adult	Adult	Sesuai	Sesuai
2	Citra 2	Adult	Adult	Sesuai	Sesuai
3	Citra 3	Old	Old	Sesuai	Sesuai
4	Citra 4	Adult	Adult	Sesuai	Sesuai
5	Citra 5	Teenager	Teenager	Sesuai	Sesuai



Gambar 4.8 Uji Coba Antarmuka Pengujian Model Citra 1



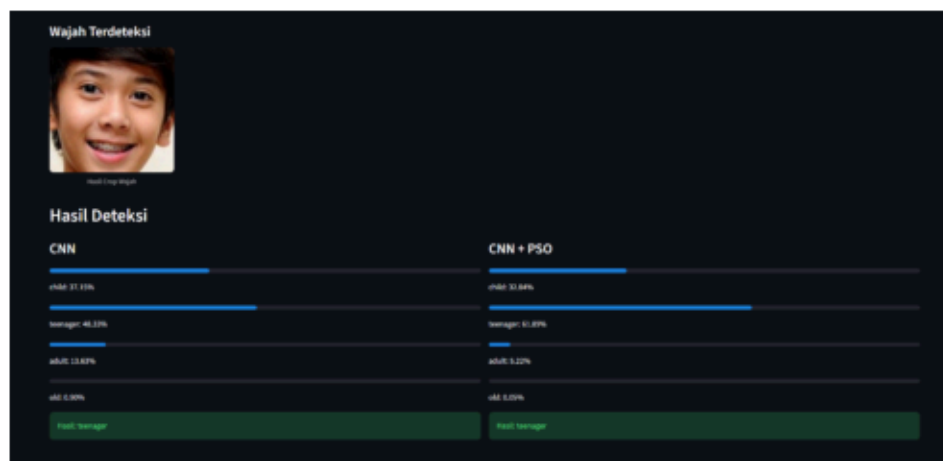
Gambar 4.9 Uji Coba Antarmuka Pengujian Model Citra 2



Gambar 4.10 Uji Coba Antarmuka Pengujian Model Citra 3



Gambar 4.11 Uji Coba Antarmuka Pengujian Model Citra 4



Gambar 4.12 Uji Coba Antarmuka Pengujian Model Citra 5

3. Analisis Hasil Pengujian Antarmuka

Berdasarkan hasil uji coba yang dilakukan, dapat diketahui bahwa model CNN hasil optimasi PSO memiliki performa yang lebih baik dibandingkan dengan model CNN baseline dalam konteks pengujian menggunakan citra statis melalui antarmuka. Keberhasilan prediksi pada seluruh sampel menunjukkan bahwa optimasi hyperparameter menggunakan PSO memberikan peningkatan terhadap kemampuan model dalam melakukan generalisasi terhadap data baru.

Sebaliknya, model CNN tanpa optimasi masih menunjukkan keterbatasan dalam membedakan karakteristik wajah pada kelompok usia tertentu, terutama pada kondisi citra dengan pencahayaan yang kurang optimal atau ekspresi wajah yang bervariasi.

Meskipun demikian, hasil pengujian ini masih bersifat uji coba terbatas dengan jumlah sampel yang relatif sedikit. Oleh karena itu, hasil ini belum dapat dijadikan sebagai representasi performa model secara keseluruhan. Diperlukan pengujian lebih lanjut dengan jumlah data yang lebih besar serta variasi kondisi yang lebih beragam, seperti perbedaan pencahayaan, pose wajah, dan kualitas citra, untuk memperoleh evaluasi yang lebih komprehensif.

4.2.9 Hasil Implementasi Realtime

Pada tahap akhir penelitian, model CNN hasil optimasi Particle Swarm Optimization (PSO) terbaik diimplementasikan ke dalam aplikasi klasifikasi usia berbasis realtime. Implementasi ini dilakukan untuk mengetahui apakah model yang telah dilatih dan diuji pada tahap sebelumnya dapat diterapkan secara langsung pada citra wajah yang diperoleh dari kamera. Aplikasi dibangun dengan memanfaatkan webcam sebagai sumber input, deteksi wajah menggunakan metode Haar Cascade, serta model CNN + PSO terbaik sebagai komponen utama untuk melakukan prediksi kelompok usia.

1. Hasil Tampilan Aplikasi Realtime

Berdasarkan hasil implementasi, aplikasi berhasil mendeteksi wajah pengguna pada frame kamera dan menampilkan hasil klasifikasi usia secara langsung. Area wajah yang terdeteksi ditandai dengan bounding box, kemudian sistem menampilkan label hasil prediksi kelompok usia pada bagian atas area wajah. Hasil tersebut menunjukkan bahwa proses pengambilan citra, deteksi wajah, preprocessing, hingga inferensi model dapat berjalan dalam satu alur aplikasi secara fungsional. Dengan demikian, model yang telah dilatih sebelumnya dapat diintegrasikan ke dalam aplikasi realtime dan digunakan untuk melakukan klasifikasi kelompok usia secara langsung.

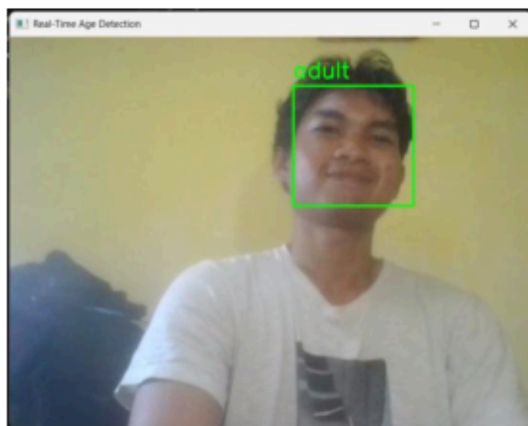
2. Hasil Uji Coba Terbatas pada Pengguna

Untuk memberikan gambaran yang lebih nyata terhadap performa sistem realtime, dilakukan uji coba terbatas terhadap 10 orang pengguna. Setiap pengguna diuji secara langsung di depan kamera, kemudian hasil prediksi sistem dibandingkan dengan kelompok usia sebenarnya. Hasil pengujian menunjukkan bahwa 9 dari 10 responden berhasil diklasifikasikan dengan sesuai oleh sistem, sedangkan 1 responden mengalami kesalahan prediksi.

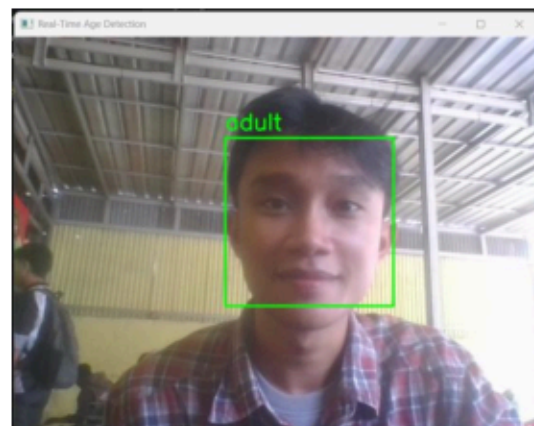
Hasil ini menunjukkan bahwa model mampu bekerja dengan cukup baik pada sebagian besar sampel uji coba terbatas yang digunakan dalam penelitian ini. Keberhasilan prediksi pada 9 responden mengindikasikan bahwa model CNN + PSO memiliki kemampuan awal yang baik untuk diterapkan pada skenario klasifikasi usia berbasis realtime. Namun demikian, adanya 1 kesalahan prediksi menunjukkan bahwa model masih memiliki keterbatasan, terutama pada kondisi tertentu seperti pencahayaan, kualitas kamera, ekspresi wajah, pose wajah, atau kemiripan karakteristik visual antar kelompok usia. Oleh karena itu, hasil ini tetap perlu dipahami dalam konteks jumlah sampel yang masih terbatas, sehingga belum dapat dijadikan dasar untuk menyimpulkan performa sistem secara umum pada populasi yang lebih luas.

Tabel 4.43 Hasil Uji Coba Aplikasi Realtime pada 10 Orang

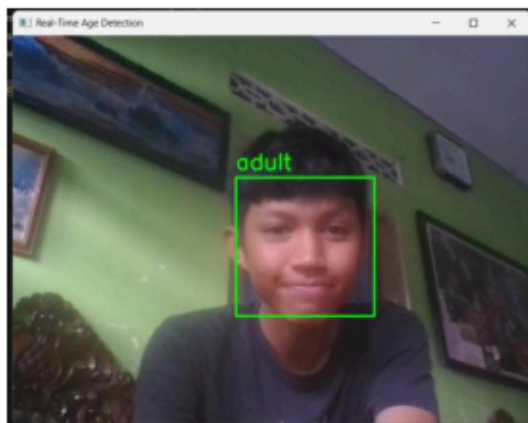
No	Inisial Responden	Kelompok Usia Sebenarnya	Hasil Prediksi Sistem	Kesesuaian
1	W	Adult	Adult	Sesuai
2	D	Adult	Adult	Sesuai
3	F	Adult	Adult	Sesuai
4	M	Adult	Adult	Sesuai
5	P	Adult	Adult	Sesuai
6	A	Adult	Middle Age	Tidak Sesuai
7	S	Adult	Adult	Sesuai
8	T	Adult	Adult	Sesuai
9	R	Adult	Adult	Sesuai
10	I	Adult	Adult	Sesuai



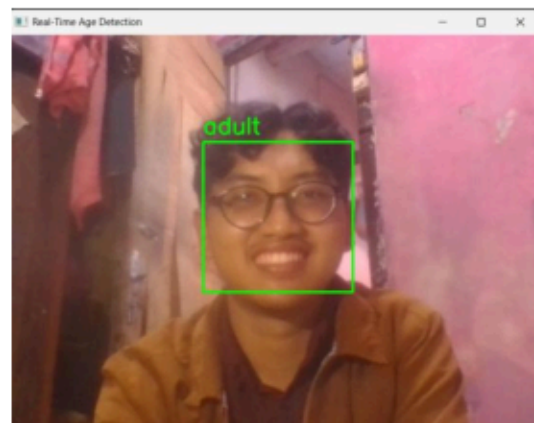
Gambar 4.13 Uji coba aplikasi inisial W



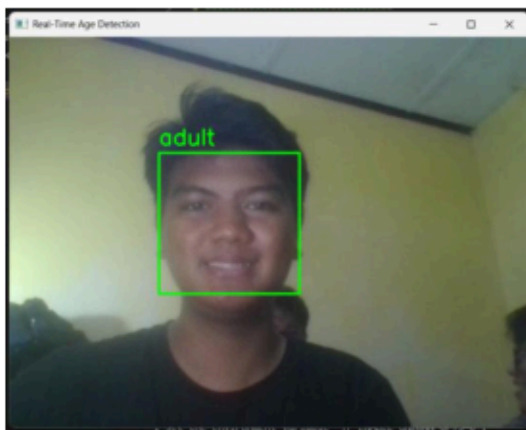
Gambar 4.14 Uji coba aplikasi Inisial D



Gambar 4.15 Uji coba aplikasi inisial F



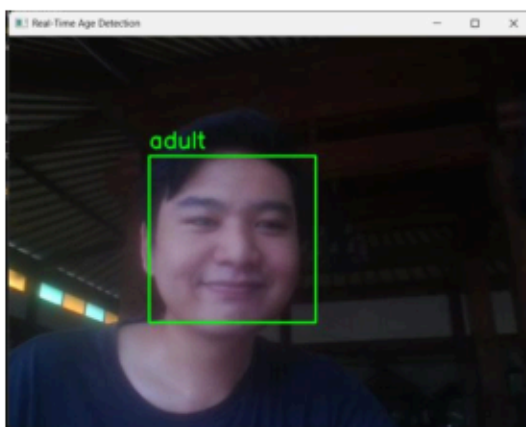
Gambar 4.16 Uji coba aplikasi inisial M



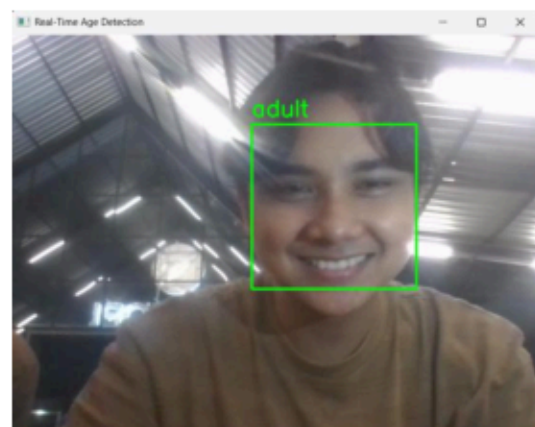
Gambar 4.17 Uji coba aplikasi inisial P



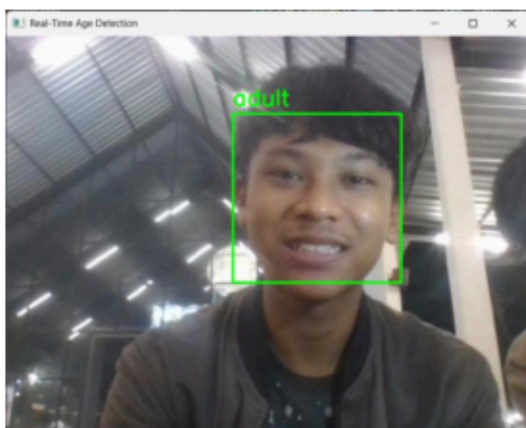
Gambar 4.18 Uji coba aplikasi inisial A



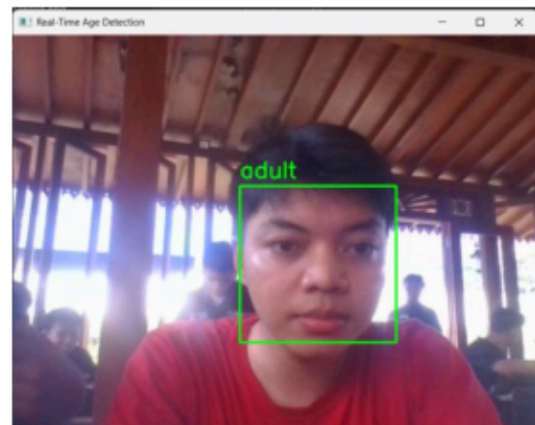
Gambar 4.19 Uji coba aplikasi inisial S



Gambar 4.20 Uji coba aplikasi inisial T



Gambar 4.21 Uji coba aplikasi inisial R



Gambar 4.22 Uji coba aplikasi inisial I

Gambar 4.13 sampai Gambar 4.22 menunjukkan hasil uji coba aplikasi klasifikasi usia berbasis realtime pada 10 responden yang berbeda. Pada setiap pengujian, sistem berhasil mendeteksi area wajah menggunakan bounding box berwarna hijau dan menampilkan hasil prediksi kelompok usia secara langsung pada bagian atas wajah. Berdasarkan hasil yang ditampilkan, sebagian besar responden berhasil diprediksi sesuai

dengan kelompok usia sebenarnya, yaitu 9 dari 10 responden, sedangkan 1 responden mengalami kesalahan prediksi, yaitu diprediksi sebagai Middle Age meskipun kelompok usia sebenarnya adalah adult. Hasil tersebut menunjukkan bahwa model CNN hasil optimasi PSO mampu diimplementasikan dengan baik pada aplikasi realtime dan dapat melakukan proses deteksi wajah serta klasifikasi usia secara langsung pada kondisi penggunaan nyata. Meskipun demikian, hasil ini masih bersifat uji coba terbatas, sehingga diperlukan pengujian lebih lanjut dengan jumlah responden dan variasi kondisi lingkungan yang lebih beragam agar performa sistem dapat dievaluasi secara lebih komprehensif.

3. Analisis Hasil Implementasi Realtime

Berdasarkan hasil uji coba terbatas, implementasi model CNN + PSO ke dalam aplikasi realtime dapat dinyatakan berhasil secara fungsional dan menunjukkan hasil yang cukup baik pada pengujian awal terhadap pengguna. Keberhasilan klasifikasi pada 9 dari 10 responden menunjukkan bahwa model tidak hanya dapat diterapkan pada data uji, tetapi juga mampu bekerja pada citra wajah yang diperoleh secara langsung dari kamera dalam kondisi nyata. Namun, adanya 1 kesalahan prediksi menunjukkan bahwa sistem masih memiliki keterbatasan dalam mengenali karakteristik usia tertentu, terutama pada kondisi citra yang dapat dipengaruhi oleh pencahayaan, pose wajah, jarak kamera, ekspresi, maupun kualitas perangkat.

Meskipun demikian, hasil tersebut belum cukup untuk menyatakan bahwa sistem telah memiliki performa realtime yang optimal secara menyeluruh, karena jumlah responden yang digunakan masih terbatas dan belum mencakup variasi kondisi yang lebih luas. Oleh karena itu, hasil implementasi realtime pada penelitian ini lebih tepat diposisikan sebagai bukti bahwa model dapat diterapkan dan menunjukkan performa awal yang baik pada pengujian langsung, namun masih memerlukan pengujian lebih lanjut dengan jumlah sampel dan kondisi yang lebih beragam.

4.3 Pembahasan

Berdasarkan hasil pengujian yang telah dilakukan, penerapan Particle Swarm Optimization (PSO) pada model Convolutional Neural Network (CNN) memberikan pengaruh terhadap performa klasifikasi kelompok usia berdasarkan citra wajah. Pengaruh tersebut terlihat dari perubahan konfigurasi hyperparameter, hasil evaluasi model pada data uji, serta implementasi model pada antarmuka pengujian dan aplikasi realtime. Pada penelitian ini, pengujian dilakukan dalam dua skenario pengelompokan usia, yaitu skenario 4 kelas berdasarkan jurnal acuan dan skenario 5 kelas sebagai skenario utama penelitian. Pada masing-masing skenario, model CNN baseline dibandingkan dengan model CNN yang telah dioptimasi menggunakan PSO.

Pada tahap preprocessing, dataset UTKFace yang digunakan terdiri dari 23.708 citra wajah yang dibagi menjadi data train, validation, dan test dengan rasio 80:10:10. Distribusi data menunjukkan adanya ketidakseimbangan jumlah sampel antar kelas. Kelas adult memiliki jumlah data paling besar, yaitu 12.505 citra, sedangkan kelas teenager memiliki jumlah data paling sedikit, yaitu 1.082 citra. Ketidakseimbangan ini berpengaruh terhadap performa model karena model cenderung lebih mudah mengenali kelas mayoritas seperti adult, tetapi lebih sulit mengenali kelas minoritas seperti teenager. Oleh karena itu, evaluasi tidak hanya dilihat dari accuracy, tetapi juga precision, recall, F1-Score, dan confusion matrix.

Pada skenario 4 kelas, model CNN baseline jurnal yang bisa dilihat pada Tabel 4.5 memperoleh accuracy sebesar 83,87%, precision sebesar 78,49%, recall sebesar 72,06%, dan F1-Score sebesar 74,14%. Setelah dilakukan optimasi menggunakan PSO, model CNN + PSO jurnal 4 kelas memperoleh accuracy sebesar 83,99%, precision sebesar 77,85%, recall sebesar 72,94%, dan F1-Score sebesar 74,88% atau dapat dilihat pada Tabel 4.20. Hasil ini menunjukkan bahwa PSO memberikan peningkatan pada accuracy sebesar 0,12%, recall sebesar 0,88%, dan F1-Score sebesar 0,74%. Namun, precision mengalami sedikit penurunan sebesar 0,64%. Dengan demikian, pada skenario 4 kelas, optimasi PSO tidak meningkatkan seluruh metrik secara merata, tetapi lebih terlihat pada peningkatan recall dan F1-Score.

Peningkatan pada skenario 4 kelas dapat dikatakan relatif kecil. Hal ini mengindikasikan bahwa model CNN baseline pada skenario 4 kelas sudah memiliki performa yang cukup tinggi, sehingga ruang peningkatan melalui optimasi hyperparameter menjadi terbatas. Selain itu, skenario 4 kelas memiliki tingkat kesulitan yang lebih rendah karena data usia 41 tahun ke atas masih digabung ke dalam satu kelas old. Penggabungan ini membuat model tidak perlu membedakan karakteristik wajah usia middle age dan old secara terpisah, sehingga proses klasifikasi menjadi lebih sederhana dibandingkan skenario 5 kelas.

Meskipun secara keseluruhan skenario 4 kelas menghasilkan accuracy yang tinggi, hasil evaluasi per kelas menunjukkan bahwa model masih memiliki kelemahan pada kelas teenager. Pada model CNN baseline jurnal 4 kelas, kelas teenager memperoleh precision sebesar 54,5%, recall sebesar 27,5%, dan F1-Score sebesar 36,6%. Setelah optimasi PSO, recall kelas teenager meningkat menjadi 32,1% dan F1-Score meningkat menjadi 39,1%, meskipun precision menurun menjadi 50,0%. Hal ini menunjukkan bahwa PSO membantu model mengenali lebih banyak data teenager, tetapi masih belum sepenuhnya mampu

mengurangi kesalahan prediksi pada kelas tersebut. Banyak data teenager masih diprediksi sebagai adult karena kemiripan karakteristik visual antara remaja dan dewasa, serta jumlah data teenager yang jauh lebih sedikit dibandingkan kelas adult.

Berbeda dengan skenario 4 kelas, skenario 5 kelas memiliki tingkat kesulitan yang lebih tinggi karena kelas old dipisahkan menjadi dua kategori, yaitu middle age untuk usia 41 sampai 60 tahun dan old untuk usia di atas 60 tahun. Pemisahan ini membuat model harus membedakan karakteristik wajah pada kelompok usia yang berdekatan. Akibatnya, performa model pada skenario 5 kelas lebih rendah dibandingkan skenario 4 kelas. Namun, skenario 5 kelas lebih representatif karena kategori usia yang digunakan lebih spesifik dan dapat menggambarkan fase usia manusia secara lebih rinci.

Pada skenario 5 kelas, model CNN baseline memperoleh accuracy sebesar 79,20%, precision sebesar 72,85%, recall sebesar 65,31%, dan F1-Score sebesar 67,22% yang dapat dilihat pada Tabel 4.24. Jika dilihat dari evaluasi per kelas, performa model belum merata. Kelas child memperoleh F1-Score sebesar 92,6% dan kelas adult memperoleh F1-Score sebesar 86,8%, sedangkan kelas teenager hanya memperoleh F1-Score sebesar 27,4% dengan recall sebesar 18,3%. Nilai recall yang rendah menunjukkan bahwa sebagian besar data teenager belum berhasil dikenali dengan benar oleh model baseline. Kondisi ini memperlihatkan bahwa accuracy secara keseluruhan belum cukup untuk menggambarkan performa model secara lengkap, karena model masih memiliki kelemahan pada kelas tertentu.

Proses optimasi PSO pada skenario 5 kelas dilakukan dengan 10 partikel dan 10 iterasi, sehingga terdapat 100 kandidat kombinasi hyperparameter yang dievaluasi. Hyperparameter yang dioptimasi meliputi learning rate, dropout CNN, dropout dense, dan dense units. Berdasarkan hasil optimasi, PSO menemukan konfigurasi terbaik pada iterasi ke-10 dengan nilai validation accuracy sebesar 0,7590. Konfigurasi terbaik tersebut adalah learning rate sebesar 0,000506, dropout CNN sebesar 0,365, dropout dense sebesar 0,210, dan dense units sebesar 160. Jika dibandingkan dengan konfigurasi baseline, learning rate mengalami penurunan dari 0,00092 menjadi 0,000506, dropout CNN turun dari 0,5 menjadi 0,365, dropout dense sedikit meningkat dari 0,2 menjadi 0,210, dan dense units meningkat dari 128 menjadi 160.

Perubahan hyperparameter tersebut menunjukkan bahwa konfigurasi baseline belum sepenuhnya optimal. Penurunan learning rate menunjukkan bahwa model hasil optimasi menggunakan langkah pembelajaran yang lebih kecil, sehingga proses pembaruan bobot dapat berjalan lebih stabil dan tidak terlalu agresif. Penurunan dropout CNN menunjukkan bahwa model membutuhkan lebih banyak fitur hasil ekstraksi CNN untuk dipertahankan selama pelatihan, sehingga informasi visual dari citra wajah tidak terlalu banyak dinonaktifkan. Sementara itu, peningkatan dense units menunjukkan bahwa bagian klasifikasi membutuhkan kapasitas representasi yang lebih besar untuk membedakan lima kelompok usia. Dengan demikian, PSO tidak mengubah struktur utama CNN, tetapi menyempurnakan konfigurasi hyperparameter agar model dapat belajar lebih efektif.

Setelah dilakukan retraining menggunakan hyperparameter hasil optimasi PSO, model CNN + PSO pada skenario 5 kelas menghasilkan performa yang lebih baik dibandingkan model baseline. Model CNN + PSO memperoleh accuracy sebesar 80,34%,

precision sebesar 73,42%, recall sebesar 67,88%, dan F1-Score sebesar 69,51% atau bisa dilihat pada Tabel 4.40. Jika dibandingkan dengan model CNN baseline, accuracy meningkat sebesar 1,14%, precision meningkat sebesar 0,57%, recall meningkat sebesar 2,57%, dan F1-Score meningkat sebesar 2,29%. Peningkatan terbesar terdapat pada recall dan F1-Score, yang menunjukkan bahwa optimasi PSO tidak hanya meningkatkan jumlah prediksi benar secara keseluruhan, tetapi juga membantu model menjadi lebih baik dalam mengenali data pada beberapa kelas.

Peningkatan performa juga terlihat pada beberapa kelas. Pada kelas teenager, recall meningkat dari 18,3% pada model baseline menjadi 23,9% pada model CNN + PSO, sedangkan F1-Score meningkat dari 27,4% menjadi 32,9%. Pada kelas middle age, F1-Score meningkat dari 56,4% menjadi 59,7%, dan pada kelas old meningkat dari 72,9% menjadi 74,2%. Kelas adult juga mengalami peningkatan F1-Score dari 86,8% menjadi 87,9%. Meskipun peningkatan ini menunjukkan adanya perbaikan, kelas teenager tetap menjadi kelas dengan performa terendah. Hal ini menunjukkan bahwa PSO mampu meningkatkan performa model, tetapi belum sepenuhnya menyelesaikan masalah ketidakseimbangan data dan kemiripan fitur visual antara kelas teenager dan adult.

Berdasarkan confusion matrix model CNN + PSO pada skenario 5 kelas, prediksi benar paling dominan terdapat pada kelas child, adult, dan old. Kelas child berhasil diklasifikasikan dengan benar sebanyak 312 data atau sekitar 91,2%, kelas adult sebanyak 1137 data atau sekitar 90,9%, dan kelas old sebanyak 181 data atau sekitar 75,1%. Sebaliknya, kesalahan klasifikasi masih banyak terjadi pada kelas teenager yang sering diprediksi sebagai adult. Hal ini terjadi karena kedua kelompok usia tersebut memiliki karakteristik wajah yang relatif berdekatan, terutama pada fase transisi remaja menuju dewasa. Kesalahan juga masih terjadi pada kelas middle age dan old, yang menunjukkan bahwa batas visual antara kelompok usia yang berdekatan belum sepenuhnya dapat dipisahkan oleh model.

Dari sisi proses pelatihan, model CNN + PSO pada skenario 5 kelas menunjukkan hasil yang lebih stabil dibandingkan baseline. Validation accuracy terbaik model CNN baseline mencapai 0,8062 pada epoch ke-46, sedangkan model CNN + PSO mencapai validation accuracy terbaik sebesar 0,8130 pada epoch ke-49. Selain itu, validation loss model CNN + PSO mencapai nilai terendah sebesar 0,5917 pada epoch ke-50. Hasil ini menunjukkan bahwa model hasil optimasi PSO mampu memperoleh performa validasi yang sedikit lebih baik dibandingkan baseline. Kurva training juga menunjukkan bahwa tidak terdapat selisih yang terlalu besar antara training dan validation, sehingga model tidak memperlihatkan indikasi overfitting yang kuat.

Implementasi model ke dalam antarmuka pengujian berbasis Streamlit menunjukkan bahwa kedua model dapat digunakan untuk melakukan prediksi pada citra wajah statis yang diunggah pengguna. Pada pengujian terhadap 5 sampel citra, model CNN baseline maupun model CNN + PSO sama-sama berhasil mengklasifikasikan seluruh sampel dengan benar sesuai dengan kelompok usia sebenarnya. Hasil tersebut menunjukkan bahwa kedua model telah mampu diimplementasikan dengan baik pada antarmuka aplikasi dan dapat digunakan untuk melakukan klasifikasi kelompok usia pada citra wajah statis. Meskipun demikian, model CNN + PSO tetap menunjukkan performa evaluasi yang sedikit lebih baik pada tahap

pengujian dataset secara keseluruhan berdasarkan nilai akurasi, precision, recall, dan F1-Score yang diperoleh sebelumnya. Namun, peningkatan akurasi dari model baseline ke model hasil optimasi PSO relatif kecil, yaitu sekitar 0,12, sehingga perbedaan hasil prediksi antara kedua model pada implementasi antarmuka tidak terlihat secara signifikan. Selain itu, jumlah sampel pengujian yang masih terbatas menyebabkan hasil implementasi ini belum dapat dijadikan dasar generalisasi yang kuat terhadap seluruh kondisi citra wajah.

Selain antarmuka pengujian model, model CNN + PSO juga berhasil diimplementasikan ke dalam aplikasi realtime berbasis webcam. Pada pengujian terbatas terhadap 10 responden, sistem berhasil memprediksi 9 responden sesuai dengan kelompok usia sebenarnya, sedangkan 1 responden mengalami kesalahan prediksi. Hasil ini menunjukkan bahwa model dapat diterapkan dalam bentuk prototipe aplikasi realtime yang mampu melakukan deteksi wajah, preprocessing, dan klasifikasi usia secara langsung. Akan tetapi, kesalahan prediksi pada pengujian realtime menunjukkan bahwa performa sistem masih dipengaruhi oleh faktor eksternal, seperti pencahayaan, kualitas kamera, posisi wajah, ekspresi, jarak wajah terhadap kamera, serta kemiripan karakteristik antar kelompok usia.

Secara keseluruhan, hasil penelitian menunjukkan bahwa PSO berhasil diterapkan sebagai metode optimasi hyperparameter CNN dan mampu meningkatkan performa model dalam klasifikasi kelompok usia berdasarkan citra wajah. Pada skenario 4 kelas, peningkatan performa yang dihasilkan relatif kecil, yaitu accuracy meningkat dari 83,87% menjadi 83,99%. Pada skenario 5 kelas, peningkatan terlihat lebih jelas, yaitu accuracy meningkat dari 79,20% menjadi 80,34%, precision meningkat dari 72,85% menjadi 73,42%, recall meningkat dari 65,31% menjadi 67,88%, dan F1-Score meningkat dari 67,22% menjadi 69,51%. Dengan demikian, PSO dalam penelitian ini lebih tepat dipahami sebagai metode fine-tuning untuk menyempurnakan konfigurasi CNN, bukan sebagai solusi tunggal untuk seluruh permasalahan klasifikasi usia.

Permasalahan utama yang masih tersisa adalah ketidakseimbangan data, terutama pada kelas teenager, serta kemiripan karakteristik wajah pada kelompok usia yang berdekatan. Selain itu, penambahan kelas middle age pada skenario 5 kelas membuat klasifikasi menjadi lebih detail, tetapi juga meningkatkan tingkat kesulitan model dalam membedakan kelompok usia yang berdekatan. Oleh karena itu, penelitian selanjutnya perlu mempertimbangkan strategi tambahan seperti class weighting, oversampling, undersampling terkontrol, augmentasi khusus kelas minoritas, atau penggunaan arsitektur CNN yang lebih kuat agar performa antar kelas menjadi lebih seimbang.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, kesimpulan yang diperoleh dapat dirumuskan sesuai dengan tujuan penelitian sebagai berikut:

1. Penerapan Particle Swarm Optimization (PSO) pada optimasi hyperparameter Convolutional Neural Network (CNN) untuk klasifikasi kelompok usia berdasarkan citra wajah telah berhasil dilakukan. PSO digunakan untuk mengoptimasi hyperparameter berupa learning rate, dropout CNN, dropout dense, dan dense units melalui proses pencarian solusi terbaik secara iteratif berdasarkan nilai validation accuracy sebagai fungsi fitness. Proses optimasi dilakukan pada dua skenario pengelompokan usia, yaitu skenario 4 kelas berdasarkan jurnal acuan dan skenario 5 kelas sebagai skenario utama penelitian. Hasil optimasi menunjukkan bahwa PSO mampu menemukan kombinasi hyperparameter terbaik yang kemudian digunakan untuk melatih ulang model CNN.
2. Penggunaan PSO mampu meningkatkan kinerja model CNN pada skenario 4 kelas maupun skenario 5 kelas, meskipun peningkatan yang diperoleh bersifat moderat. Pada skenario 4 kelas, model CNN baseline memperoleh accuracy sebesar 83,87%, precision sebesar 78,49%, recall sebesar 72,06%, dan F1-Score sebesar 74,14%. Setelah dilakukan optimasi menggunakan PSO, model CNN + PSO memperoleh accuracy sebesar 83,99%, precision sebesar 77,85%, recall sebesar 72,94%, dan F1-Score sebesar 74,88%. Hasil tersebut menunjukkan bahwa pada skenario 4 kelas terjadi peningkatan pada accuracy, recall, dan F1-Score, meskipun precision mengalami sedikit penurunan. Pada skenario 5 kelas, model CNN baseline memperoleh accuracy sebesar 79,20%, precision sebesar 72,85%, recall sebesar 65,31%, dan F1-Score sebesar 67,22%, sedangkan model CNN + PSO memperoleh accuracy sebesar 80,34%, precision sebesar 73,42%, recall sebesar 67,88%, dan F1-Score sebesar 69,51%. Hasil ini menunjukkan bahwa PSO berperan sebagai metode penyempurnaan konfigurasi hyperparameter sehingga model menjadi lebih baik dibandingkan baseline, tetapi belum menghasilkan peningkatan performa yang sangat besar. Dengan demikian, PSO pada penelitian ini lebih tepat dipahami sebagai metode fine-tuning terhadap model CNN baseline.
3. Model CNN hasil optimasi PSO berhasil diimplementasikan ke dalam aplikasi klasifikasi usia berbasis realtime dan dapat berjalan dengan baik. Sistem mampu melakukan deteksi wajah, preprocessing, dan klasifikasi usia secara langsung menggunakan kamera sebagai input. Pada uji coba terbatas terhadap 10 orang, sebanyak 9 responden berhasil diprediksi sesuai dengan kelompok usia sebenarnya, sedangkan 1 responden mengalami kesalahan prediksi. Hal ini menunjukkan bahwa model tidak hanya bekerja pada tahap eksperimen menggunakan dataset, tetapi juga dapat diterapkan dalam bentuk prototipe

aplikasi realtime. Namun, implementasi ini masih terbatas pada pengujian skala kecil dan belum dianalisis lebih lanjut dari sisi latency, frame rate, serta kestabilan prediksi pada berbagai kondisi.

Secara keseluruhan, penelitian ini menunjukkan bahwa PSO dapat digunakan sebagai metode optimasi hyperparameter untuk meningkatkan performa CNN dalam klasifikasi kelompok usia berdasarkan citra wajah. Akan tetapi, peningkatan yang diperoleh masih bersifat moderat dan belum sepenuhnya mengatasi permasalahan utama pada dataset, khususnya ketidakseimbangan kelas dan rendahnya performa pada kelas minoritas seperti teenager. Selain itu, penambahan kelas Middle Age pada skenario 5 kelas membuat klasifikasi menjadi lebih spesifik, tetapi juga meningkatkan tingkat kesulitan model dalam membedakan kelompok usia yang memiliki karakteristik visual berdekatan. Oleh karena itu, diperlukan pendekatan tambahan seperti class weighting, oversampling, undersampling terkontrol, augmentasi khusus kelas minoritas, atau penggunaan arsitektur CNN yang lebih kuat agar performa antar kelas menjadi lebih seimbang.

5.2 Saran

Berdasarkan hasil penelitian dan keterbatasan yang telah diidentifikasi, terdapat beberapa saran yang dapat diberikan untuk pengembangan penelitian selanjutnya:

1. Penelitian selanjutnya disarankan menggunakan strategi penanganan data tidak seimbang yang lebih spesifik, karena pada penelitian ini distribusi data antar kelas usia masih tidak seimbang, terutama pada kelas teenager yang memiliki jumlah data jauh lebih sedikit dibandingkan kelas adult. Kondisi ini memengaruhi kemampuan model dalam mengenali kelas minoritas secara optimal. Oleh karena itu, penelitian berikutnya dapat mempertimbangkan penerapan teknik seperti oversampling, undersampling yang lebih terkontrol, atau metode augmentasi yang lebih terarah pada kelas minoritas.
2. Penelitian selanjutnya disarankan membandingkan PSO dengan metode optimasi hyperparameter lainnya, seperti Grid Search, Random Search, atau Genetic Algorithm, sehingga dapat diketahui secara lebih komprehensif efektivitas PSO dalam meningkatkan performa model CNN pada klasifikasi kelompok usia berbasis citra wajah. Pada penelitian ini, PSO terbukti meningkatkan performa model, tetapi peningkatannya masih bersifat moderat, sehingga perbandingan dengan metode optimasi lain akan memberikan gambaran yang lebih kuat.
3. Penelitian selanjutnya disarankan mengeksplorasi arsitektur model yang lebih beragam, baik dengan memodifikasi arsitektur CNN yang digunakan maupun dengan mencoba model deep learning lain yang lebih modern, seperti transfer learning menggunakan VGG, ResNet, MobileNet, atau EfficientNet. Hal ini penting karena pada penelitian ini PSO hanya difokuskan pada optimasi hyperparameter, bukan pada pencarian arsitektur model terbaik.

DAFTAR PUSTAKA

- Almufti, S. M., Yahya Zebari, A., & Khalid Omer, H. (2019). A comparative study of particle swarm optimization and genetic algorithm. *Journal of Advanced Computer Science & Technology*, 8(2), 40–45.
- Antonio, R., Tyandra, N., Indryani, T. A., Edbert, I. S., & Aulia, A. (2024). Human identification system based on ear shape using convolutional neural network. *Procedia Computer Science*, 245, 657–666. <https://doi.org/10.1016/J.PROCS.2024.10.292>
- Aure, F., Rokhana, R., Sigit, R., Sena Bayu Dewantara, B., Teknik Elektro, D., & Elektronika Negeri Surabaya, P. (2023). Fusi Algoritma K-Means dan CNN untuk Klasifikasi Emosi pada Anak. *Techno.Com*, 22(3), 622–634. <https://doi.org/10.33633/TC.V22I3.8667>
- Azhar, A., Siswoyo, B., Pratama, D., Anam, K., & Susana, H. (2024). Penerapan Algoritma Convolutional Neural Network (CNN) Untuk Diagnosa Tumor Otak. *JATI (Jurnal Mahasiswa Teknik Informatika)*, 8(2), 1797–1801. <https://doi.org/10.36040/JATI.V8I2.8242>
- Azizul Hakim, E., Al Ghufuran, T., Effendy, M., & Setyawan, N. (2020). MPPT Menggunakan Algoritme Particle Swarm Optimization dan Artificial Bee Colony (MPPT Using Particle Swarm Optimization and Artificial Bee Colony Algorithm). *Jurnal Nasional Teknik Elektro dan Teknologi Informasi* |, 9(2).
- Elbeltagi, A., Srivastava, A., Ehsan, M., Sharma, G., Yu, J., Khadke, L., Gautam, V. K., Awad, A., & Jinsong, D. (2024). Advanced stacked integration method for forecasting long-term drought severity: CNN with machine learning models. *Journal of Hydrology: Regional Studies*, 53, 101759. <https://doi.org/10.1016/J.EJRH.2024.101759>
- Febri Hafifah, Sayuti Rahman, & Munjiat Setiani Asih. (2021, Oktober 5). *Klasifikasi Jenis Kendaraan Pada Jalan Raya Menggunakan Metode Convolutional Neural Networks (CNN)*. <https://repository.harapan.ac.id/files/fulltext/884e4513f76d53e81ed5f6566020b92f915-Article%20Text-3054-1-10-20211028.pdf>
- Fouad, Z., Alfonse, M., Roushdy, M., & Salem, A. B. M. (2021). Hyper-parameter optimization of convolutional neural network based on particle swarm optimization algorithm. *Bulletin of Electrical Engineering and Informatics*, 10(6), 3377–3384. <https://doi.org/10.11591/eei.v10i6.3257>
- Hendriyana Hendriyana, & Hilman Maulana, Y. (2020). Identification of Types of Wood using Convolutional Neural Network with Mobilenet Architecture. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, 4(1), 70–76. <https://doi.org/10.29207/RESTI.V4I1.1445>
- Ibrahim, N., Ibrahim, N., Saetmidah, S., Hidayat, B., & Darana, S. (2022). Klasifikasi Grade Telur Ayam Negeri secara non- Invasive menggunakan Convolutional Neural Network. *ELKOMIKA: Jurnal Teknik Energi Elektrik, Teknik Telekomunikasi, & Teknik Elektronika*, 10(2), 297. <https://doi.org/10.26760/elkomika.v10i2.297>

- Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *32nd International Conference on Machine Learning, ICML 2015*, 1, 448–456. <https://arxiv.org/pdf/1502.03167>
- Kurnia, D. , & Wibowo, A. T. (2021, Agustus 1). *Klasifikasi Spesies Tanaman Kaktus Grafting Berdasarkan...* - *Google Scholar*. https://scholar.google.com/scholar?hl=en&as_sdt=0%2C5&q=+Klasifikasi+Spesies+Tanaman+Kaktus+Grafting++Berdasarkan+Citra+Scion+Menggunakan+Metode+Convolutional+Neural+Network+%28Cnn&btnG=
- Li, D., You, S., Liao, Q., Sheng, M., & Tian, S. (2023). Prediction of Shale Gas Production by Hydraulic Fracturing in Changning Area Using Machine Learning Algorithms. *Transport in Porous Media*, 149(1), 373–388. <https://doi.org/10.1007/s11242-023-01935-3>
- Lusiana Anggraini, & Yuni Yamasari. (2023). *Klasifikasi Citra Wajah Untuk Rentang Usia Menggunakan Metode Artificial Neural Network | Journal of Informatics and Computer Science (JINACS)*. <https://ejournal.unesa.ac.id/index.php/jinacs/article/view/56907>
- M Afif Rizky A. (2023, Maret 21). *Confusion Matrix? Matrix Bingung!?* <https://www.linkedin.com/pulse/confusion-matrix-bingung-m-afif-rizky-a/>
- Mustofa, A., Mustofa, A., Okfalisa, O., Cynthia, E. P., Yelfi, Y., & Gusti, S. K. (2022). Klasifikasi Penerima Bantuan Covid-19 Menggunakan Metode Weighted K-Nearest Neighbour. *Jurnal Nasional Komputasi dan Teknologi Informasi (JNKTI)*, 5(3), 415–422. <https://doi.org/10.32672/jnkti.v5i3.4399>
- Negara, M. S., Irzan, M., Dia'ul Haqqi, A., & Bimantoro, F. (2024). Implementasi Convolutional Neural Network pada Multi-label Classification Wajah Manusia Berdasarkan Usia, Gender, dan Ras. *DIELEKTRIKA*, 11(2), 135. <https://doi.org/10.29303/DIELEKTRIKA.V11I2.389>
- Nima, R., & Shila, F. (2020). Crack classification in rotor-bearing system by means of wavelet transform and deep learning methods: an experimental investigation. *Journal of Mechanical Engineering, Automation and Control Systems*, 1(2), 102–113. <https://doi.org/10.21595/JMEACS.2020.21799>
- Norhikmah, Afdhal Lutfhi, & Rumini. (2022). The effect of layer batch normalization and droupout of CNN model performance on facial expression classification. *joiv.orgA LutfhiJOIV: International Journal on Informatics Visualization*, 2022•joiv.org. <https://joiv.org/index.php/joiv/article/view/921>
- Paraijun, F., Nur Aziza, R., & Kuswardani, D. (2022). Implementasi Algoritma Convolutional Neural Network Dalam Mengklasifikasi Kesegaran Buah Berdasarkan Citra Buah. *KILAT*, 11(1), 1–9. <https://doi.org/10.33322/KILAT.V10I2.1458>
- Peryanto, A., Yudhana, A., & Umar, D. R. (2020). Rancang Bangun Klasifikasi Citra Dengan Teknologi Deep Learning Berbasis Metode Convolutional Neural Network. *Format : Jurnal Ilmiah Teknik Informatika*, 8(2), 138–147. <https://doi.org/10.22441/FORMAT.2019.V8.I2.007>

- Putra, A. T., Usman, K., & Saidah, S. (2021). Webinar Student Presence System Based On Regional Convolutional Neural Network Using Face Recognition. *Jurnal Teknik Informatika (Jutif)*, 2(2), 109–118. <https://doi.org/10.20884/1.JUTIF.2021.2.2.82>
- Putu Dita Ariani Sukma Dewi, N., Windu Antara Kesiman, M., Made Gede Sunarya, I., Gusti Ayu Agung Diatri Indradewi, I., & Gede Andika, I. (2024). Klasifikasi Jenis Daun Tumbuhan Herbal Berdasarkan Lontar Usada Taru Pramana Menggunakan CNN. *Techno.Com*, 23(1), 271–283. <https://doi.org/10.62411/TC.V23I1.9510>
- Rafly Alwanda, M., Putra, R., Ramadhan, K., & Alamsyah, D. (2020). Implementasi Metode Convolutional Neural Network Menggunakan Arsitektur LeNet-5 untuk Pengenalan Doodle. *Algoritme Jurnal Mahasiswa Teknik Informatika*, 1(1), 45–56. <https://doi.org/10.35957/ALGORITME.V1I1.434>
- Ratna Juwita, A., Al Mudzakir, T., Rizky Pratama, A., Husodo, P., Sulaiman Vol, R., Al Mudzakir, T., & Sulaiman, R. (2021). Identifikasi Citra Batik Dengan Metode Convolutional Neural Network. *BUANA ILMU*, 6(1), 192–208. <https://doi.org/10.36805/BI.V6I1.1996>
- Retno, K. R., #1, W., Suryalim, H., Lewi, V. J., #3, E., & Christian, H. (2023). Analisis Pemilihan Optimizer dalam Arsitektur Convolution Neural Network VGG16 dan Inception untuk Sistem Pengenalan Wajah. *JEPIN (Jurnal Edukasi dan Penelitian Informatika)*, 9(2), 186–194. <https://doi.org/10.26418/JP.V9I2.60432>
- Rizki, M., Basuki, S., & Azhar, Y. (2020). Implementasi Deep Learning Menggunakan Arsitektur Long Short Term Memory(LSTM) Untuk Prediksi Curah Hujan Kota Malang. *Jurnal Repositor*, 2(3), 331–338. <https://doi.org/10.22219/REPOSITOR.V2I3.30499>
- Robet, R., Chandra, C., & Setiawan, J. (2025). Aplikasi Deteksi Usia Berbasis Citra Menggunakan Model Deep Learning dengan Arsitektur CNN. *TAMIKA: Jurnal Tugas Akhir Manajemen Informatika & Komputerisasi Akuntansi*, 5(1), 97–102. <https://doi.org/10.46880/TAMIKA.VOL5NO1.PP97-102>
- Sari, D. N., Rahman, Muh. A., & Wihandika, R. C. (2022). Uji Parameter dan Arsitektur Convolutional Neural Network untuk Mendeteksi Citra Wajah Bermasker. *Jurnal Teknologi Informasi dan Ilmu Komputer*, 9(7), 1707–1714. <https://doi.org/10.25126/JTIK.2022976776>
- Sriani, S., & Nabila, A. (2024). Implementasi Deep Learning Untuk Mengidentifikasi Umur Manusia Menggunakan Convolutional Neural Network (CNN). *Jurnal Informatika dan Teknik Elektro Terapan*, 12(3), 2830–7062. <https://doi.org/10.23960/JITET.V12I3.4457>
- Subandi, R., Yudhana, A., & Info Kata Kunci, A. (2023). Pre-Processing Pada Klasifikasi Citra Medis Pneumonia. *Decode: Jurnal Pendidikan Teknologi Informasi*, 4(1), 86–93. <https://doi.org/10.51454/DECODE.V4I1.198>
- Talebi, H., & Milanfar, P. (2021). Learning to Resize Images for Computer Vision Tasks. *Proceedings of the IEEE International Conference on Computer Vision*, 487–496. <https://doi.org/10.1109/ICCV48922.2021.00055>
- Triwijoyo, B. K., & Triwijoyo, B. K. (2019). Model Fast Tansfer Learning pada Jaringan Syaraf Tiruan Konvolusional untuk Klasifikasi Gender Berdasarkan Citra Wajah.

- MATRIK: Jurnal Manajemen, Teknik Informatika dan Rekayasa Komputer*, 18(2), 211–221. <https://doi.org/10.30812/matrik.v18i2.376>
- Verdy, & Hartati, E. (2024). Klasifikasi Penyakit Mata Menggunakan Convolutional Neural Network Model ResNet-50. *Jurnal Rekayasa Sistem Informasi dan Teknologi*, 1(3), 199–206. <https://doi.org/10.59407/JRSIT.V1I3.529>
- Zaid Munantri, N., Sofyan, H., & Yanu, M. (2019). Aplikasi Pengolahan Citra Digital Untuk Identifikasi Umur Pohon. *Telematika: Jurnal Telematika dan Teknologi Informasi*, 16(2), 97–104. <https://doi.org/10.31315/TELEMATIKA.V16I2.3183>

